

UE23CS352B - Object Oriented Analysis and Design

GRASP



Prof. Shridevi A Sawant
Dept. of Computer Science and Engineering

Acknowledgements:

Dr. J. MANNAR MANNAN

Dept. of Computer Science and Engineering

- **Introduction to GRASP**
- **Responsibility**
- **Types of Responsibility**
- **References**
- **GRASP Design Principles**

- Stands for **General Responsibility Assignment Software Patterns**
- GRASP - Consists of **guidelines for assigning responsibility to classes and objects in object-oriented design.**
- **GRASP with object-oriented programming classifies problems and their solutions together as patterns. With these problems and solutions well defined, they can be applied in other similar instances.**
- First published by **Craig Larman in 1977, with set of "nine fundamental principles in object design and responsibility"**
- **GRASP is occasionally coupled with other design patterns such as SOLID**

- **A contract / obligation that a class / module / component must accomplish**
- Responsibility can be:
 - * Accomplished by a single object
 - * A group of object collaboratively accomplish a responsibility.
- Identify the objects and responsibilities from the problem domain, and also identify how objects interact with each other. Helps us in deciding which responsibility should be assigned to which object/class.
- **Define blueprint for those objects** – i.e., class with methods implementing those responsibilities

- Design of software objects and larger scale components is in terms of **responsibilities, roles, and collaborations.** - RDD – Responsibility Driven Design

RDD – An abstraction of what they do

UML - A contract or obligation of a classifier

- Related to the obligations or behavior of an object in terms of its role
- **Assigned to classes of objects during object design.**

Example: I may declare that “a Sale is responsible for creating SalesLineItems”

- The **translation of responsibilities into classes and methods** is influenced by the **granularity of the responsibility**

Types of Responsibility

- **Doing:**

Doing something itself, such as creating an object or doing a calculation

Initiating action in other objects

Controlling and coordinating activities in other objects

- **Knowing:**

Knowing about private encapsulated data

Knowing about related objects

Knowing about things it can derive or calculate

Object Oriented Analysis and Design

GRASP Design principles

Different patterns/principles used are

- **Creator**
- **Information expert**
- **Low coupling**
- **Controller**
- **High cohesion**
- **Polymorphism**
- **Indirection**
- **Protected variations**
- **Pure fabrication**

**Design principle (Larman: “pattern”) –
An attempt at a methodical way to do
object design**

- Who creates an Object? Or who should create a new instance of some class?
- Assign the responsibility of creating instances of a class to the class that has the most knowledge about when and how to create them
- “Container” object creates “contained” objects.
- Decide who can be creator based on the object's association and their interaction
- **Example:** Who creates the square object in monopoly game?

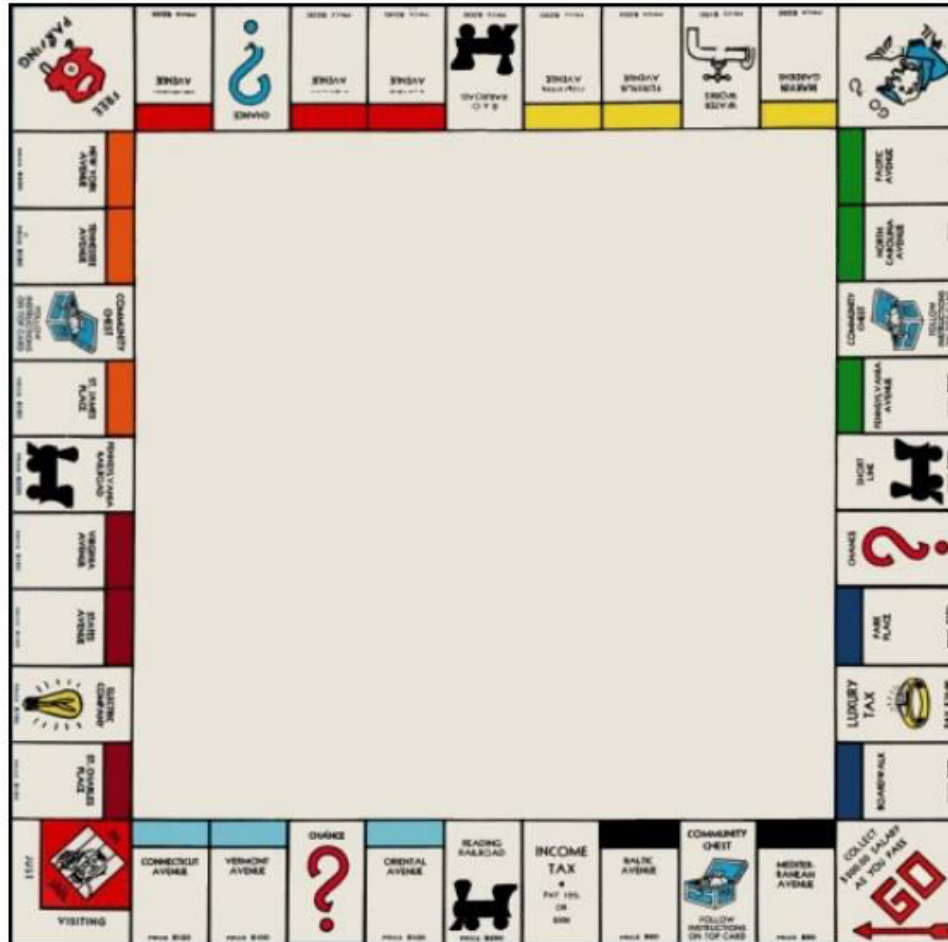
- **Identify Object Creation Scenarios:** Determine which classes are responsible for creating instances of other classes within the system.
- **Delegate Object Creation:** Assign responsibility for creating objects to classes that are best suited for the task.
- **Encapsulate Initialization Logic:** Ensure that the classes responsible for creating objects encapsulate the initialization logic within them.
- **Promote Loose Coupling:** Aim to minimize dependencies between classes by decoupling object creation logic from the classes that use the created objects.
- **Consider Dependency Inversion:** When designing classes that create objects, consider using dependency inversion principles to invert dependencies and promote abstraction.

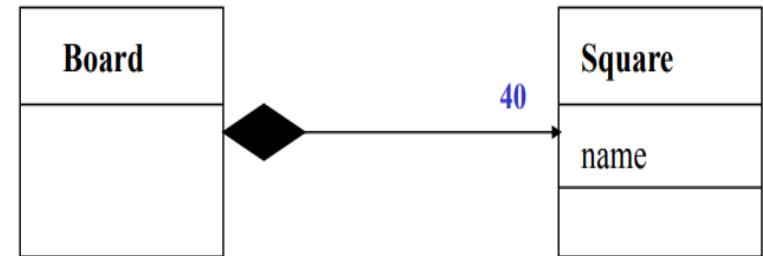
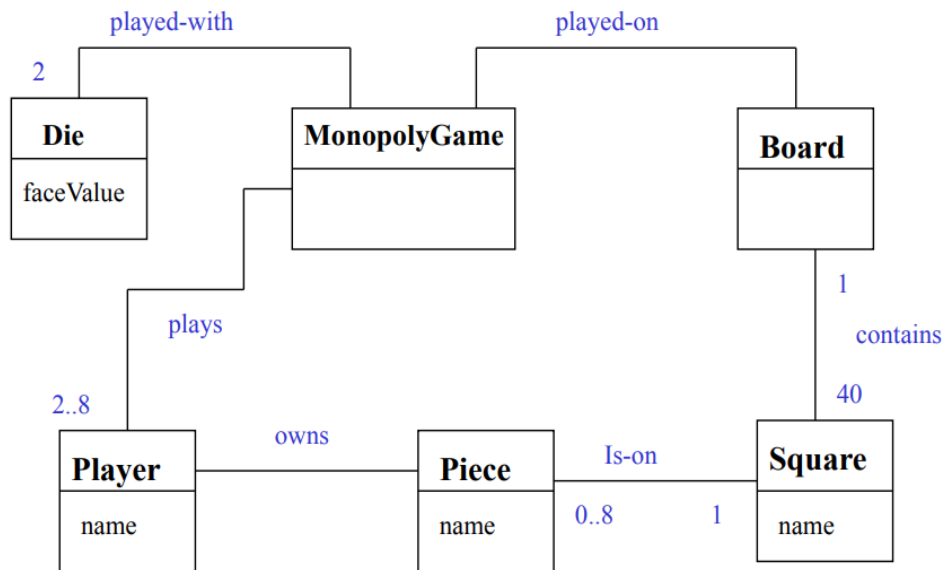
```
public class OrderService {  
  
    private final OrderRepository orderRepository;  
  
    public OrderService(OrderRepository orderRepository) {  
        this.orderRepository = orderRepository;  
    }  
  
    public void createOrder(Customer customer, List<Product> products) {  
        // Create a new order object  
        Order order = new Order(customer, products);  
  
        // Save the order to the repository  
        orderRepository.saveOrder(order);  
    }  
}
```

- In this example, the **OrderService** class is responsible for creating instances of **Order** objects.
- It encapsulates the initialization logic for creating orders and delegates the task of persisting orders to an **OrderRepository** class.
- By centralizing object creation logic within the **OrderService** class, we promote encapsulation and loose coupling in the system.

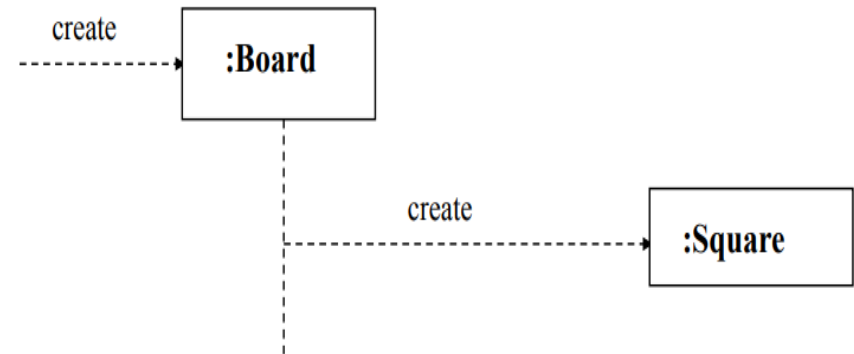
Object Oriented Analysis and Design

Monopoly game





Applying the Creator pattern in Static Model



Dynamic Model – illustrates Creator Pattern

Name:	Creator
Problem:	Who creates an A?
Solution: (this can be viewed as advice)	Assign class B the responsibility to create an instance of class A if one of these is true (the more the better): ■ B “contains” or compositely aggregates A. ■ B records A. ■ B closely uses A. ■ B has the initializing data for A.

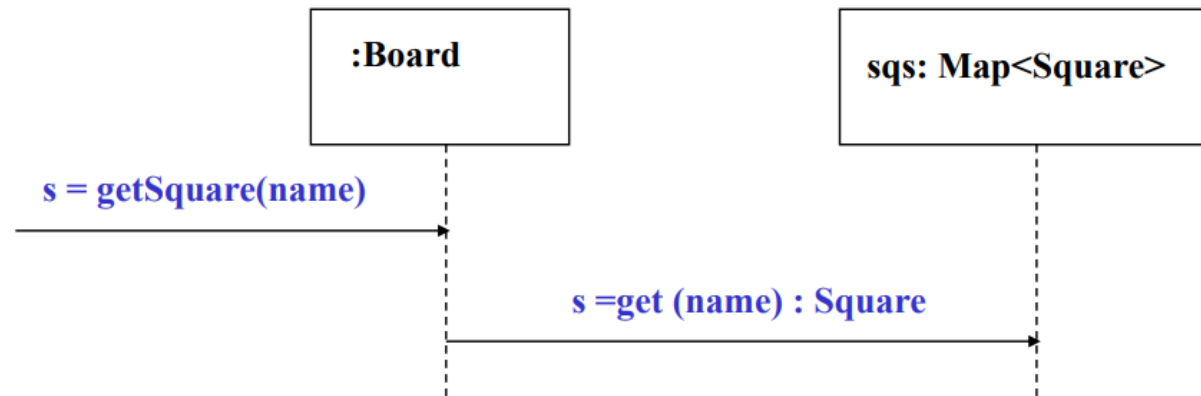
- Start with domain model.
- Since a Sale contains SalesLineItem objects, it should be responsible according to the Creator pattern
- Note: There is a related design pattern called **Factory pattern** for more complex creation situations

- Problem: What is a basic principle by which to assign responsibilities to objects?
- Solution: Assign a responsibility to the class that has the information needed to fulfill it.
- **“objects do things related to the information they have”**
- Information necessary may be spread across several classes => **objects interact via messages**
- **Definition:**
 - Name: Information Expert
 - Problem: What is the basic principle by which to assign responsibilities to object
 - Solution: Assign a responsibility to the class that has the information needed to respond to it.
- Just because an object has information necessary doesn't mean it will have responsibility for action related to the information

- They have all the information needed to perform operations, or in some cases they collaborate with others to fulfill their responsibilities.
- **Example:** (from POS system) Who should be responsible for knowing the grand total of a sale?
- Start with domain model and look for associations with Sale
- What information is necessary to calculate grand total and which objects have the information?

Design Class	Responsibility
Sale	knows sale total
SalesLineItem	knows line-item subtotal
ProductDescription	knows product price

- The player Marker needs to find the square to which it is to move and the options pertaining to that square.
- The Board aggregates all of the Squares, so the Board has the Information needed to fulfill this responsibility.



Make board information expert

```
public class ShoppingCart {  
  
    private List<Item> items = new ArrayList<>();  
  
    public void addItem(Item item) {  
        items.add(item);  
    }  
  
    public double calculateTotalPrice() {  
        double totalPrice = 0;  
  
        for (Item item : items) {  
            totalPrice += item.getPrice();  
        }  
  
        return totalPrice;  
    }  
}
```

```
public class Item {  
  
    private String name;  
    private double price;  
  
    public Item(String name, double price) {  
        this.name = name;  
        this.price = price;  
    }  
  
    public double getPrice() {  
        return price;  
    }  
}
```

- In this example, the “**Information Expert**” principle is applied by assigning responsibility for calculating the total price of items in the shopping cart to the **ShoppingCart** class.
- Since the **ShoppingCart** class has access to the list of items and their prices, it is the most suitable class to perform this calculation.
- The **Item** class, on the other hand, encapsulates information about individual items, such as their name and price.
- By delegating responsibilities based on expertise and information, we promote high cohesion and encapsulation in the design.

- **Coupling** – Coupling describes the degree of dependency between classes or modules. It indicates how closely connected different classes or modules are, and how much they rely on each other. There are two main types of coupling
- **Tight Coupling:** High dependency between classes. Changes in one class often require changes in the coupled class. Tight coupling occurs when a class is highly dependent on the specifics of another class. This often happens when one class creates an instance of another class and directly accesses its methods and fields.
- **Loose Coupling:** Low dependency between classes. Classes interact through well-defined interfaces, reducing the impact of changes. Loose coupling occurs when classes are less dependent on the specifics of other classes. This can be achieved by using interfaces, abstract classes, or dependency injection, allowing classes to interact without knowing the implementation details of each other.

- **Definition:**

Name: Low Coupling

Problem: How to support low dependency, low change impact, increased reuse?

Solution: Assign a responsibility so coupling is low.

- **Minimizes the dependency** hence making system **maintainable, efficient and code reusable**
- **Example: Inheritance, Composition, aggregation, association**

Low coupling is an evaluative pattern that dictates how to assign responsibilities to support

- lower dependency between the classes
- change in one class having a lower impact on other classes
- higher reuse potential

- A class with high coupling leads to problems:
 - Changes in related classes force local changes
 - Harder to understand in isolation
 - Harder to reuse
- Low Coupling – How can we reduce the impact of change in depended elements on dependent elements? **Assign responsibilities so that coupling remains low.**
- Two **elements are coupled**, if – One element has aggregation/composition association with another element. – One element implements/extends other element

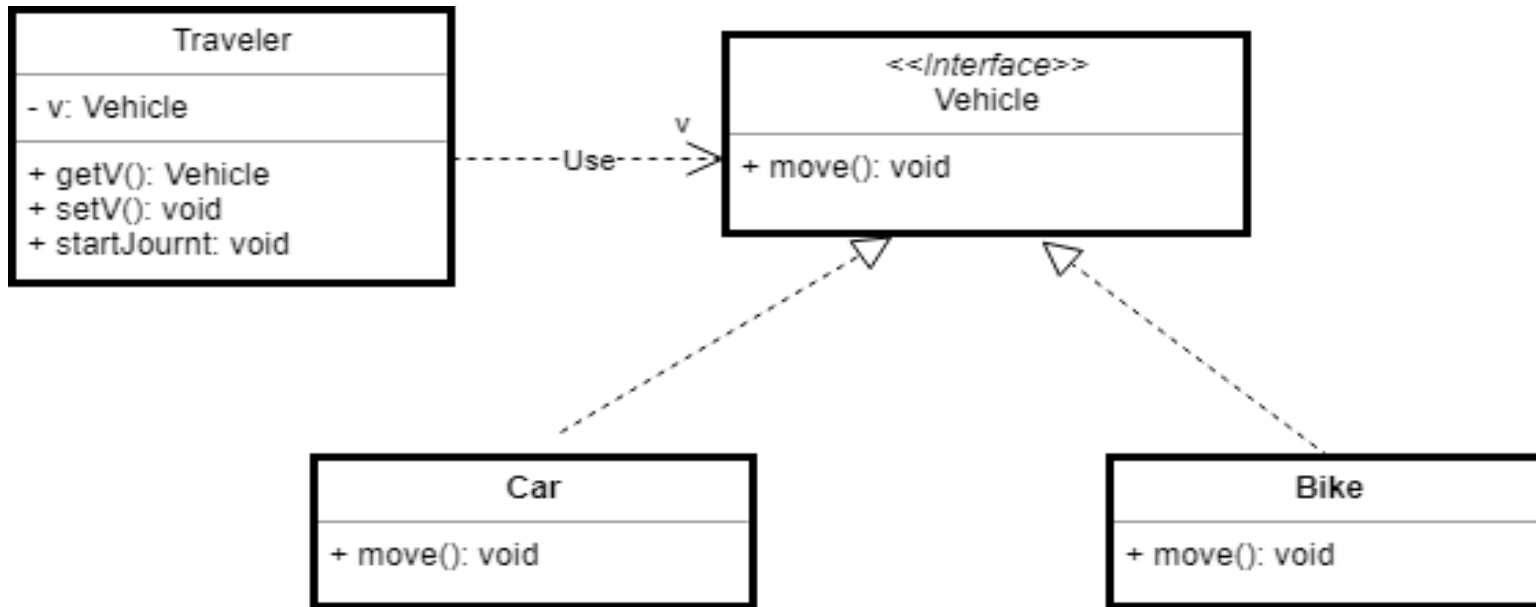
Assign a responsibility so that coupling remains low.

A class, for example, with high (or strong) coupling relies on many other classes. Such classes may be undesirable; some suffer from the following problems:

- Forced local changes because of changes in related classes.
- Harder to understand in isolation.
- Harder to reuse because its use requires the additional presence of the classes on which it is dependent.

Good to know below are some key points about low coupling

- Low dependencies between “artifacts” (classes, modules, components).
- There shouldn’t be too much of dependency between the modules, even if there is a dependency it should be via the interfaces and should be minimal.
- Avoid tight-coupling for collaboration between two classes (if one class wants to call the logic of a second class, then first class needs an object of second class it means the first class creates an object of the second class).
- Strive for loosely coupled design between objects that interact.
- Inversion Of Control (IoC) / Dependency Injection (DI) - With DI objects are given their dependencies at creation time by some third party (i.e. Java EE CDI, Spring DI...) that coordinates each object in the system. Objects aren’t expected to create or obtain their dependencies—dependencies are injected into the objects that need them. The key benefit of DI—loose coupling.



```
// Vehicle interface to allow loose coupling
implementation
interface Vehicle {
    void move();
}
```

```
// Car class implements Vehicle interface
class Car implements Vehicle {
    @Override
    public void move() {
        System.out.println("Car is moving");
    }
}
```

```
// Bike class implements Vehicle interface
class Bike implements Vehicle {
    @Override
    public void move() {
        System.out.println("Bike is moving");
    }
}
```

Now create a Traveler class which holds the reference to the Vehicle interface.

```
class Traveler {
    private Vehicle v;

    public Vehicle getV() {
        return v;
    }
    public void setV(Vehicle v) {
        this.v = v;
    }
    public void startJourney() {
        v.move();
    }
}
```

Test class for loose coupling example - Traveler is an example of loose coupling

```
public static void main(String[] args) {  
    Traveler traveler = new Traveler();  
    traveler.setV(new Car()); // Inject Car dependency  
    traveler.startJourney(); // start journey by Car  
    traveler.setV(new Bike()); // Inject Bike dependency  
    traveler.startJourney(); // Start journey by Bike  
}
```

- **Example 2** (from POS system): Consider Payment, Register, Sale
- To create a Payment and associate it with a Sale, who is responsible?
- Register “records” a Payment. So, **Register is the creator**
- Register creates Payment p then sends p to a Sale => **coupling of Register class to Payment class**

- **Alternate approach:** Register requests Sale to create the Payment

Note:

- In both cases, Sale needs to know about a Payment
- However, a Register needs to know about a Payment in first but not in second
- **Second approach has lower coupling**

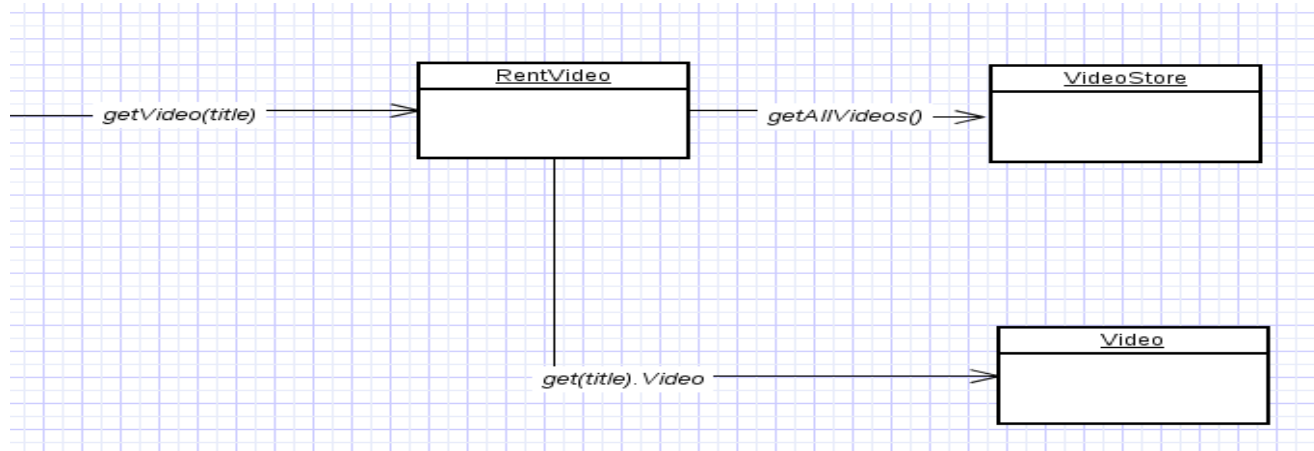
- Low coupling is an evaluative principle, i.e., keep it in mind when evaluating designs
- Examples of coupling in Java (think “dependencies”):
 - TypeX has an attribute of TypeY
 - TypeX calls on services of a TypeY object
 - TypeX has a method that references an instance of TypeY
 - TypeX is a subclass of TypeY
 - TypeY is an interface and TypeX implements the interface

Note: subclassing => high coupling

Object Oriented Analysis and Design

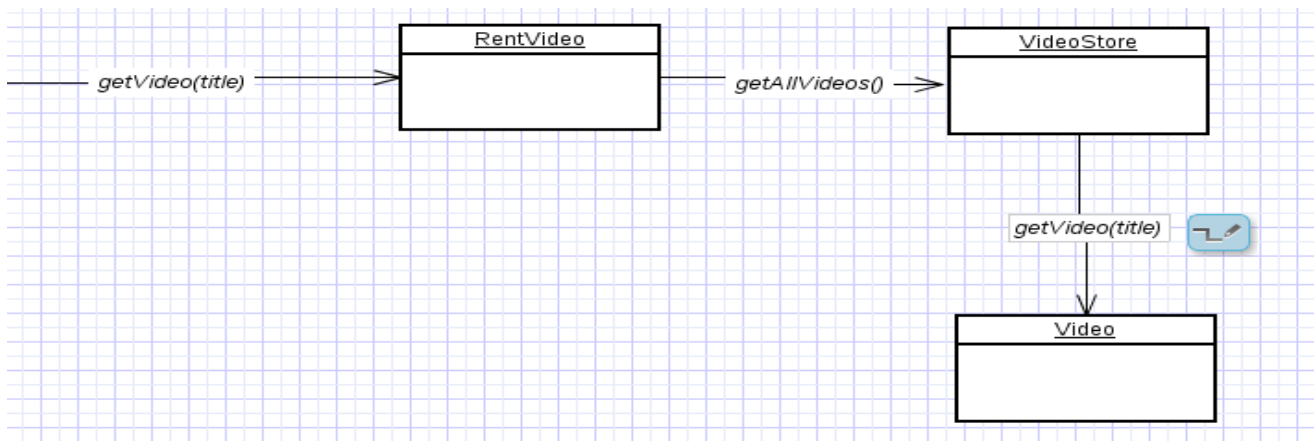
Example: Low Coupling

High Coupling

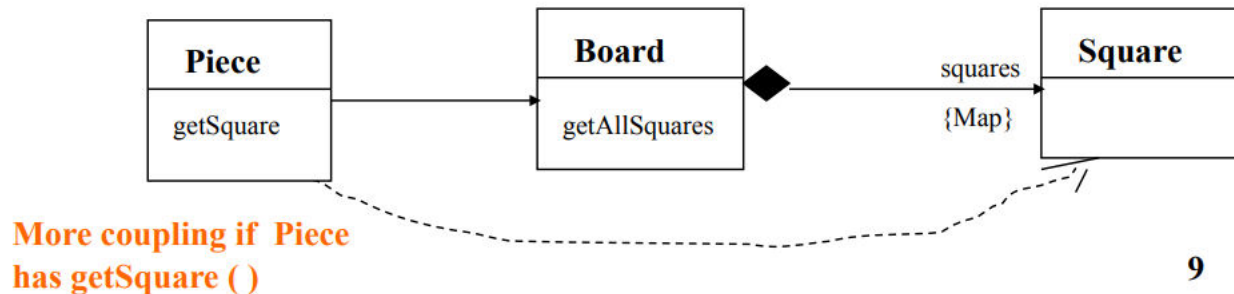
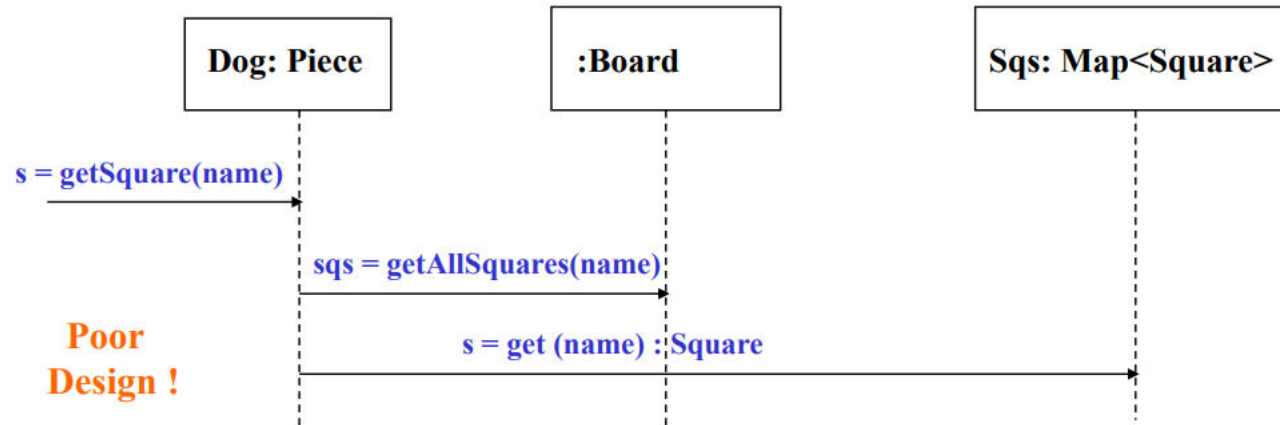


here class `RentVideo` knows about both `VideoStore` and `Video` objects. `Rent` is depending on both the classes.

Low Coupling



`VideoStore` and `Video` class are coupled, and `Rent` is coupled with `VideoStore`. **Thus, providing low coupling.**



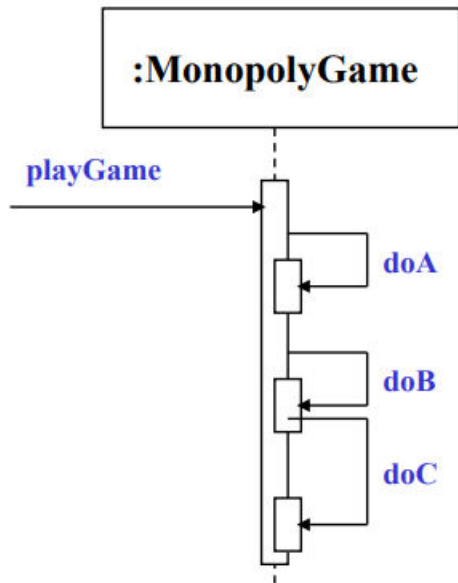
- **Cohesion** - A measure of how strongly related and focused the responsibilities of an element (class, subsystem, etc.)
- **Definition:**

Name: High Cohesion

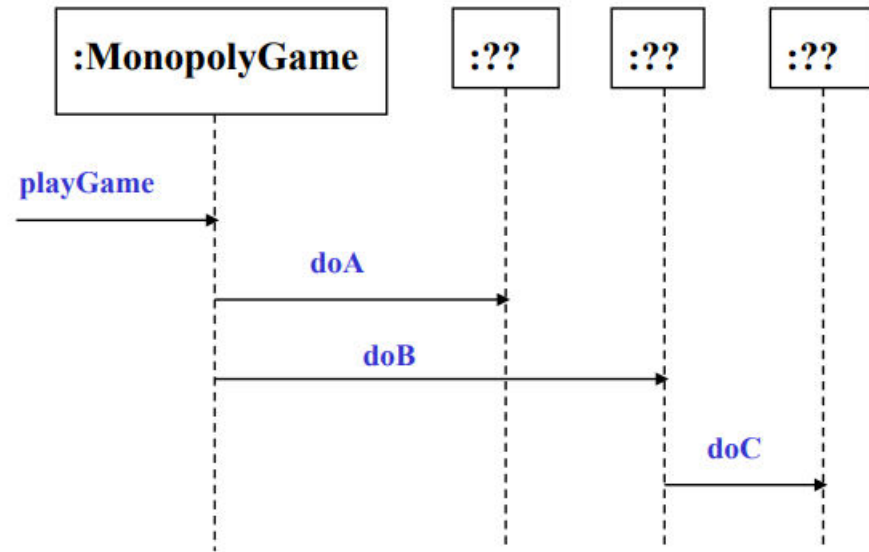
Problem: How to keep complexity manageable, understandable and support low coupling?

Solution: Assign the responsibility so that cohesion remains high.

- Problems from low cohesion (does many unrelated things or does too much work):
 - Hard to understand/comprehend
 - Hard to reuse
 - Hard to maintain
 - Brittle – easily affected by change



Poor (low) Cohesion in the MonopolyGame object



Better Design

Object Oriented Analysis and Design

MCQ

1. GRASP stands for:

- A) General Responsibility Assignment Software Patterns
- B) General Rules for Application Software Programming
- C) Generic Reusable Architecture System Principles
- D) Global Responsibility Allocation Software Process

2. GRASP mainly provides guidelines for:

- A) Database normalization
- B) Assigning responsibilities to classes and objects
- C) Writing algorithms
- D) Network communication

Object Oriented Analysis and Design

MCQ

3. GRASP contains how many fundamental principles of object design?

- A) Five
- B) Seven
- C) Nine
- D) Twelve

4. In object-oriented design, responsibility refers to:

- A) Data storage only
- B) Database queries
- C) Compilation rules
- D) A contract or obligation of a class or component

Object Oriented Analysis and Design

MCQ

5. Loose coupling means:

- A) High dependency between classes
- B) No interaction between classes
- C) Low dependency between classes
- D) Only inheritance relationships

6. Cohesion measures:

- A) Database connections
- B) Relationship between modules
- C) How strongly related responsibilities of a class are
- D) Execution speed

Object Oriented Analysis and Design

MCQ SOLUTION

- 1.A
- 2.B
- 3.C
- 4.D
- 5.C
- 6.C

- [Source Code Examples](#)
- [Ad-hoc, Inclusion, Parametric & Coercion Polymorphisms – GeeksforGeeks](#)
- [Larman - Monopoly Game.pdf - THE MONOPOLY GAME SYSTEM • The software version of the game will run as a simulation • One person will start the game and | Course Hero](#)
- <http://www.kamilgrzybek.com/design/grasp-explained/>



THANK YOU

Prof. Shridevi A Sawant
Computer Science and Engineering



UE23CS352B - Object Oriented Analysis and Design

Prof. Shridevi A. Sawant
Dept. of Computer Science and Engineering

Acknowledgements:

Dr. J. MANNAR MANNAN

Dept. of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

- Deals with **how to delegate the request from the UI layer objects to domain layer objects.**
- Helps in **minimizing the dependency** between GUI components and the system operation classes
- When a request comes from UI layer object, Controller as a pattern helps us in determining which is the first object that should receive the message from the UI layer objects. –

Controller Object

- Delegates the work to other class and coordinates all activities.
- **Benefits** - Can reuse the controller class. Can use to maintain the state of the use case. Can control the sequence of the activities

- **Definition**

Problem: Who should be responsible for handling a system event? (Or, what object receives and coordinates a system operation?)

Solution: Assign the responsibility for receiving and/or handling a system event to one of following choices:

Object that represents overall system, device or subsystem (façade controller)

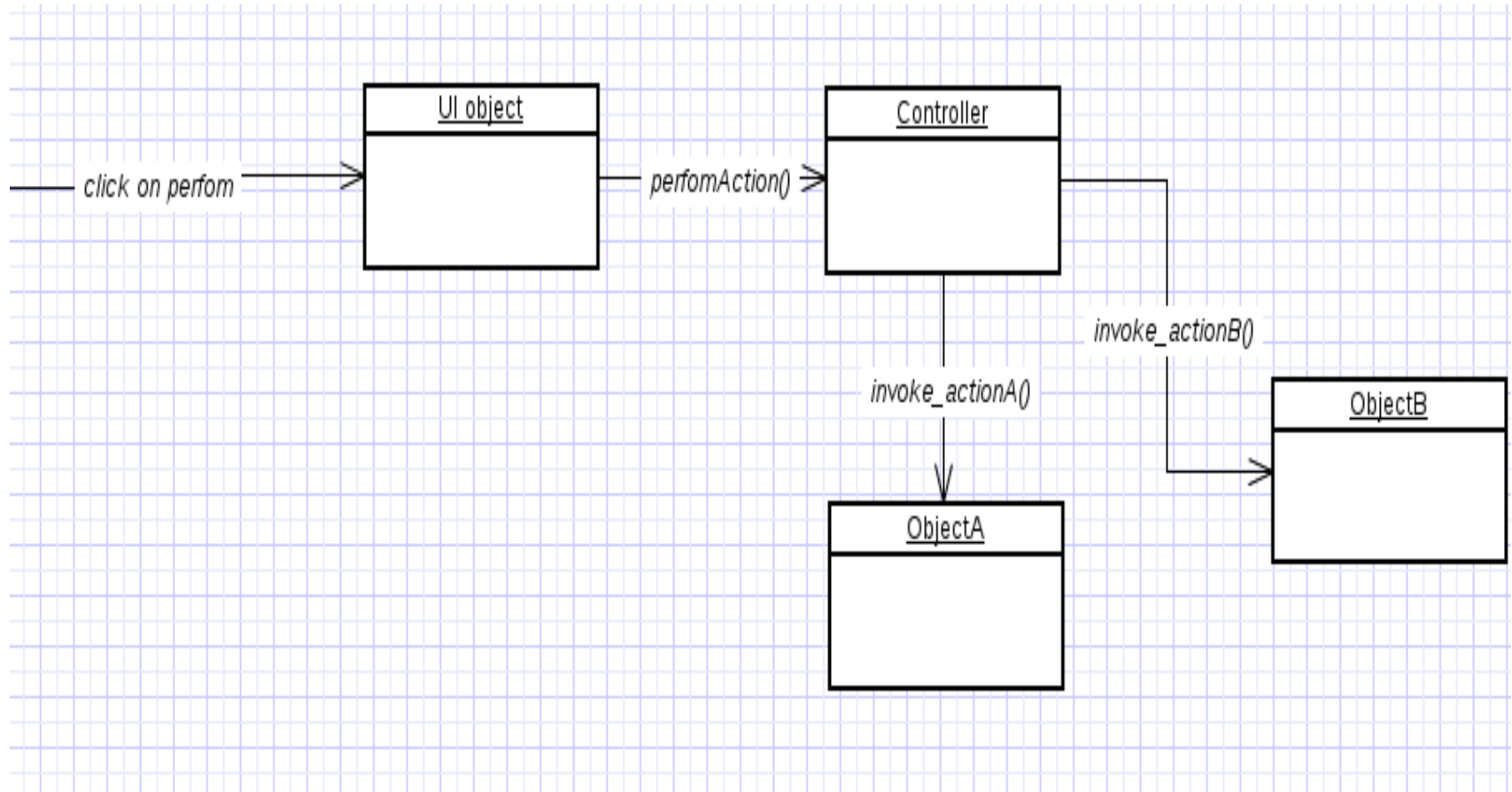
Object that represents a use case scenario within which the system event occurs (a <UseCase>Handler)

- **Input system event** – Event generated by an external actor associated with a system operation

- Normally controller coordinates activity but delegates work to other objects rather than doing work itself
- **Façade Controller** : A class that represents the overall system, device subsystems
- **Use case Controller**: A class that represent a use case, whereby performs handling particular system operations Often uses same controller class for all system events of one-use case so that one can maintain state information, e.g., events must occur in a certain order
- **Bloated Controller**
 - Single class receiving all system events and there are many of them
 - Controller performs many tasks rather than delegating them
 - Controller has many attributes and maintains significant information about system which should have been distributed among other objects

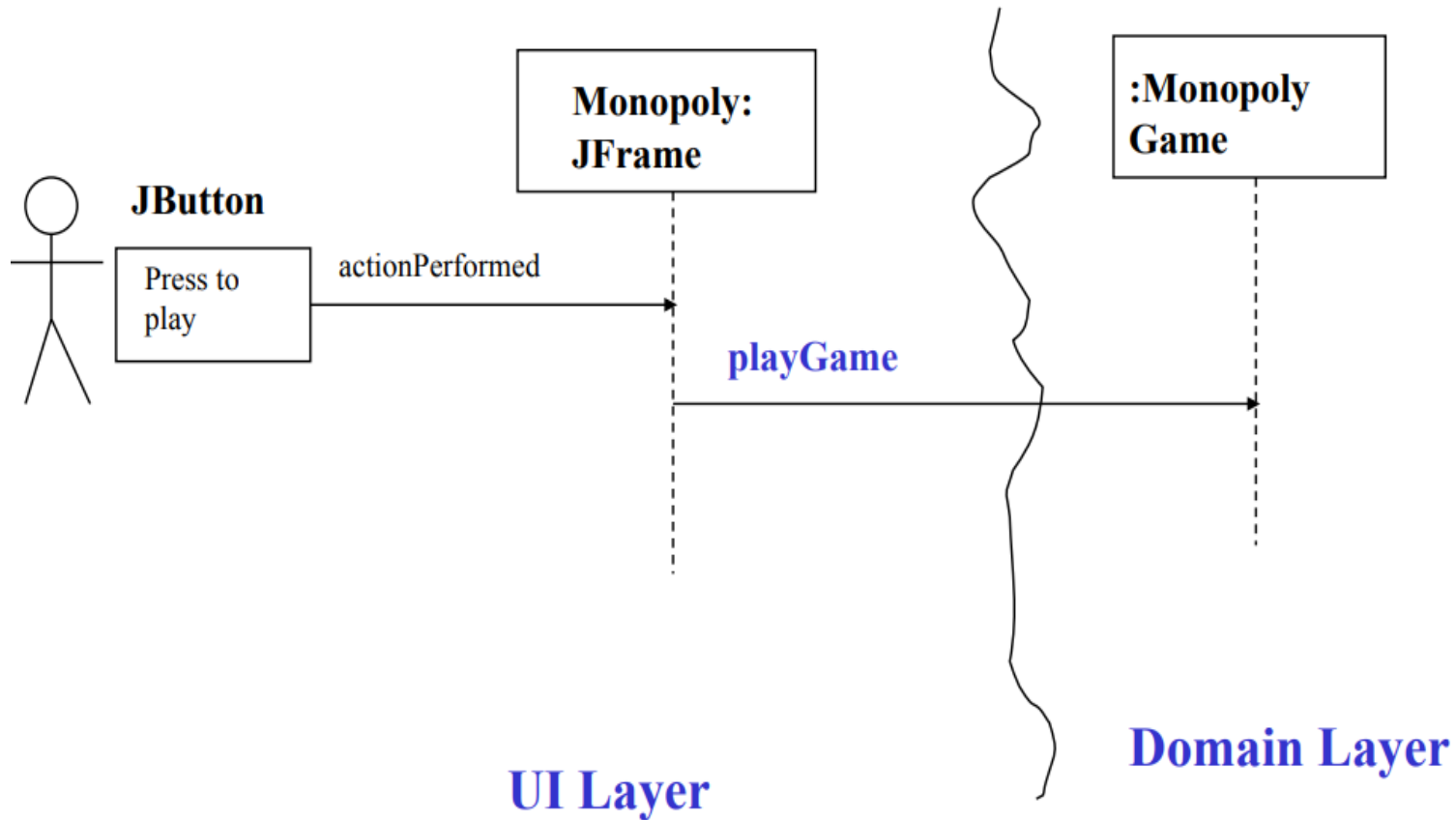
Object Oriented Analysis and Design

Example: Controller



Object Oriented Analysis and Design

Example: Monopoly example



Here's how to implement the “**Controller**” component in Java:

Identify System Boundaries: Determine the boundaries of the system, including user interfaces, external systems, or other modules that interact with the core functionality of the application. These boundaries represent entry points for system events and inputs.

Define Controller Classes: Create controller classes to handle incoming requests or events from system boundaries. Each controller class is responsible for a specific set of related actions or interactions within the system.

Separate Concerns: Ensure that controller classes focus on coordinating activities and delegating work to other parts of the system, rather than performing business logic or data manipulation directly.

Handle User Inputs: Implement methods or event handlers within controller classes to process user inputs from user interfaces or external systems.

Coordinate Interactions: Use controller classes to coordinate interactions between different components or modules within the system.

Promote Loose Coupling: Ensure that controller classes interact with other parts of the system through well-defined interfaces and abstractions, rather than direct dependencies.

Handle Exceptions: Implement error handling and exception management within controller classes to gracefully handle unexpected scenarios and communicate errors to users or other parts of the system.

- Deals with How to act different depending on object's class
- How to handle related but varying elements based on type?
- Guides us in deciding which object is responsible for handling those varying elements.

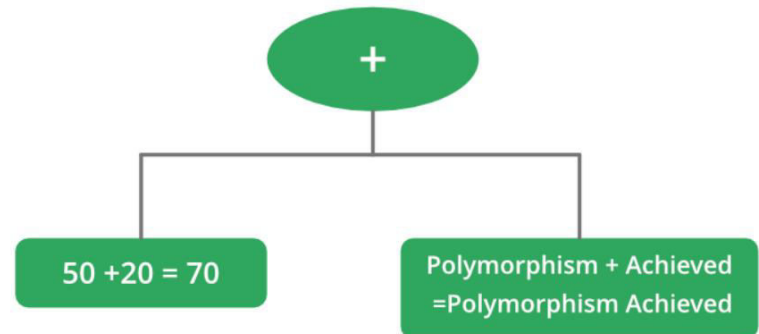
Types:

- Adhoc – Overloading
- Parametric – Early binding
- Inclusion – Sub typing
- Coercion - Casting

Ad-hoc – Overloading

Ad-hoc Polymorphism, also called as Overloading Ad-hoc Polymorphism allows functions having **same name to act differently for different types**. For example: The + operator adds two integers and concatenates two strings.

Ad-hoc Polymorphism



Parametric – Early binding

Parametric Polymorphism, also called as Early Binding Parametric Polymorphism opens a way to use the same piece of code for different types. Parametric polymorphism is implemented using Generics in Java.

Inclusion Polymorphism, also called as Subtyping. Inclusion Polymorphism is the ability to use derived classes through base class references.

- It is also known as **Run-time polymorphism**. In Java, runtime polymorphism is achieved through dynamic method dispatch. The method to be invoked is determined at runtime based on the actual object type, not the reference type.

Coercion Polymorphism, also called as **Casting Coersion**. **Polymorphism occurs when an object or primitive is cast into some other type.** It could be either Implicit or Explicit. Implicit casting happens as a responsibility of Compiler itself.

An object of one type is implicitly converted to another type to match an expected interface or function parameter.

In Java, coercion polymorphism occurs when the compiler automatically converts one type to another compatible type (widening), or when a developer explicitly casts a value to another type.

```
int x = 10;  
double y = x; // int → double (implicit widening)
```

```
// explicit cast - converting from a larger type to a smaller type  
double d = 9.7;  
int i = (int) d;
```

- **Definition:**

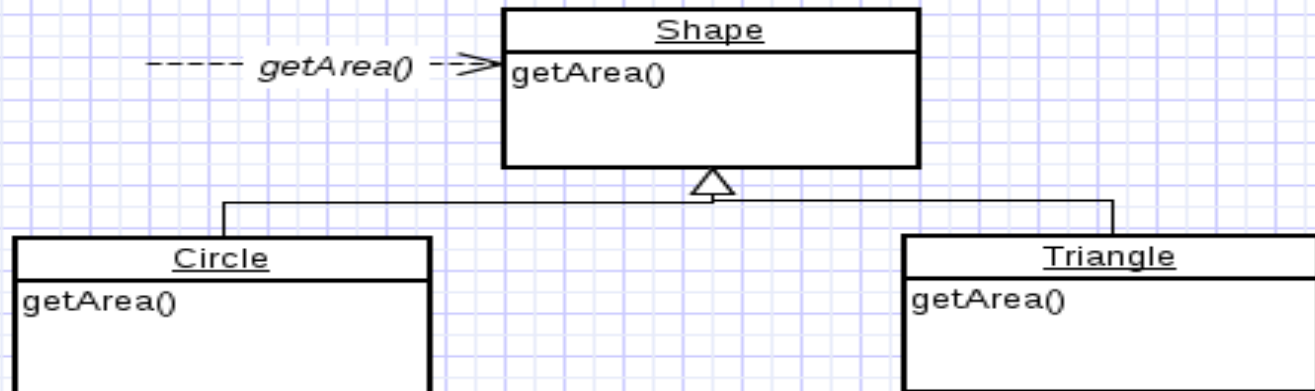
Name: Polymorphism

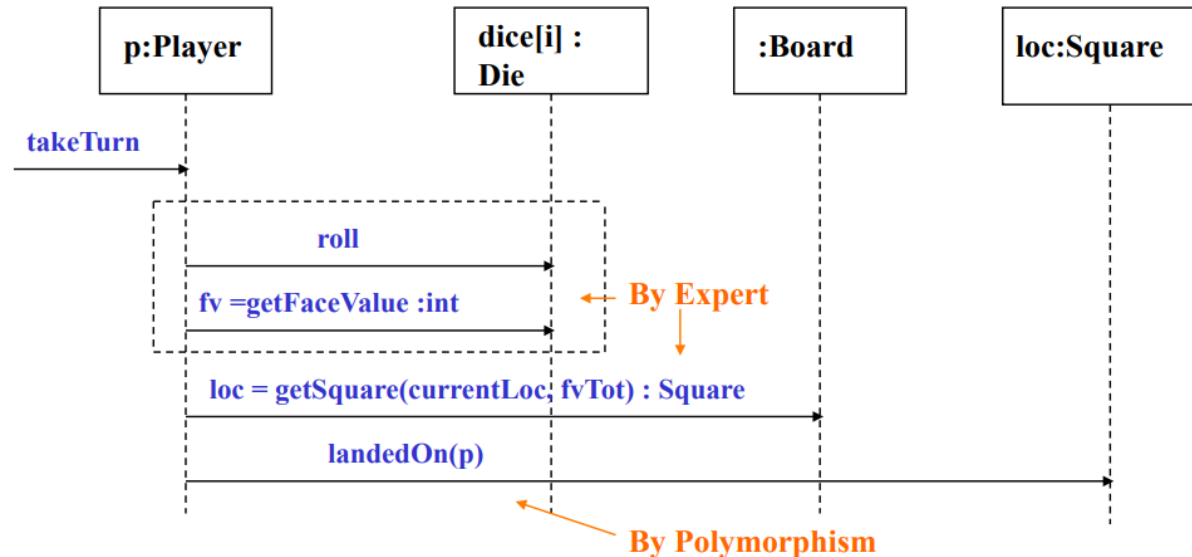
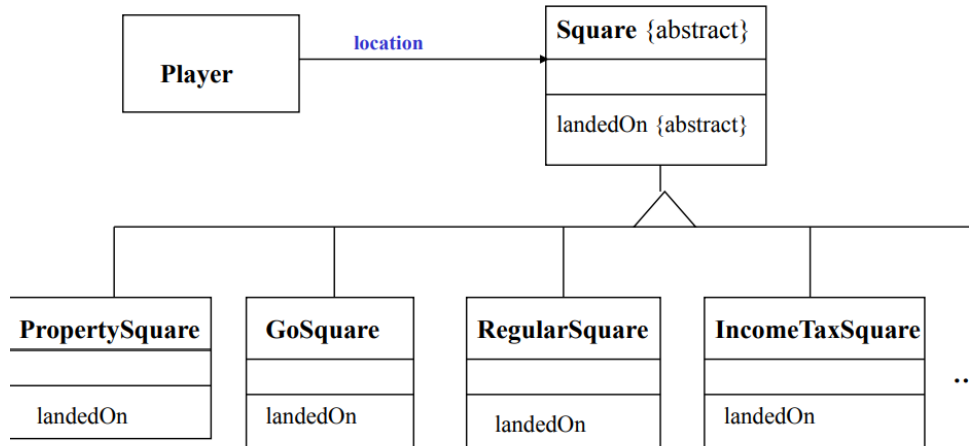
Problem: How to handle alternatives based on type.

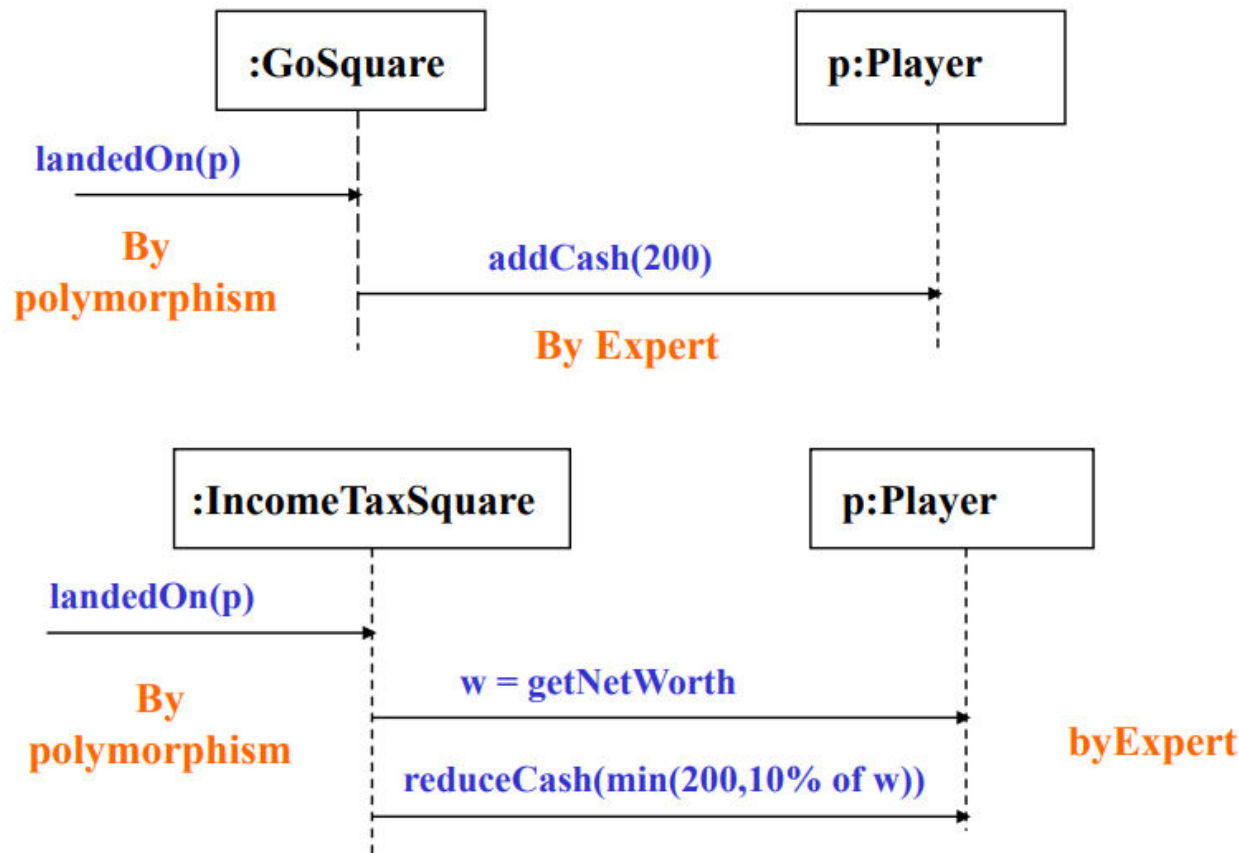
Pluggable software components -- how can you replace one server component with another without affecting the client?

Solution: When related alternatives or behaviors vary by type (class), assign responsibility using polymorphic operations – to the types for which the behavior varies. In this context, polymorphism means giving the same name to similar or related services

- The `getArea()` varies by the type of shape, so we assign that responsibility to the subclasses
- By sending message to the Shape object, a call will be made to the corresponding sub class object – Circle or Triangle







Polymorphic cases

Object Oriented Analysis and Design

MCQ

1. The Controller pattern is mainly responsible for:

- A) Performing database operations
- B) Delegating requests from UI to domain objects
- C) Rendering user interfaces
- D) Managing memory

2. The Controller pattern helps minimize dependency between:

- A) Database and server
- B) Packages and modules
- C) Methods and variables
- D) GUI components and system operation classes

3. In GRASP, a controller usually:

- A) Performs all business logic
- B) Stores all system data
- C) Coordinates activities and delegates work
- D) Replaces domain classes

Object Oriented Analysis and Design

MCQ

4. Polymorphism addresses the problem of:

- A) Handling alternatives based on type
- B) Memory Allocation
- C) File storage
- D) Data serialization

5. Giving the same name to related services is known as:

- A) Inheritance
- B) Encapsulation
- C) Polymorphism
- D) Aggregation

Object Oriented Analysis and Design

MCQ SOLUTION

- 1.B
- 2.D
- 3.C
- 4.A
- 5.C

- [GRASP: Guiding Object-Oriented Design in Java | by Alex Klimenko | Medium](#)
- [Source Code Examples](#)
- [Ad-hoc, Inclusion, Parametric & Coercion Polymorphisms – GeeksforGeeks](#)
- [Larman - Monopoly Game.pdf - THE MONOPOLY GAME SYSTEM • The software version of the game will run as a simulation • One person will start the game and | Course Hero](#)
- <http://www.kamilgrzybek.com/design/grasp-explained/>



THANK YOU

Prof. Shridevi A. Sawant
Computer Science and Engineering



UE23CS352B - Object Oriented Analysis and Design

Prof. Shridevi A Sawant
Dept. of Computer Science and Engineering

Acknowledgements:

Dr. J. MANNAR MANNAN

Dept. of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

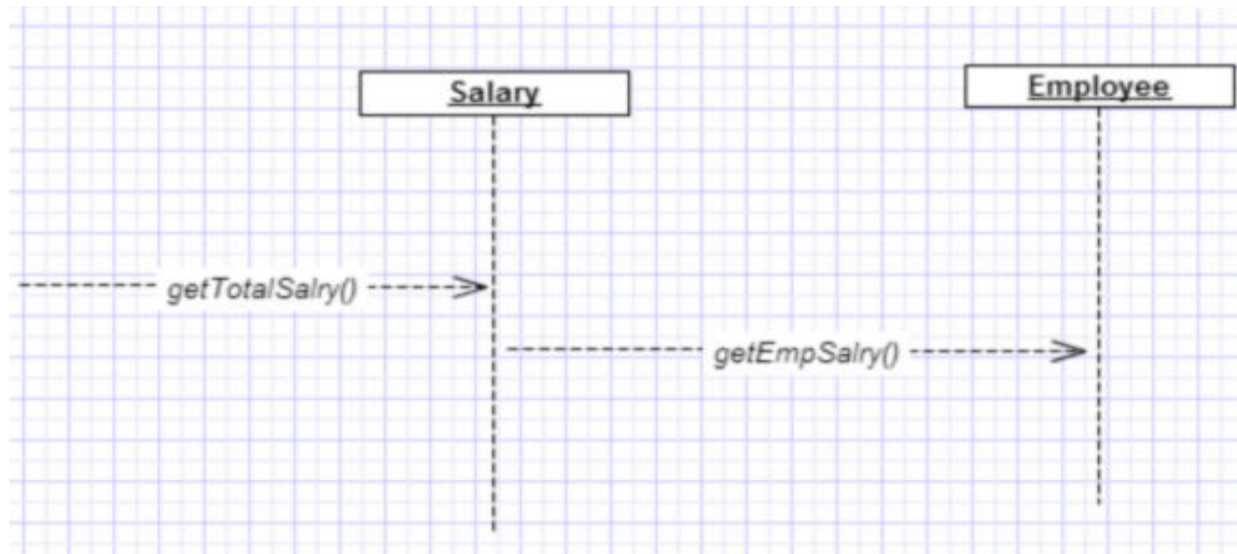
- **Definition:**

Name: Indirection

Problem: How to assign responsibilities in order to avoid direct coupling between two components and keep ability for reuse.

Solution: Assign responsibility to intermediate class for providing linking between objects not linking directly

- Related Design patterns: Adapter, Bridge, Mediator.



Salary serves as the intermediary between the caller and Employee to obtain salary details.

- **Identify Mediating Responsibilities:** Determine which responsibilities involve mediating interactions between different parts of the system or coordinating activities across multiple components. These responsibilities may include routing requests, translating between interfaces, or managing dependencies.
- **Introduce Intermediate Objects:** Create intermediary classes or components to handle mediating responsibilities. These intermediate objects act as indirection points, decoupling the classes or modules that interact with each other.

- **Encapsulate Mediating Logic:** Ensure that the intermediate objects encapsulate the logic for mediating interactions or delegating tasks. This helps to maintain encapsulation and prevents the proliferation of mediating logic throughout the codebase.
- **Promote Loose Coupling:** By introducing indirection, classes or modules become less dependent on each other's concrete implementations, leading to greater flexibility and easier maintenance. Changes to one part of the system are less likely to impact other parts, as long as the indirection layer remains stable.
- **Facilitate Evolution and Extension:** Indirection facilitates the evolution and extension of the system by providing a flexible point of interaction between components.

- **Definition:**

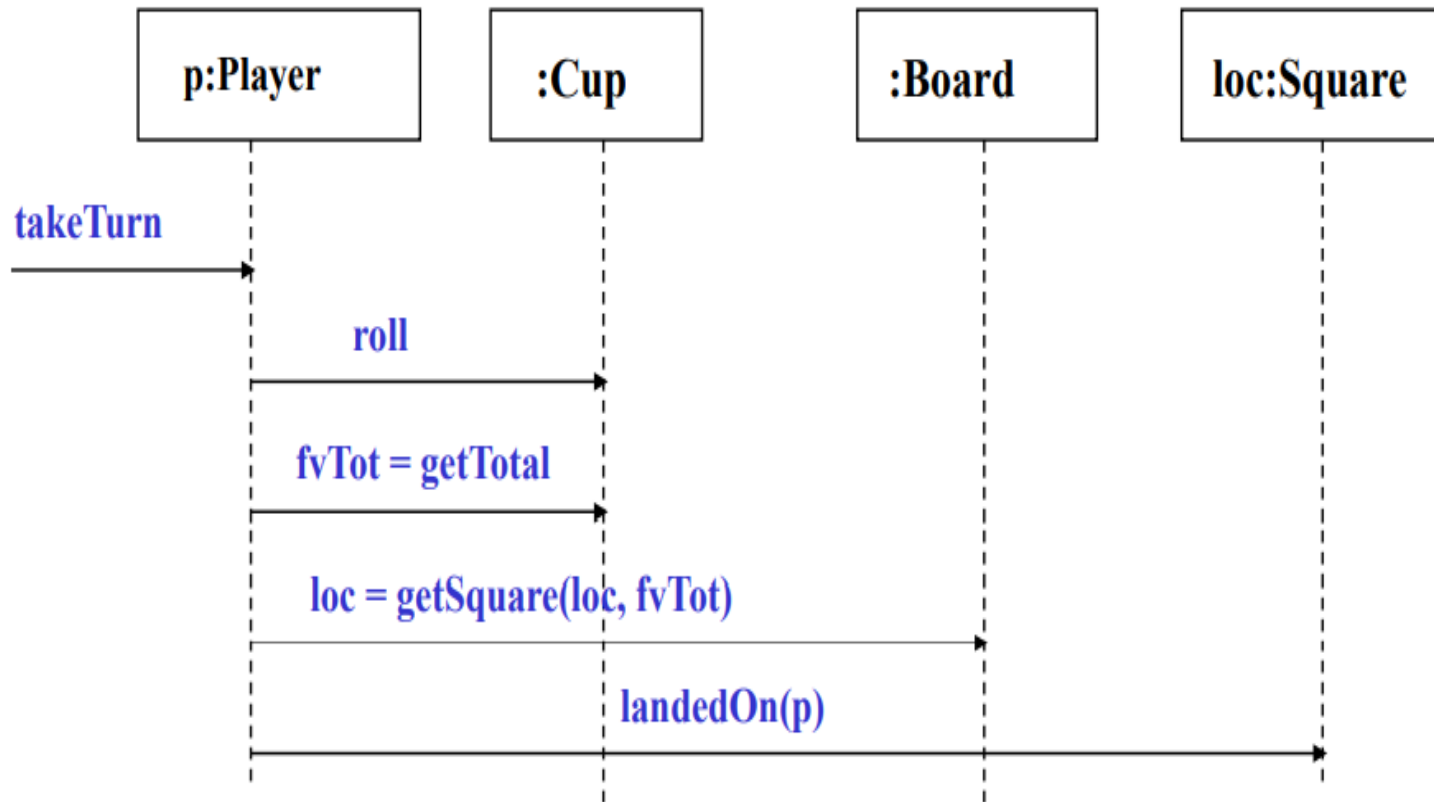
Name: Pure Fabrication

Problem: What object should have responsibility when you do not want to violate High Cohesion and Low Coupling, or other goals, but solutions offered by Expert (for example) are not appropriate? Sometimes assigning responsibilities only to domain layer software classes leads to problems like poor cohesion or coupling, or low reuse potential.

Solution: Assign a highly cohesive set of responsibilities to an artificial or convenience class that does not represent a domain concept

- Here's how to understand “Pure Fabrication” in Java:
- **Identify Complex Responsibilities:** In Java application design, there may be certain responsibilities or tasks that do not naturally fit within the domain of any existing classes.
- **Introduce Fabrication Classes:** Once complex responsibilities are identified, the “Pure Fabrication” principle suggests introducing additional classes or objects solely for the purpose of encapsulating these responsibilities.
- **Encapsulation of Non-Natural Responsibilities:** The pure fabrication classes encapsulate the complex or non-natural responsibilities, isolating them from the core domain objects.

- **Promote Modularity and Reusability:** By encapsulating complex responsibilities in pure fabrication classes, Java developers can promote modularity and reusability in their codebase.
- **Examples of Pure Fabrication:** In Java, pure fabrication classes may include utility classes, service classes, or helper classes that perform generic functions such as data validation, formatting, or error handling.



Use a Cup to hold the dice, roll them, and know their total. It can be reused in many different applications where dice are involved.

- **Definition:**

Name: Protected Variation

Problem: How to design objects, subsystems, and systems so that the variations or instability in these elements does not have an undesirable impact on other elements?.

Solution: Identify points of predicted variation or instability; assign responsibilities to create a stable interface around them.

- Identify points of likely change (variation points) and create a stable interface around them so that changes don't affect the rest of the system.
- Example:- Open-Closed Principle similar to CV/PV pattern
- Protected variation pattern applying for both **variation and Evolution points**

Protected variation pattern applying for both **variation** and **Evolution point**.

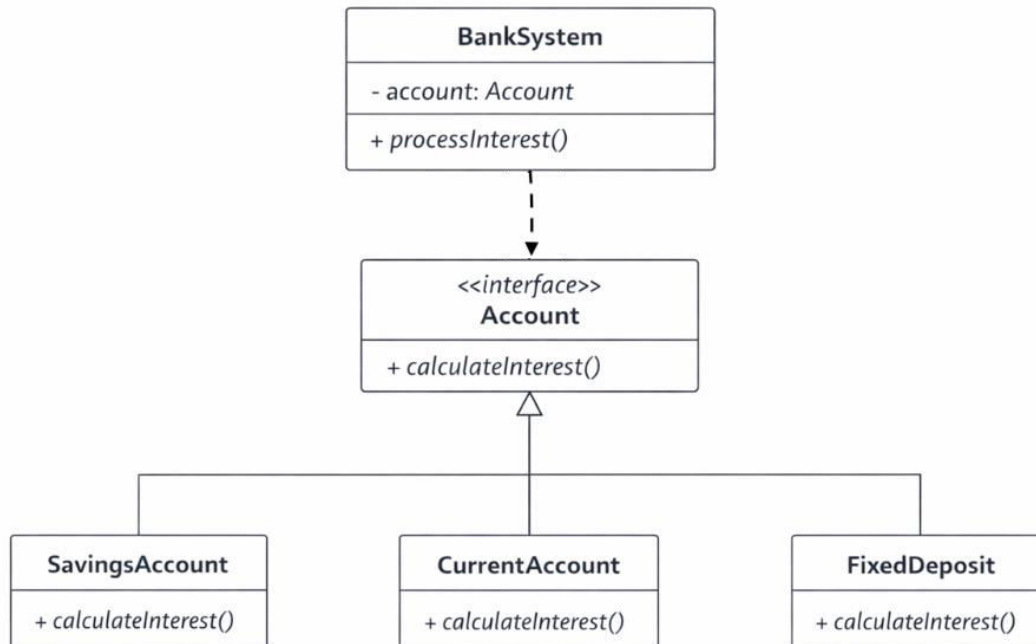
- **Variation point**

Branching point on existing system or in requirements.

- **Evolution point**

Supposed branching point, which might occur in future, but does not declare by existing requirements

Protected Variation in Banking System



- ✓ Variation Point: Interest logic
- ✓ Stable Interface: Account
- ✓ New Account Type → No modification of BankSystem
- ✓ Low Coupling
- ✓ High Cohesion
- ✓ Supports Open-Closed Principle

Object Oriented Analysis and Design

MCQ

1. The main purpose of the Indirection principle is to:

- A) Increase system performance
- B) Avoid direct coupling between components
- C) Replace inheritance
- D) Reduce the number of classes

2. Indirection works by introducing:

- A) Additional databases
- B) Static methods
- C) Global variables
- D) Intermediate classes or objects

Object Oriented Analysis and Design

MCQ

3. Pure Fabrication refers to:

- A) Creating domain objects
- B) Assigning responsibilities to artificial classes
- C) Removing classes
- D) Using inheritance for every design

4. Pure Fabrication is used when:

- A) Assigning responsibility to domain classes causes design issues
- B) Domain classes already solve the problem
- C) There is no need for modularity
- D) The system is very small

Object Oriented Analysis and Design

MCQ

5. Protected Variation works by:

- A) Adding more methods
- B) Creating stable interfaces around change points
- C) Removing modules
- D) Avoiding design principles

6. A variation point refers to:

- A) Random change in code
- B) Branching point in existing system requirements
- C) Database migration
- D) User interface redesign

Object Oriented Analysis and Design

MCQ SOLUTION

1.B

2.D

3.B

4.A

5.B

6.B

- [Source Code Examples](#)
- [Ad-hoc, Inclusion, Parametric & Coercion Polymorphisms – GeeksforGeeks](#)
- [Larman - Monopoly Game.pdf - THE MONOPOLY GAME SYSTEM • The software version of the game will run as a simulation • One person will start the game and | Course Hero](#)
- <http://www.kamilgrzybek.com/design/grasp-explained/>



THANK YOU

Prof. Shridevi A Sawant
Computer Science and Engineering



Object Oriented Analysis and Design

UE23CS352B

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

UE23CS352B: Object Oriented Analysis and Design

SOLID: Single Responsibility

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements:

Prof. Shilpa S

Dept. of Computer Science and Engineering

Object Oriented Analysis and Design

Agenda



- **Important aspects of Bad Design**
- **Good Design**
- **Introduction – SOLID**
- **SRP**
- **Implementation of SRP**
- **References**

Object Oriented Analysis and Design

Important Aspects of Bad Design

Rigidity

- the impact of a change is unpredictable
- every change causes a cascade of changes in dependent modules
- costs become unpredictable

Fragility

- the software tends to break in many places on every change
- the breakage occurs in areas with no conceptual relationship
- on every fix the software breaks in unexpected ways

Immobility

- it's almost impossible to reuse interesting parts of the software
- the useful modules have too many dependencies
- the cost of rewriting is less compared to the risk faced to separate those parts

Viscosity

- a hack is cheaper to implement than the solution within the design
- preserving design moves are difficult to think and to implement
- it's much easier to do the wrong thing rather than the right one

Object Oriented Analysis and Design

Good Design



- What's the reason why a design becomes rigid, fragile, immobile, and viscous?

Improper
dependencies
between modules

- Characteristics of a good design

High cohesion

Low coupling

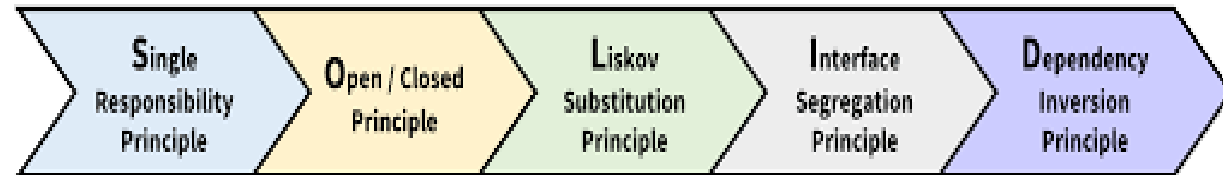
- How can we achieve a good design? - SOLID

Object Oriented Analysis and Design

Introduction to SOLID

- Is an acronym for 5 important design principles introduced by **Robert J. Martin**.
- Leads to more **flexible and stable** software architecture that's easier to maintain and extend, and **less likely to break**
- Acronym identified by **Michael Feathers**.

SOLID



- SOLID principles are software design coding standards
- Helps us to obtain good software with low coupling and high cohesion
- Applied to reduce dependencies - changes in one part of software will not be impacting others.

Object Oriented Analysis and Design

Single Responsibility Principle(SRP)

- States that **every module or class should have responsibility over a single part of the functionality provided by the software.**
- A class should have one, and only one, reason to change - Robert C. Martin
 - Each class only does one thing.
 - Every class or module only has responsibility for one part of the software's functionality.
 - Ensures low coupling code.
 - Ensures easier coding to understand and maintain.
 - Can be applied to classes, software components, and microservices.
 - Code becomes easier to test and maintain, it makes software easier to implement, and it helps to avoid unanticipated side-effects of future changes.



Object Oriented Analysis and Design

Violation of SRP

- The code violates the Single Responsibility Principle, as the Book class has two responsibilities.
- First, it sets the data related to the books (title and author).
- Second, it searches for the book in the inventory.

Note: The setter methods change the Book object, which might cause problems when we want to search the same book in the inventory.

```
class Book {  
    String title;  
    String author;  
  
    String getTitle() {  
        return title;  
    }  
  
    void setTitle(String title) {  
        this.title = title;  
    }  
  
    String getAuthor() {  
        return author;  
    }  
  
    void setAuthor(String author) {  
        this.author = author;  
    }  
  
    void searchBook() {...}  
}
```


Object Oriented Analysis and Design

Solution



- Identify things that are changing for different reasons
- Group together things that change for the same reason
- Decouple the responsibilities.
- In the refactored code, the Book class will only be responsible for getting and setting the data of the Book object.
- Create another class called InventoryView that will be responsible for checking the inventory.
- Move the searchBook() method to the new class.

Object Oriented Analysis and Design

Good Design using SRP



```
class Book {
    String title;
    String author;

    String getTitle() {
        return title;
    }

    void setTitle(String title) {
        this.title = title;
    }

    String getAuthor() {
        return author;
    }

    void setAuthor(String author) {
        this.author = author;
    }
}
```

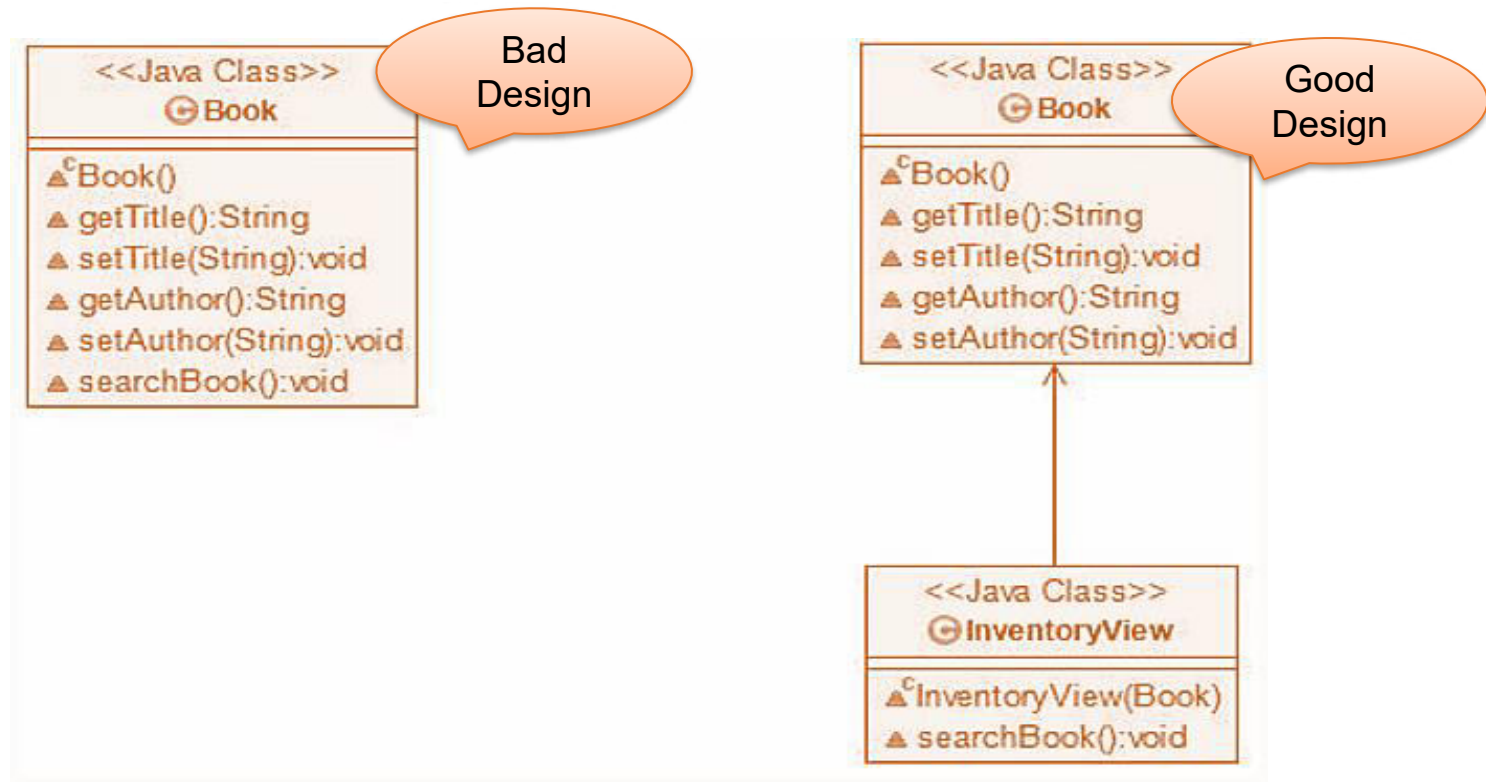
```
class InventoryView {
    Book book;

    InventoryView(Book book) {
        this.book = book;
    }

    void searchBook() {...}
}
```

Object Oriented Analysis and Design

UML Class Diagram



The Book class should only represent book data, not perform search operations.

SRP: One class → One responsibility → One reason to change.

Object Oriented Analysis and Design

Advantages



- **Testing** – A class with one responsibility will have fewer test cases.
- **Lower coupling** – Less functionality in a single class will have fewer dependencies.
- **Organization** – Smaller, well-organized classes are easier to search than monolithic ones.

Object Oriented Analysis and Design

SRP - Implementation



DEMO

MCQ

1. Who introduced the SOLID principles?

- A) Michael Feathers
- B) Robert J. Martin
- C) Erich Gamma
- D) James Gosling

2. According to SRP, a class should have how many reasons to change?

- A) One and only one
- B) At least two
- C) Multiple
- D) None

MCQ

3. Which is listed as an advantage of SRP?

- A) More dependencies
- B) Increased rigidity
- C) Fewer test cases
- D) Larger monolithic classes

4. SOLID principles help to:

- A) Increase coupling
- B) Create rigid design
- C) Increase viscosity
- D) Reduce dependencies

Object Oriented Analysis and Design



MCQ SOLUTION

1. B
2. A
3. C
4. D

Object Oriented Analysis and Design

References



- [Introduction To SOLID Principles \(c-sharpcorner.com\)](http://c-sharpcorner.com)
- [A Solid Guide to SOLID Principles | Baeldung](#)
- [SOLID Principle in Programming: Understand With Real Life Examples - GeeksforGeeks](#)



THANK YOU

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

shridevias@pes.edu



Object Oriented Analysis and Design

UE23CS352B

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

UE23CS352B: Object Oriented Analysis and Design

SOLID: Liskov Substitution Principle (LSP)

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements:

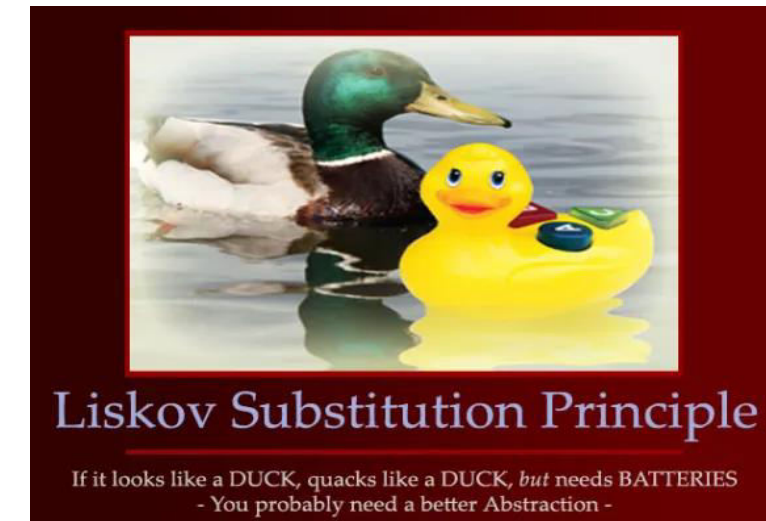
Prof. Sindhu R Pai

Dept. of Computer Science and Engineering

Object Oriented Analysis and Design

Liskov Substitution Principle (LSP)

- Introduced by Barbara Liskov in 1987
- **“Let S be a subtype of T , then for each object $o1$ of type S there is an object $o2$ of type T such that for all programs P defined in terms of T , the behaviour of P is unchanged when $o1$ is substituted for $o2$ ”.**
- **Derived types must be completely substitutable for their base types**
- An object of a superclass should be replaceable by objects of its subclasses without causing issues in the application.
- A child class should never change the characteristics of its parent class
- Derived classes should never do less than their base class.
- Applies to inheritance(is – a relationship) hierarchies and avoids overuse/misuse



Object Oriented Analysis and Design

Example code for violation of LSP



- Consider the same book store example. The book store asks us to add a new delivery functionality to the application. The store also sells fancy hardcovers. It wants to deliver that to their high street shops
- Create a BookDelivery class that informs customers about the number of locations where they can collect their order. Create a new HardcoverDelivery subclass that extends BookDelivery and overrides the getDeliveryLocations() method with its own functionality

```
class BookDelivery {  
    String titles;  
    int userID;  
    void getDeliveryLocations()  
{...}  
}
```

```
class HardcoverDelivery extends BookDelivery {  
    @Override  
    void getDeliveryLocations() {...}  
}
```

Object Oriented Analysis and Design

Example code for violation of LSP



- Later, the store asks us to create delivery functionalities for audiobooks.
- Now, we extend the existing BookDelivery class with an AudiobookDelivery subclass

```
class AudiobookDelivery extends BookDelivery {  
    @Override  
    void getDeliveryLocations() { /* can't be implemented */ }  
    // audiobooks can't be delivered to physical locations.  
}
```

- We have to change some characteristics of the getDeliveryLocations() method. That would violate the Liskov Substitution Principle.
- After the modification, we couldn't replace the BookDelivery superclass with the AudiobookDelivery subclass without breaking the application

Object Oriented Analysis and Design

Solution



- To solve the problem, we need to fix the inheritance hierarchy.
- Let's introduce an extra layer of abstraction that better differentiates book delivery types.
- The new OfflineDelivery and OnlineDelivery classes split up the BookDelivery superclass.
- Move the getDeliveryLocations() method to OfflineDelivery and create a new getSoftwareOptions() method for the OnlineDelivery class (as this is more suitable for online deliveries).

Object Oriented Analysis and Design

Good Design



```
class BookDelivery {  
    String title;  
    int userID;  
}  
  
class OfflineDelivery extends BookDelivery {  
    void getDeliveryLocations() {...}  
}  
  
class OnlineDelivery extends BookDelivery {  
    void getSoftwareOptions() {...}  
}
```

Object Oriented Analysis and Design

Good Design



- In the refactored code, HardcoverDelivery will be the child class of OfflineDelivery and it will override the `getDeliveryLocations()` method with its own functionality.
- AudiobookDelivery will be the child class of OnlineDelivery, now it doesn't have to deal with the `getDeliveryLocations()` method.
- Instead, it can override the `getSoftwareOptions()` method of its parent with its own implementation (for instance, by listing and embedding available audio players).

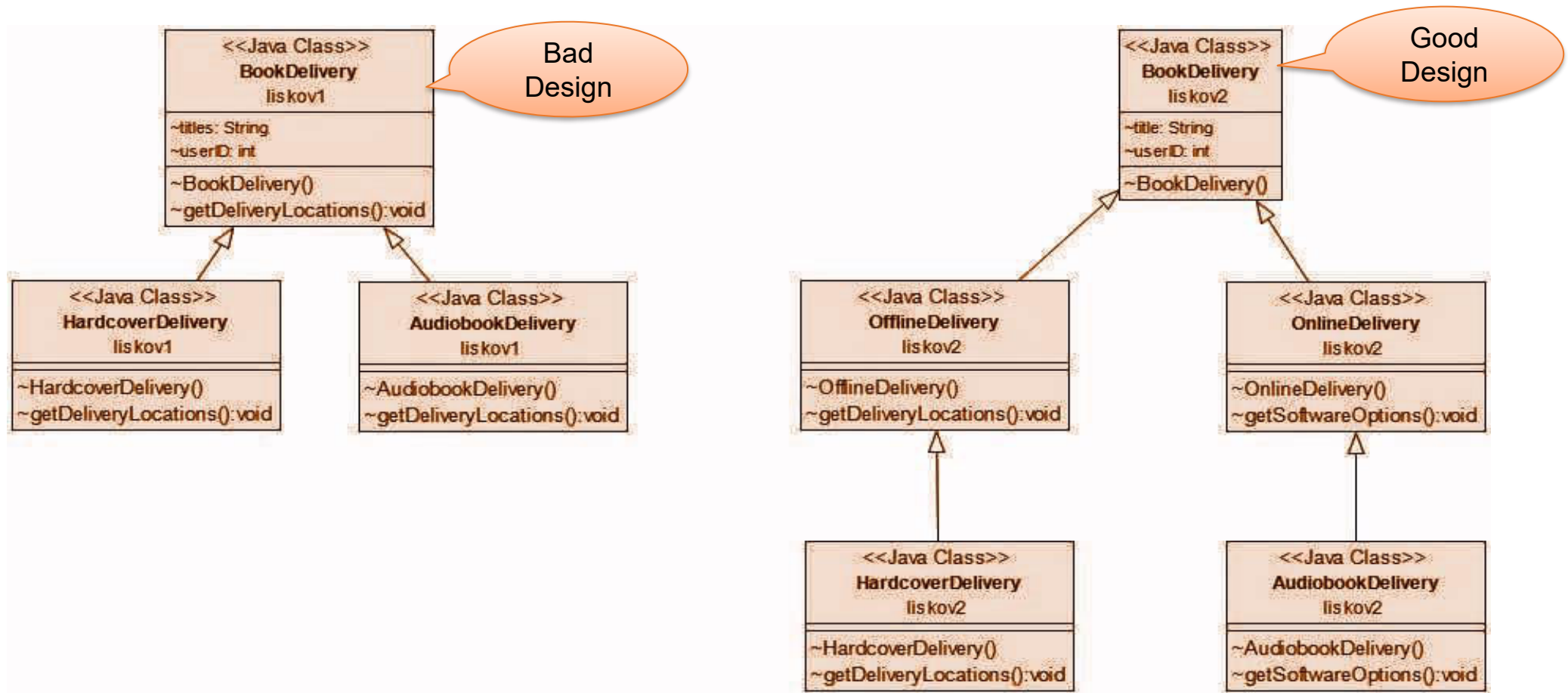
```
class HardcoverDelivery extends OfflineDelivery {  
    @Override  
    void getDeliveryLocations() {...}  
}
```

```
class AudiobookDelivery extends OnlineDelivery {  
    @Override  
    void getSoftwareOptions() {...}  
}
```

Object Oriented Analysis and Design

UML Class Diagram

- An extra layer is added to the inheritance hierarchy.
- New architecture is more complex, it provides us with a more flexible design.



Object Oriented Analysis and Design

MCQ

1. LSP primarily applies to which type of relationship?

- A) Has-a relationship
- B) Uses-a relationship
- C) Is-a relationship
- D) Depends-on relationship

2. According to LSP, derived types must be:

- A) More complex than base types
- B) Completely substitutable for their base types
- C) Independent of base types
- D) Modified frequently

Object Oriented Analysis and Design

MCQ

3. LSP ensures that substituting a subclass object should:

- A) Improve performance
- B) Change the program structure
- C) Leave program behavior unchanged
- D) Remove base class methods

4. According to LSP, a child class should:

- A) Redefine all parent behavior
- B) Never change the characteristics of its parent class
- C) Remove inherited methods
- D) Reduce functionality

Object Oriented Analysis and Design

MCQ

5. LSP is concerned with preserving:

- A) Method names
- B) Class size
- C) Behavioral consistency
- D) Code comments

6. LSP ensures that an object of a superclass should be replaceable by:

- A) Any unrelated class
- B) Only abstract classes
- C) Objects of its subclasses
- D) Static methods

Object Oriented Analysis and Design

MCQ SOLUTION

1.C

2.B

3.C

4.B

5.C

6.C

Object Oriented Analysis and Design

References



- [Liskov's Substitution Principle | SOLID Design Principles \(ep 1 part 1\) – YouTube](#)
- [SOLID Design Principles Explained: Interface Segregation with Code Examples \(stackify.com\)](#)
 - [Interface Segregation Principle explained with example in Java – JavaBrahman](#)
 - [Interface Segregation Principle in Java with Example \(javaguides.net\)](#)
 - [\(221\) Interface Segregation Principle - YouTube](#)



THANK YOU

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

shridevias@pes.edu



Object Oriented Analysis and Design

UE23CS352B

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

UE23CS352B: Object Oriented Analysis and Design

SOLID: Open – Closed Principle

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements:

Prof. Sindhu R Pai

Dept. of Computer Science and Engineering

SOLID: Open-Closed Principle (OCP)

- States that **classes, modules, microservices, and other code units should be open for extension but closed for modification.**
- **Bertrand Meyer** originated the term OCP
 - A software entity must be easily extensible with new features without having to modify its existing code in use.
 - We should be able to **extend the existing code using OOP features like inheritance via subclasses and interfaces.**
 - Never modify classes, interfaces, and other code units that already exist, as it can lead to unexpected behavior.
 - While adding a new feature extend the code rather than modifying it, so that the risk of failure is minimized

Violation of OCP

- The store wants to hand out cookbooks at a discount price
- Based on the Single Responsibility Principle, Create two separate classes:

CookbookDiscount - to hold the details of the discount

DiscountManager - to apply the discount to the price

```
class CookbookDiscount {  
    String getCookbookDiscount() {  
        String discount = "30% between Dec 1 and 24";  
        return discount;  
    }  
}  
  
class DiscountManager {  
    void processCookbookDiscount(CookbookDiscount  
discount) {...}  
}
```

Violation of OCP continued

The code works fine until the store management informs us that their cookbook discount sales were so successful that they want to extend it

```
class BiographyDiscount
{
    String getBiographyDiscount()
    {
        String discount = "50% on the
subject's birthday.";
        return discount;
    }
}
```

To process the new type of discount, we need to add the new functionality to the DiscountManager class, too:

```
class DiscountManager {
    void processCookbookDiscount(CookbookDiscount
discount) {...}
    void
processBiographyDiscount(BiographyDiscount
discount) {...}
}
```

Solution

- Refactor the code by adding an extra layer of abstraction that represents all types of discounts.
- Create a new interface called BookDiscount that, the CookbookDiscount and BiographyDiscount classes will implement.

```
public interface BookDiscount{  
    String getBookDiscount();  
}
```

```
class CookbookDiscount implements BookDiscount {  
    @Override  
    public String getBookDiscount() {  
        String discount = "30% between Dec 1 and  
24";  
        return discount;  
    }  
}  
class BiographyDiscount implements BookDiscount {  
    @Override  
    public String getBookDiscount() {  
        String discount = "50% on the subject's  
birthday";  
        return discount;  
    }  
}
```

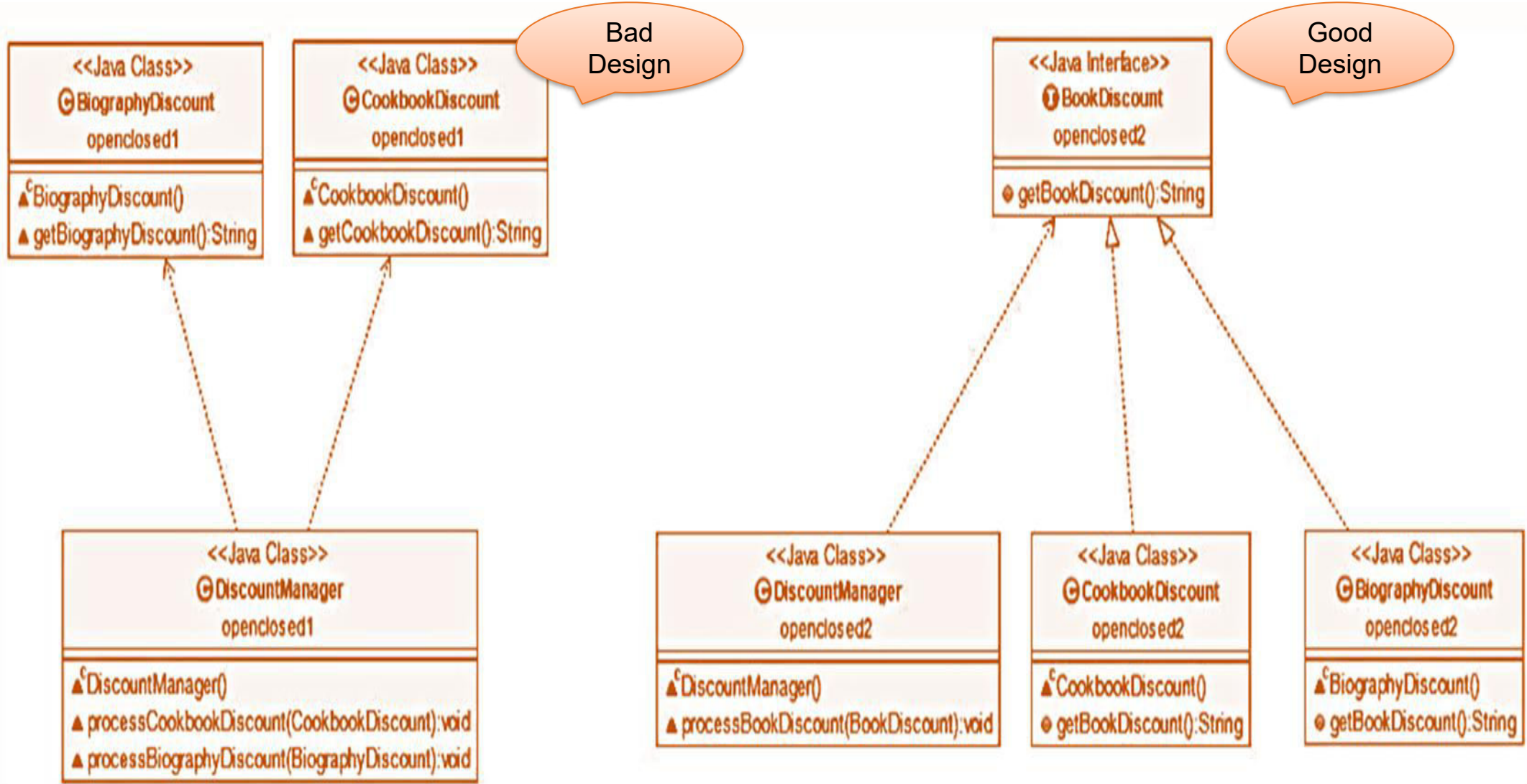
Solution continued



Now, DiscountManager can refer to the BookDiscount interface instead of the concrete classes. When the processBookDiscount() method is called, pass both CookbookDiscount and BiographyDiscount as an argument, as both are the implementation of the BookDiscount interface.

```
class DiscountManager
{
    void processBookDiscount(BookDiscount discount)
    {...}
}
```


UML Class Diagram



Why OCP?



- If not followed
 - ☐ End up testing the entire functionality
 - ☐ QA Team needs to test the entire flow
 - ☐ Costly process for the organization
 - ☐ Breaks the SRP as well
 - ☐ Maintenance overhead increase on the classes

Object Oriented Analysis and Design

MCQ

1. Which OOP features are suggested to extend existing code?

- A) Encapsulation and Polymorphism
- B) Inheritance and Interfaces
- C) Abstraction only
- D) Composition only

2. According to OCP, software entities should be:

- A) Open for modification and closed for extension
- B) Closed for both modification and extension
- C) Open for extension but closed for modification
- D) Open for modification only

Object Oriented Analysis and Design

MCQ

3. If OCP is not followed, what happens to testing?

- A) Only new features are tested
- B) No testing required
- C) Entire functionality must be tested
- D) Testing effort decreases

4. Violating OCP may also:

- A) Strengthen SRP
- B) Break SRP
- C) Reduce cohesion
- D) Eliminate abstraction

Object Oriented Analysis and Design

MCQ SOLUTION

1.B

2.C

3.C

4.B

Object Oriented Analysis and Design

References



- [Introduction To SOLID Principles \(c-sharpcorner.com\)](http://c-sharpcorner.com)
- [A Solid Guide to SOLID Principles | Baeldung](#)
- [SOLID Principle in Programming: Understand With Real Life Examples - GeeksforGeeks](#)



THANK YOU

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

shridevias@pes.edu



Object Oriented Analysis and Design

UE23CS352B

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

UE23CS352B: Object Oriented Analysis and Design

SOLID: Interface Segregation Principle (ISP)

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements:

Prof. Sindhu R Pai

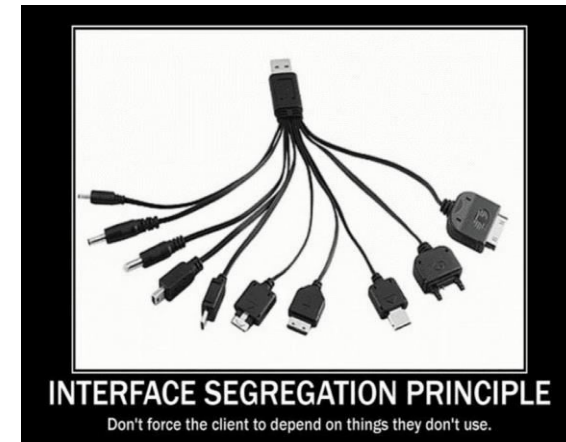
Dept. of Computer Science and Engineering

Object Oriented Analysis and Design

Interface Segregation Principle (ISP)

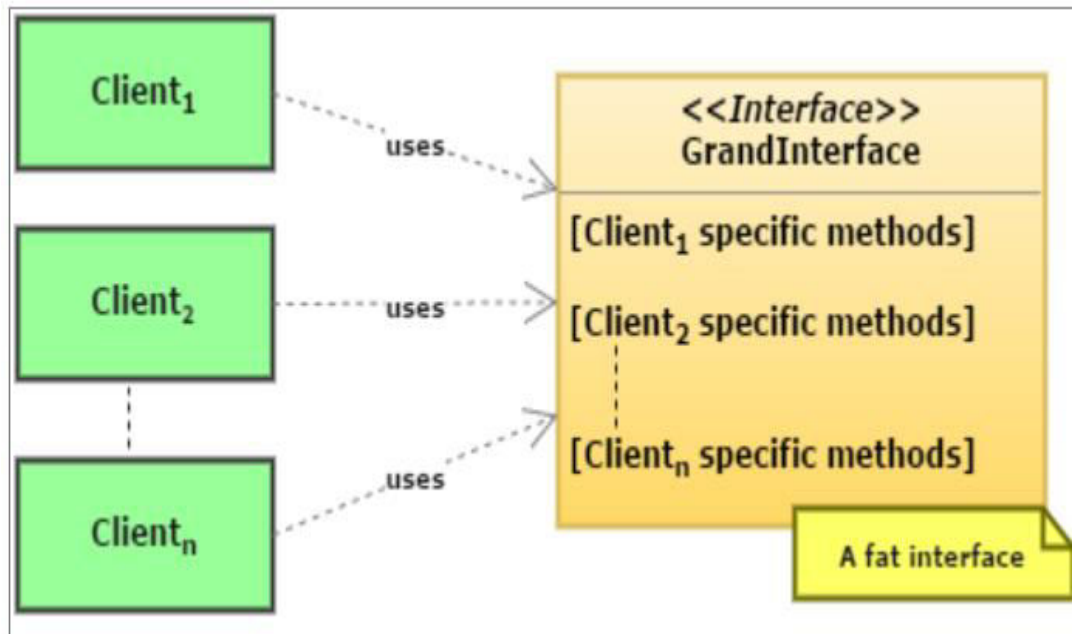
States that “Make fine grained interfaces that are client-specific. Clients should not be forced to implement interfaces they do not use”.

- Break the fat interfaces which has too many methods.
- The principle states that many client-specific interfaces are better than one general-purpose interface.
- Clients should not be forced to implement a function they do not need.
- Instead, start by building a new interface and then let your class implement multiple interfaces as needed.
- Create smaller interfaces that you can implement more flexibly
- Failure to comply with this principle means that in our implementations we will have dependencies on methods that we do not need but that we are obliged to define.

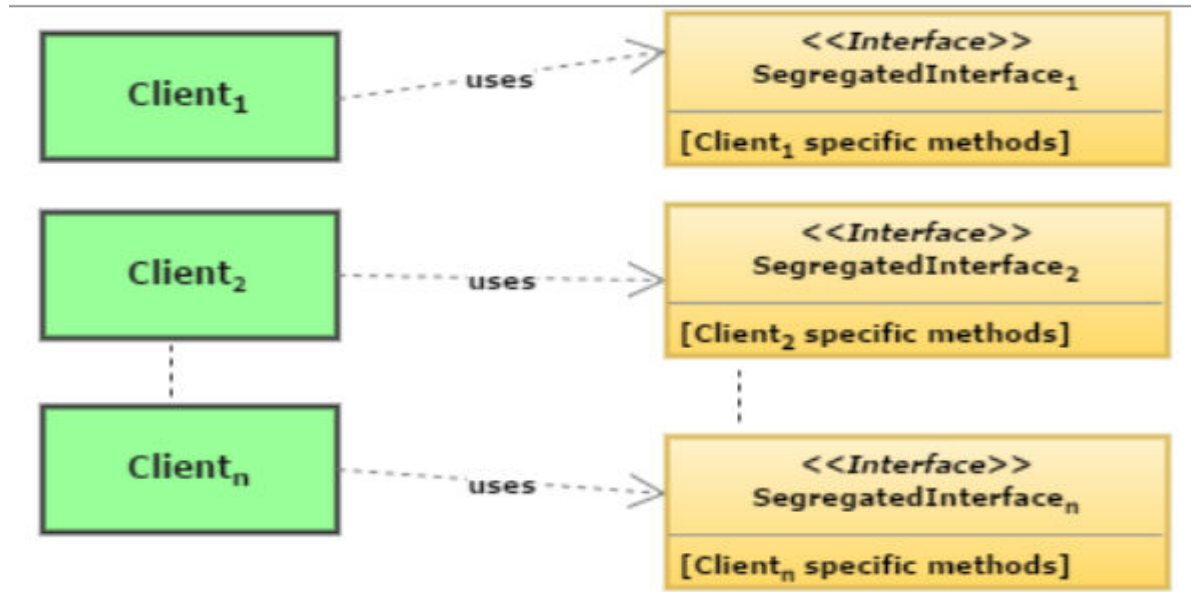


Object Oriented Analysis and Design

Interface Segregation Principle (ISP)



Before Applying ISP: Fat interface



After Applying ISP: Lean interface/role interface

Note: Interface should not be bloated with methods that implementing classes don't require.

Object Oriented Analysis and Design

Example code

```
interface IUser{
    boolean login(String name, String password);
    boolean register(String name, String password, String email);
    void logError(String error);
    boolean sendmail(String emailContent);
}
```

Code violating ISP

Code following ISP

```
interface IUser{
    boolean login(String name, String password);
    boolean register(String name, String password, String email);
}

interface ILogger{
    void logError(String error);
}

interface IMail{
    boolean sendmail(String emailContent);
}
```

Object Oriented Analysis and Design

Example code for violation of ISP

- Adding some user actions to the online bookstore so that customers can interact with the content before making a purchase by creating an interface called BookAction with three methods:

```
public interface BookAction {  
    void seeReviews();  
    void searchSecondhand();  
    void listenSample();  
}
```



Object Oriented Analysis and Design

Example code for violation of ISP continued



- Create two classes: HardcoverUI and an AudiobookUI that implement the BookAction interface with their own functionalities
- Both classes depend on methods they don't use, so it violates the Interface Segregation Principle

```
class HardcoverUI implements BookAction {  
    @Override  
    public void seeReviews() {...}  
  
    @Override  
    public void searchSecondhand() {...}  
  
    @Override  
    public void listenSample() {...}  
}
```

```
class AudiobookUI implements BookAction {  
    @Override  
    public void seeReviews() {...}  
  
    @Override  
    public void searchSecondhand() {...}  
  
    @Override  
    public void listenSample() {...}  
}
```

Object Oriented Analysis and Design

Solution

- Extend the code with two more specific sub-interfaces: HardcoverAction and AudioAction.

```
public interface BookAction {  
    void seeReviews();  
}  
  
public interface HardcoverAction extends BookAction {  
    void searchSecondhand();  
}  
  
public interface AudioAction extends BookAction {  
    void listenSample();  
}
```

Object Oriented Analysis and Design

Solution



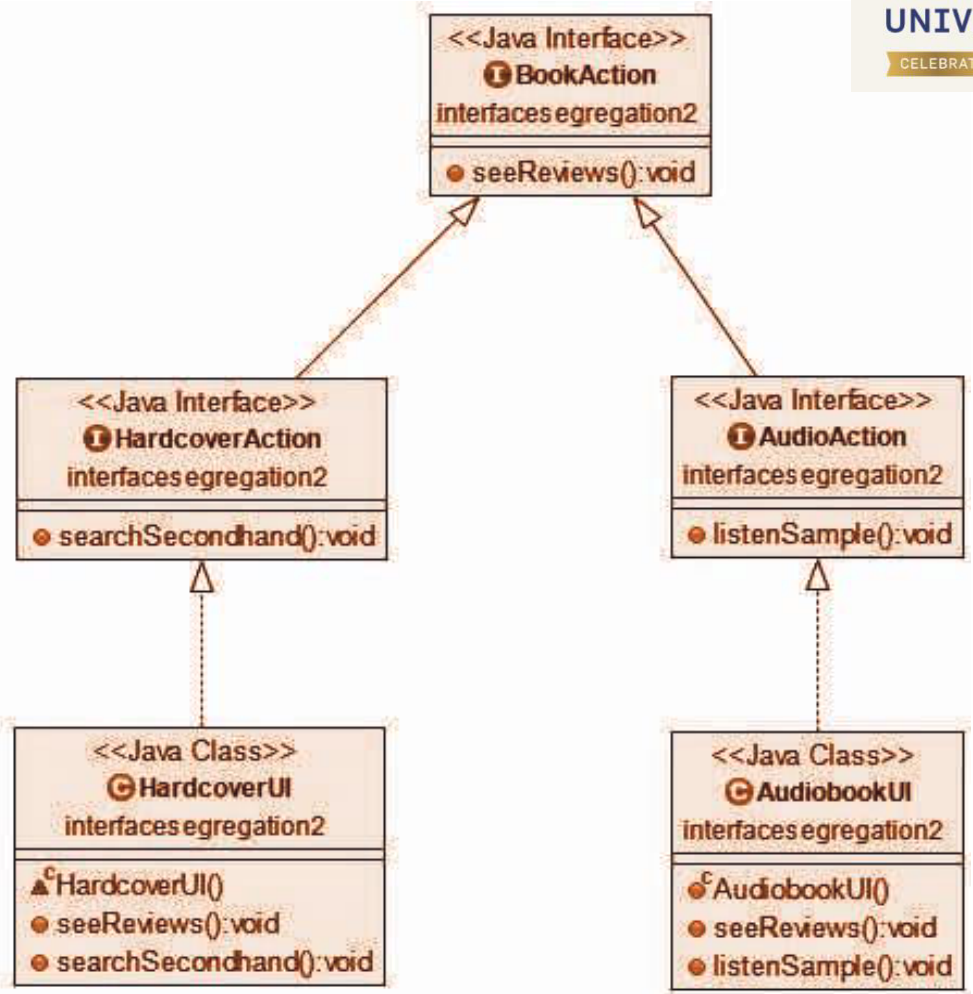
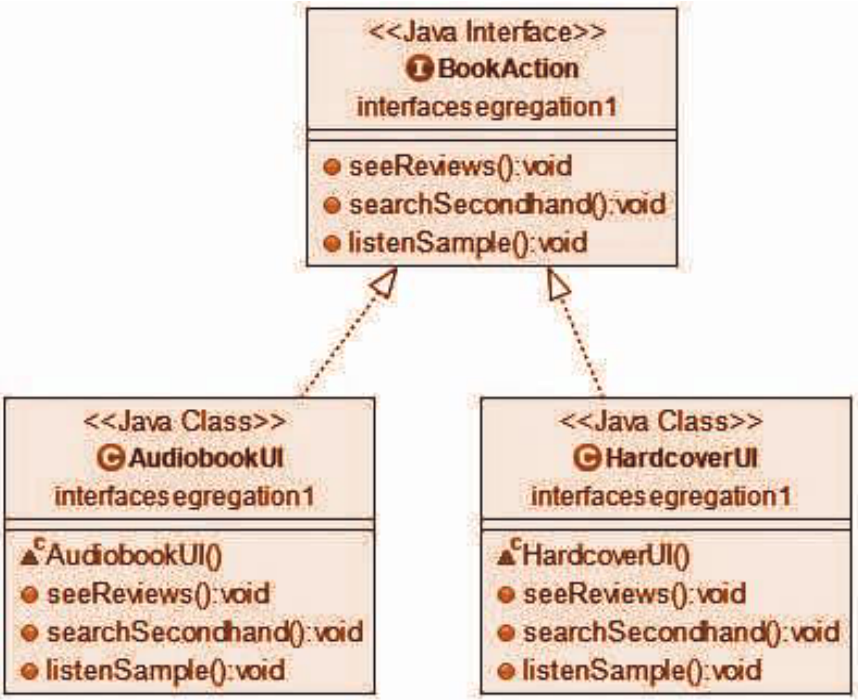
- The HardcoverUI class can implement the HardcoverAction interface and the AudiobookUI class can implement the AudioAction interface.
- Both the classes can implement the seeReviews() method of the BookAction super-interface.

```
class HardcoverUI implements HardcoverAction {  
    @Override  
    public void seeReviews() {...}  
  
    @Override  
    public void searchSecondhand() {...}  
}
```

```
class AudiobookUI implements AudioAction  
{  
    @Override  
    public void seeReviews() {...}  
  
    @Override  
    public void listenSample() {...}  
}
```


Object Oriented Analysis and Design

UML Class Diagram



Object Oriented Analysis and Design

Case Study : Xerox

- First used and formulated by Robert C. Martin while consulting for Xerox
- Xerox had created a new printer system that could perform a variety of tasks such as stapling and faxing along with the regular printing task
- The software for this system was created from the ground up
- Modifications and Deployment to the system became more complex

Problem: One large Job class having all the responsibilities of the entire system

Solution: One large Job class is segregated to multiple interfaces depending on the requirement



Object Oriented Analysis and Design



MCQ

1. The Interface Segregation Principle states that:

- A) Clients should depend on large general-purpose interfaces
- B) Interfaces should contain as many methods as possible
- C) Clients should not be forced to implement interfaces they do not use
- D) All classes must implement a single interface

2. According to ISP, implementing classes should:

- A) Implement all available methods
- B) Implement only necessary methods
- C) Avoid interfaces
- D) Use inheritance instead

Object Oriented Analysis and Design

MCQ

3. ISP suggests making interfaces:

- A) Large and reusable
- B) Fine-grained and specific
- C) Generic and broad
- D) Static

4. Failure to follow ISP results in:

- A) Reduced dependencies
- B) Dependencies on unused methods
- C) Increased abstraction
- D) Better flexibility



Object Oriented Analysis and Design

MCQ SOLUTION

- 1.C
- 2.B
- 3.B
- 4.B



Object Oriented Analysis and Design

References



- [SOLID Design Principles Explained: Interface Segregation with Code Examples \(stackify.com\)](#)
 - [Interface Segregation Principle explained with example in Java – JavaBrahman](#)
 - [Interface Segregation Principle in Java with Example \(javaguides.net\)](#)
 - [\(221\) Interface Segregation Principle - YouTube](#)



THANK YOU

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

shridevias@pes.edu



Object Oriented Analysis and Design

UE23CS352

B

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

UE23CS352B: Object Oriented Analysis and Design

SOLID - Dependency Inversion Principle (DIP)

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements:

Prof. Sindhu R Pai

Dept. of Computer Science and Engineering

Object Oriented Analysis and Design

Dependency Inversion Principle (DIP)



Program to the interface, not the implementation

- High level modules should not depend on low level modules, both should depend upon abstractions.
- Abstractions should not depend upon details , details should depend upon abstractions.
- The goal is to avoid tightly coupled code, as it easily breaks the application.
- Decouple high-level and low-level classes. The interaction between them must be thought of as an **abstract interaction**
- According to Robert C Martin, Dependency Inversion Principle is a **specific combination of the Open-Closed and Liskov Substitution Principles.**

Object Oriented Analysis and Design

Example code violates DIP

- Suppose a book store asked us to build a new feature that enables customers to put their favorite books on a shelf.
- To implement this functionality, we create a lower-level Book class and a higher-level Shelf class. The Book class will allow users to see reviews and read a sample of each book they store on their shelves. The Shelf class will let them add a book to their shelf and customize the shelf.

```
class Book {  
    void seeReviews() {...}  
    void readSample() {...}  
}
```

```
class Shelf {  
    Book book;  
    void addBook(Book book) {...}  
    void customizeShelf() {...}  
}
```



Object Oriented Analysis and Design

Example code violates DIP

- When the store asks us to enable customers to add DVDs to their shelves,
First create DVD.

```
class DVD {  
    void seeReviews() {...}  
    void watchSample() {...}  
}
```

Now, we should modify the Shelf class so that it can accept DVDs, too.

- However, this would clearly break the Open-Closed Principle.

The solution is to create an abstraction layer for the lower-level classes (Book and DVD).



Object Oriented Analysis and Design

Solution

- Introduce the Product interface and both classes will implement.

```
public interface Product {  
    void seeReviews();  
    void getSample();  
}
```

```
class Book implements Product {  
    @Override  
    public void seeReviews() {...}  
  
    @Override  
    public void getSample() {...}  
  
}
```

```
class DVD implements Product {  
    @Override  
    public void seeReviews() {...}  
  
    @Override  
    public void getSample() {...}  
  
}
```

Object Oriented Analysis and Design

Solution



Now, Shelf can reference the Product interface instead of its implementations (Book and DVD).

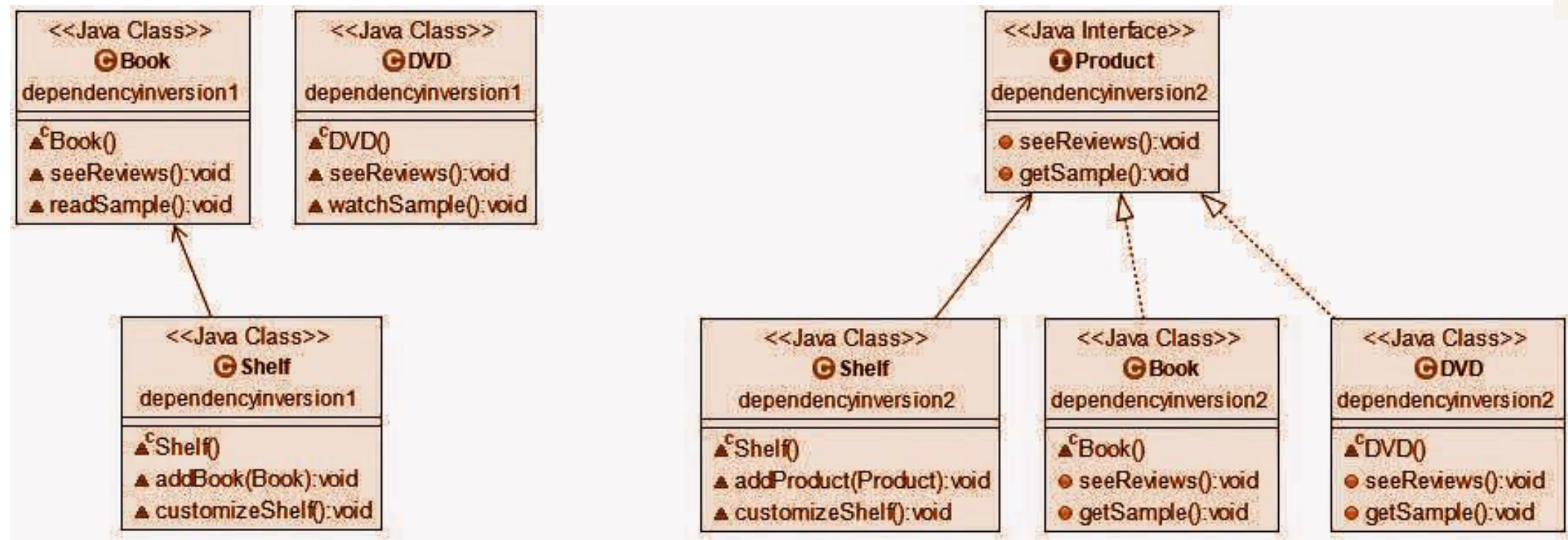
The refactored code also allows us to later introduce new product types (for instance, Magazine) that customers can put on their shelves, too.

```
class Shelf {  
    Product p;  
    void getSample();  
    void addProduct(Product product) {...}  
    void customizeShelf() {...}  
}
```

Note: Code follows the **Liskov Substitution Principle**, as the Product type can be substituted with both of its subtypes (Book and DVD) without breaking the program. At the same time, we have also implemented the **Dependency Inversion Principle**, as high-level classes don't depend on low-level classes

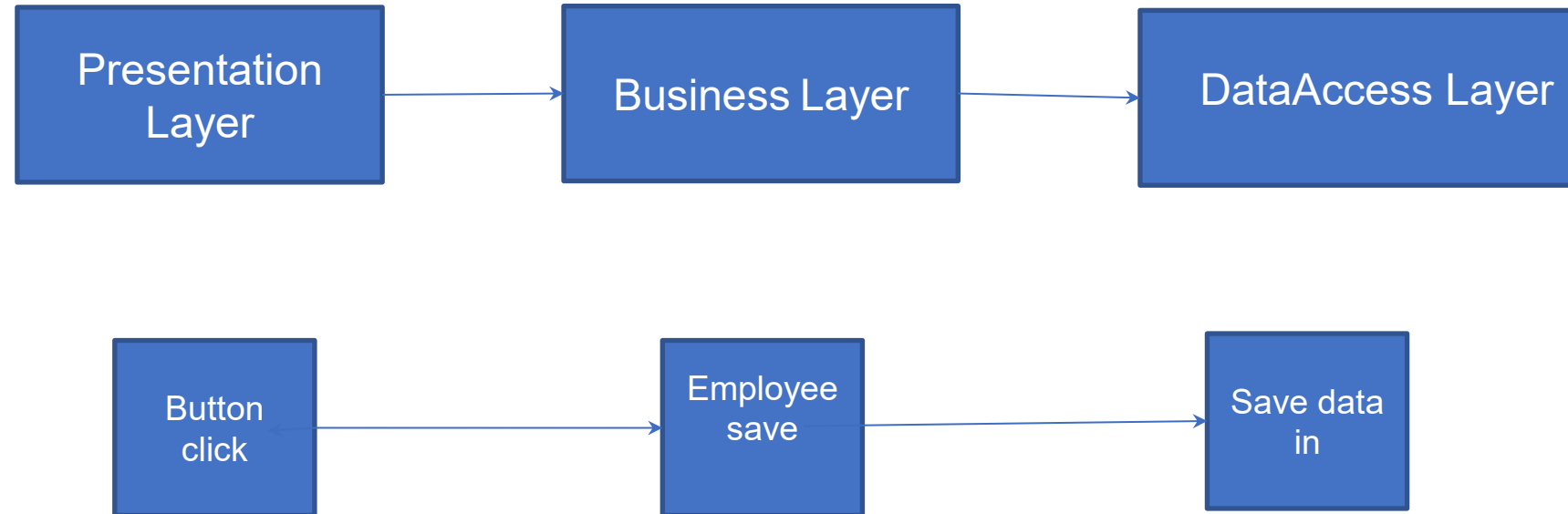
Object Oriented Analysis and Design

UML Class Diagram



Object Oriented Analysis and Design

Intention of Usage – DIP and it's Implementation



Object Oriented Analysis and Design

MCQ

1. According to DIP, high-level modules should:

- A) Depend on low-level modules
- B) Avoid abstraction
- C) Depend on abstractions
- D) Directly control low-level modules

2. In DIP, abstractions should:

- A) Depend on details
- B) Be avoided
- C) Not depend upon details
- D) Be concrete



Object Oriented Analysis and Design

MCQ

3. The primary goal of DIP is to avoid:

- A) Inheritance
- B) Polymorphism
- C) Tightly coupled code
- D) Interfaces

4. The interaction between high-level and low-level modules should be:

- A) Direct and concrete
- B) Static
- C) Abstract
- D) Hardcoded



PES

UNIVERSITY

CELEBRATING 50 YEARS

Object Oriented Analysis and Design

MCQ SOLUTION

- 1.C
- 2.C
- 3.C
- 4.C





THANK YOU

Shridevi A Sawant

Department of Computer Science and Engineering

shridevias@pes.edu



UE23CS352B - Object Oriented Analysis and Design

Dr. R. Nandhi Kesavan
Computer Science and Engineering

Scenario-based exercise to apply each SOLID principle

Online Food ordering

- A food delivery platform allows users to **order food online** from multiple restaurants. It includes the following key features:
 - ❑ **User Management** (Register/Login, Address Management)
 - ❑ **Restaurant Management** (Menu, Pricing, Availability)
 - ❑ **Order Processing** (Order Placement, Payment, Delivery Tracking)
- Now, let's **apply each SOLID principle** to design the system.

Object Oriented Analysis and Design

Single Responsibility Principle (SRP)

Problem:

A single Order class is responsible for multiple tasks:

- Managing order details (items, price, restaurant)
- Processing payments
- Sending order confirmation notifications

Single Responsibility Principle (SRP)

Solution:

```
class Order {  
    private List<Item> items;  
    private double totalPrice;  
  
    public void addItem(String name,  
int price) {  
        /* Add item logic */  
    }  
  
    public double calculateTotal() {  
        /* Calculate total price */  
        return totalPrice;  
    }  
}
```

```
class PaymentProcessor {  
    public void processPayment(Order order,  
PaymentMethod method) {  
        /* Payment logic */  
    }  
}  
  
class NotificationService {  
    public void sendOrderConfirmation(User  
user, Order order) {  
        /* Send email/SMS */  
    }  
}
```


Object Oriented Analysis and Design

Open-Closed Principle (OCP)

Problem:

The **PaymentProcessor** class has a **processPayment()** method that only supports **Credit Card** payments. If we need to add **PayPal, UPI, or Wallets**, we have to modify the existing class, violating OCP.

Solution:

Use **Polymorphism**—create an interface for payments, and extend it for new payment methods **without modifying existing code**.

Open-Closed Principle (OCP)

```
interface PaymentMethod {  
    void pay(double amount);  
}  
  
class CreditCardPayment implements PaymentMethod {  
    public void pay(double amount) {  
        /* Credit Card logic */  
    }  
}  
  
class PayPalPayment implements PaymentMethod {  
    public void pay(double amount) {  
        /* PayPal logic */  
    }  
}  
  
class PaymentProcessor {  
    public void processPayment(PaymentMethod method, double amount)  
    {  
        method.pay(amount);  
    }  
}
```

Liskov Substitution Principle (LSP)

Problem:

We have a `DeliveryPerson` class that has a `deliverOrder()` method. Now, we introduce **DroneDelivery** for automated delivery.

If `DroneDelivery` extends `DeliveryPerson`, it **inherits methods like `assignVehicle()` that drones don't need**, leading to incorrect behavior.

Solution:

Use a **common interface** instead of forcing inheritance.

Liskov Substitution Principle (LSP)

```
interface DeliveryService {  
    void deliverOrder(Order order);  
}  
  
class DeliveryPerson implements DeliveryService {  
    public void deliverOrder(Order order) {  
        /* Assign person and vehicle */  
    }  
}  
  
class DroneDelivery implements DeliveryService {  
    public void deliverOrder(Order order) {  
        /* Autonomous drone delivery */  
    }  
}
```

Interface Segregation Principle (ISP)

Problem:

A RestaurantService interface has methods:

1. addItem()
2. updateStock()
3. generateMonthlyReport()

A **small home-based restaurant** only wants to manage menu items but is **forced to implement** generateMonthlyReport(), which it doesn't need.

Solution:

- Split into smaller interfaces.

Interface Segregation Principle (ISP)

```
interface MenuManagement {  
    void addMenuItem(Item item);  
    void updateStock(Item item, int quantity);  
}  
  
interface Reporting {  
    void generateMonthlyReport();  
}  
  
class SmallRestaurant implements MenuManagement {  
    public void addMenuItem(Item item) {  
        /* Add item logic */  
    }  
  
    public void updateStock(Item item, int quantity) {  
        /* Stock logic */  
    }  
}
```

Dependency Inversion Principle (DIP)

Problem:

- OrderService directly depends on MySQLDatabase to store orders. If we switch to **MongoDB**, we need to change multiple parts of the system.

Solution:

- Depend on **abstractions** instead of concrete classes.

Dependency Inversion Principle (DIP)

```
interface Database {  
    void saveOrder(Order order);  
}  
  
class MySQLDatabase implements Database {  
    public void saveOrder(Order order) {  
        /* MySQL logic */  
    }  
}  
  
class MongoDBDatabase implements Database {  
    public void saveOrder(Order order) {  
        /* MongoDB logic */  
    }  
}
```


Object Oriented Analysis and Design

Dependency Inversion Principle (DIP)

```
class OrderService {  
    private Database database;  
  
    public OrderService(Database database) {  
        this.database = database;  
    }  
  
    public void placeOrder(Order order) {  
        database.saveOrder(order);  
    }  
}
```



THANK YOU

Dr. R. Nandhi Kesavan
Computer Science and Engineering



Object Oriented Analysis and Design

UE23CS352B

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

UE23CS352B: Object Oriented Analysis and Design

Introduction To Design Patterns

Acknowledgements:

Dr. Sudeepa Roy Dey and Dr Geetha D

Dept. of Computer Science and Engineering

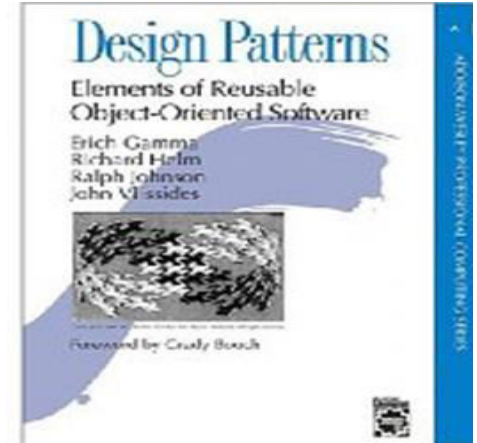
- The Beginning of Design Patterns
- Design Challenges
- What is a design pattern?
- What is the advantage of knowing/using design patterns?

Christopher Alexander, architect

- A Pattern Language--Towns, Buildings, Construction
- Timeless Way of Building (1979)
- “Each pattern describes a *problem* which occurs over and over again in our environment, and then describes the core of the *solution* to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice.”

Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley Publishing Company, 1994 Written by this "gang of four"

- Dr. Erich Gamma, then Software Engineer, Taligent, Inc.
- Dr. Richard Helm, then Senior Technology Consultant, DMR Group
- Dr. Ralph Johnson, then and now at University of Illinois, Computer Science Department
- Dr. John Vlissides, then a researcher at IBM



Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley

This book defined 23 patterns in three categories

- Creational patterns deal with the process of object creation
- Structural patterns, deal primarily with the static composition and structure of classes and objects
- Behavioral patterns, which deal primarily with dynamic interaction among classes and objects

- Designing software for reuse is hard. One must find:
 - i. a good problem decomposition, and the right software
 - ii. a design with flexibility, modularity and elegance
- Designs often emerge from trial and error
- Successful designs do exist
 - i. two designs they are almost never identical
 - ii. they exhibit some recurring characteristics

Can designs be described, codified or standardized?

- this would short circuit the trial and error phase
- produce "better" software faster

Architecture

- ❑ High-level framework for structuring an application
 - “client-server based on remote procedure calls”
 - “abstraction layering”
 - “distributed object-oriented system based on CORBA”
- ❑ Defines the system in terms of computational components & their interactions

Design Patterns

- ❑ Lower level than architectures (Sometimes, called *micro-architecture*)
- ❑ Reusable collaborations that solve subproblems within an application
 - how can I decouple subsystem X from subsystem Y?

Why Design Patterns?

- ❑ Design patterns support *object-oriented reuse* at a high level of abstraction
- ❑ Design patterns provide a “framework” that guides and constrains object-oriented implementation

What are Design Patterns?

Design patterns are used to represent some of the best practices adapted by experienced object-oriented software developers. They are like pre-made blueprints that you can customize to solve a recurring design problem in your code.

- **Reusable solutions to the problems:** In software engineering, a design pattern is general reusable solution to commonly occurring problem in software design.
- **Interaction between the objects :**Object-oriented design patterns typically show relationships and interactions between classes or objects, without specifying final application classes or objects that are involved.
- **Template, not a solution :**It is not a finished design that can be transferred directly into code.
- **Language independent**

Pattern has four Essential Elements

- **Pattern Name**- is a handle we can use to describe a design problem, its solutions, and consequences in a word or two
- **The Problem** – describes when to apply the pattern. It explains the problem and its context. It might describe specific design problems such as how to represent algorithms as objects. It might describe class or object structures that are symptomatic of an inflexible design. Sometimes the problem will include a list of conditions that must be met before it makes sense to apply the pattern.
- **The Solution** - describes the elements that make up the design, their relationships, responsibilities, and collaborations.
- **Consequences** - are the results and trade-offs of applying the pattern.

- **Applicability** - What are the situations in which the design pattern can be applied?
- **Structure** - A graphical representation of the classes in the pattern using a notation based on the Object Modeling Technique (OMT)
- **Participants** - The classes and/or objects participating in the design pattern and their responsibilities.
- **Collaborations** - How the participants collaborate to carry out their responsibilities.
- **Consequences** - How does the pattern support its objectives? What are the trade-offs and results of using the pattern? What aspect of system structure does it let you vary independently?
- **Implementation** - What pitfalls, hints, or techniques should you be aware of when implementing the pattern? Are there language-specific issues?
- **Related Pattern** - What design patterns are closely related to this one? What are the important differences? With which other patterns should this one be used?

What Does a Pattern consists of?

Most patterns are described very formally so people can reproduce them in many contexts. Here are the sections that are usually present in a pattern description:

- **Pattern Name and Classification:** The pattern's name conveys the essence of the pattern succinctly. A good name is vital, because it will become part of your design vocabulary
- **Intent :**of the pattern briefly describes both the problem and the solution.
- **Motivation:** further explains the problem and the solution the pattern makes possible.
- **Structure of classes:** shows each part of the pattern and how they are related.
- **Code example:** in one of the popular programming languages makes it easier to grasp the idea behind the pattern.

Why we should learn Patterns?

☐ **Patterns are a common design vocabulary**

- Allows engineers to abstract a problem and talk about that abstraction in isolation from its implementation
- Embodies a culture; domain-specific patterns increase design speed

☐ **Patterns capture design expertise and allow that expertise to be communicated**

- promotes design reuse and avoid mistakes

☐ **Improve documentation (less is needed) and understandability (patterns are described well once)**

Pros of Design Patterns

Pros

- Add consistency to designs by solving similar problems the same way, independent of language
- Add clarity to design and design communication by enabling a common vocabulary
- Improve time to solution by providing templates which serve as foundations for good design
- Improve reuse through composition

Cons of Design Patterns

Cons

- Some patterns come with negative consequences (i.e. object proliferation, performance hits, additional layers)
- Consequences are subjective depending on concrete scenarios
- Patterns are subject to different interpretations, misinterpretations, and philosophies
- Patterns can be overused and abused--❓ Anti-Patterns

Selection of a Design Pattern

As there are 23 patterns in the catalog it will be difficult to choose a design pattern. It might become hard to find the design pattern that solves our problem, especially if the catalog is new and unfamiliar to you. Below is a list of approaches we can use to choose the appropriate design pattern:

- **Consider how design patterns solve design problems:** Considering how design patterns help you find appropriate objects, determine object granularity, specify object interfaces and several other ways in which design patterns solve the problems will let you choose the appropriate design pattern.
- **Scan intent sections:** Looking at the intent section of each design pattern's specification lets us choose the appropriate design pattern.
- **Study how patterns interrelate:** The relationships between the patterns will direct us to choose the right patterns or group of patterns.

- **Study patterns of like purpose:** Each design pattern specification will conclude with a comparison of that pattern with other related patterns. This will give you an insight into the similarities and differences between patterns of like purpose.
- **Examine a cause of redesign:** Look at your problem and identify if there are any causes of redesign. Then look at the catalog of patterns that will help you avoid the causes of redesign
- **Consider what should be variable in your design:** Consider what you want to be able to change without redesign. The focus here is on encapsulating the concept that varies.

Usage of Design Pattern-How to use a selected Design Pattern

Below steps will show you how to use a design pattern after selecting one:

- **Read the pattern once through for a overview:**

Give importance to the applicability and consequences sections(in format) to ensure that the pattern is right for your problem.

- **Study the structure, participants and collaborations sections:**

Make sure to understand the classes and objects in the pattern and how they relate to one another.

- **Look at the sample code section to see a concrete example of the pattern in action:**

Studying the code helps us to implement the pattern.

Usage of a Design Pattern-How to use a Design Pattern

- **Define the classes:**

Declare their interfaces, inheritance relationships and instance variables, which represent the data and object references. Identify the affected classes by the pattern and modify them accordingly.

- **Define application-specific names for the operations in the pattern:**

Use the responsibilities and collaborations as a guide to name the operations. Be consistent with the naming conventions.

- **Implement the operations to carry out the responsibilities and collaborations in the pattern:**

The implementation section provides hints to guide us in the implementation. The examples in the sample code section can also help as well.

MCQ

1.-Design patterns mainly represent:

- A) Programming languages
- B) Best practices used by experienced developers
- C) Database queries
- D) Compiler techniques

2. The famous “Gang of Four” book defined how many design patterns?

- A) 15
- B) 18
- C) 23
- D) 30

MCQ

3. Creational patterns are concerned with:

- A) Program execution
- B) Object creation
- C) Network communication
- D) UI design

4. Structural patterns mainly focus on:

- A) Object creation
- B) Data storage
- C) Compilation
- D) Static composition of classes and objects

MCQ

5. Behavioral patterns focus on:

- A) Dynamic interaction among classes and objects
- B) Memory allocation
- C) Hardware design
- D) Database schemas

6. One disadvantage of design patterns is:

- A) They cannot be reused
- B) They prevent modular design
- C) They eliminate abstraction
- D) They may introduce extra complexity or layers

MCQ SOLUTION

1.B

2.C

3.B

4.D

5.A

6.D

- https://sourcemaking.com/design_patterns/
- <https://refactoring.guru/design-patterns/java>
- Design Patterns: Elements of Reusable Object-Oriented Software
Erich Gamma, Ralph Johnson, Richard Helm · 1995



THANK YOU

Prof. Shridevi A Sawant

Department of Computer Science and Engineering



Object Oriented Analysis and Design

UE23CS352B

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only



UE23CS352B: Object Oriented Analysis and Design

Creational Patterns – Singleton

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements:

Prof. Nivedita Kasturi

Dept. of Computer Science and Engineering

- ☐ Introduction to creational design patterns
- ☐ Types of creational Design Patterns.
- ☐ Singleton-definition
 - ✓ Motivation
 - ✓ Intent
 - ✓ Problem(use case)
 - ✓ Solution (without singleton and with singleton)
 - ✓ Implementation
 - ✓ Applicability
 - ✓ Consequence

- ☐ Creational design patterns provide various object creation mechanisms, which increase flexibility and reuse of existing code.
- ☐ The basic form of object creation could result in design problems or added complexity to the design.
- ☐ Creational design patterns solve this problem by somehow controlling this object creation.

Recurring themes:

- ☐ Encapsulate knowledge about which concrete classes the system uses (so we can change them easily later)
- ☐ Hide how instances of these classes are created and put together (so we can change it easily later)

Object Oriented Analysis and Design

Introduction to Creational Design Pattern



Everyone knows an object is created by using *new* keyword in java.
For example:

```
StudentRecord s1=new StudentRecord();
```

The *new* operator is often considered harmful as it scatters objects all over the application. Over time it can become challenging to change an implementation because classes become tightly coupled.

Hard-Coded code is not a good programming approach. Here, we are creating the instance by using the new keyword. Sometimes, the nature of the object must be changed according to the nature of the program. In such cases, we must get the help of creational design patterns to provide more general and flexible approach.

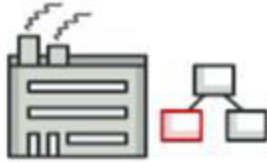
Object Oriented Analysis and Design

Types of Creational Design pattern



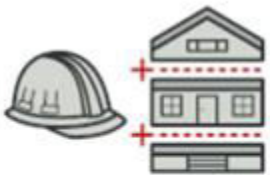
Singleton

Lets you ensure that a class has only one instance, while providing a global access point to this instance.



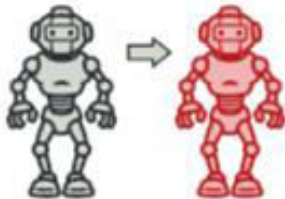
Factory Method

Provides an interface for creating objects in a superclass, but allows subclasses to alter the type of objects that will be created.



Builder

Lets you construct complex objects step by step. The pattern allows you to produce different types and representations of an object using the same construction code.



Prototype

Lets you copy existing objects without making your code dependent on their classes.

Composition over inheritance:
"Favor 'object composition' over 'class inheritance'."
(Gang of Four 1995:20)



		Purpose		
		Creational	Structural	Behavioral
Scope	Class	Factory Method (107)	Adapter (class) (139)	Interpreter (243) Template Method (325)
	Object	Abstract Factory (87) Builder (97) Prototype (117) Singleton (127)	Adapter (object) (139) Bridge (151) Composite (163) Decorator (175) Facade (185) Flyweight (195) Proxy (207)	Chain of Responsibility (223) Command (233) Iterator (257) Mediator (273) Memento (283) Observer (293) State (305) Strategy (315) Visitor (331)

Singleton: Class, Object Structural

Motivation

The Singleton is part of the Creational Design Pattern Family .Singleton Design Pattern aims to keep a check on initialization of objects of a particular class by **ensuring that only one instance of the object exists throughout the Java Virtual Machine.**

A Singleton class also provides one unique global access point to the object so that each subsequent call to the access point returns only that particular object.

A real-life example will be, **Imagine you work for a big company that has cloud storage**  **system for storing shared resources of files, images and documents we create shared storage as creating separate cloud storage**  **for every user**  **may be costly.**

Definition:

The singleton pattern is a design pattern that restricts the instantiation of a class to one object.

Why implement Singleton Design Pattern?

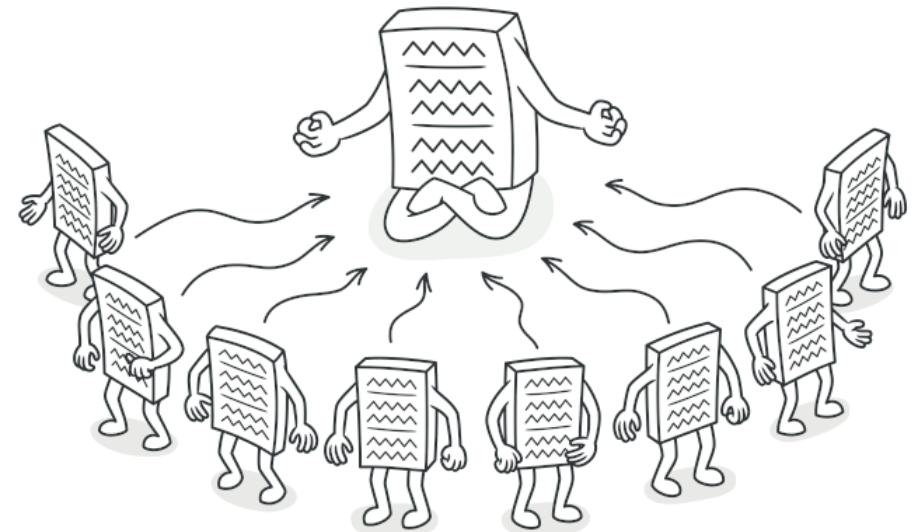
1. We can be sure that a class has only a single instance.
2. We gain a global access point to that instance.
3. The Singleton Object is initialized only when it's requested for the first time.

Intent

There are only two points in the definition of a singleton design pattern,

- 1) There should be only one instance allowed for a class and
- 2) We should allow global point of access to that single instance.

Ensure a class only has one instance, and provide a global point of access to it.



The government is an excellent example of the Singleton pattern.

A country can have only one official government.

Regardless of the personal identities of the individuals who form governments, the title, “The Government of X”, is a global point of access that identifies the group of people in charge.

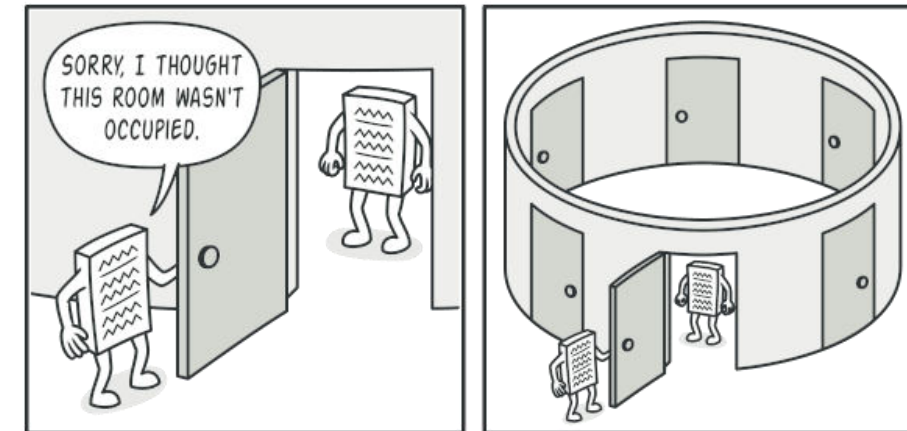
Problem

The Singleton pattern solves two problems at the same time, violating the Single Responsibility Principle:

1. Ensure that a class has just a single instance. Why would anyone want to control how many instances a class has? The most common reason for this is to control access to some shared resource—for example, a database or a file.

Here's how it works: imagine that you created an object, but after a while decided to create a new one. Instead of receiving a fresh object, you'll get the one you already created.

Note that this behavior is impossible to implement with a regular constructor since a constructor call must always return a new object by design.



Clients may not even realize that they're working with the same object all the time.

Singleton: Class, Object Structural

Problem

2. Provide a global access point to that instance. Remember those global variables that you (all right, me) used to store some essential objects? While they're very handy, they're also very unsafe since any code can potentially overwrite the contents of those variables and crash the app.

Just like a global variable, the Singleton pattern lets you access some object from anywhere in the program. However, it also protects that instance from being overwritten by other code.

There's another side to this problem: you don't want the code that solves problem #1 to be scattered all over your program. It's much better to have it within one class, especially if the rest of your code already depends on it.

Singleton: Class, Object Structural

Solution

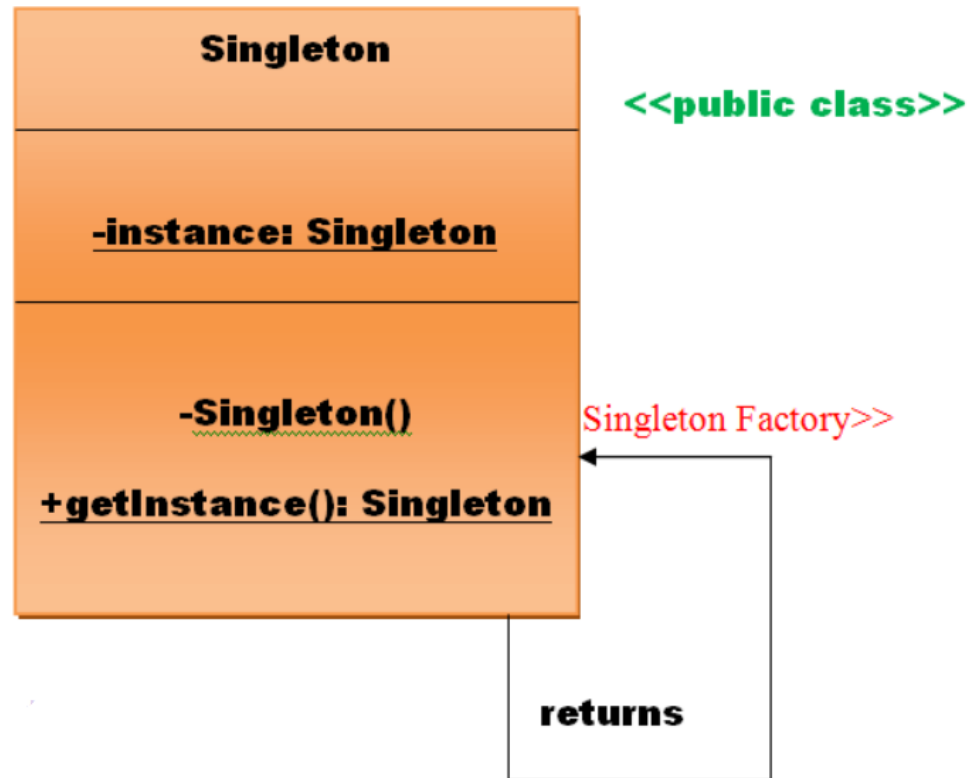
All implementations of the Singleton have these two steps in common:

- ☐ Make the default constructor private, to prevent other objects from using the new operator with the Singleton class.
- ☐ Create a static creation method that acts as a constructor. Under the hood, this method calls the private constructor to create an object and saves it in a static field. All following calls to this method return the cached object.

If your code has access to the Singleton class, then it's able to call the Singleton's static method. So whenever that method is called, the same object is always returned.

Singleton: Implementation

UML class diagram for the Singleton Pattern



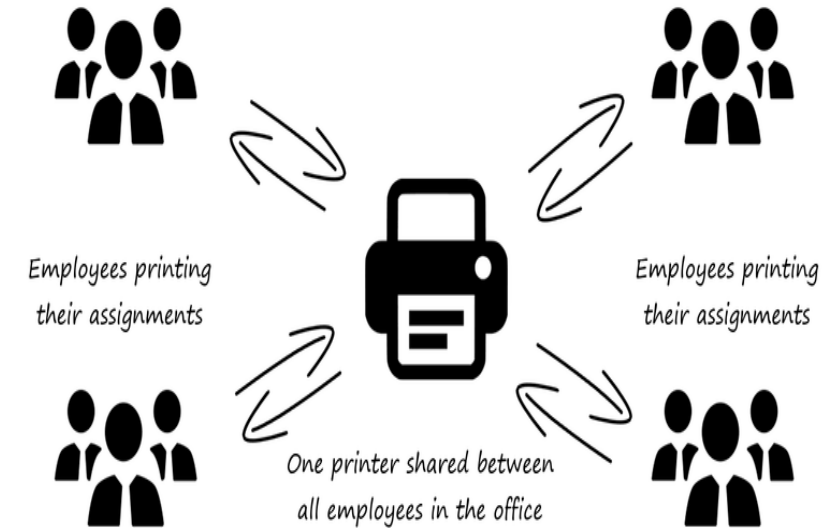
Singleton pattern is used for logging, drivers objects, caching, and thread pool.

Singleton design pattern is also used in other design patterns like Abstract Factory, Builder, Prototype, Facade, etc.

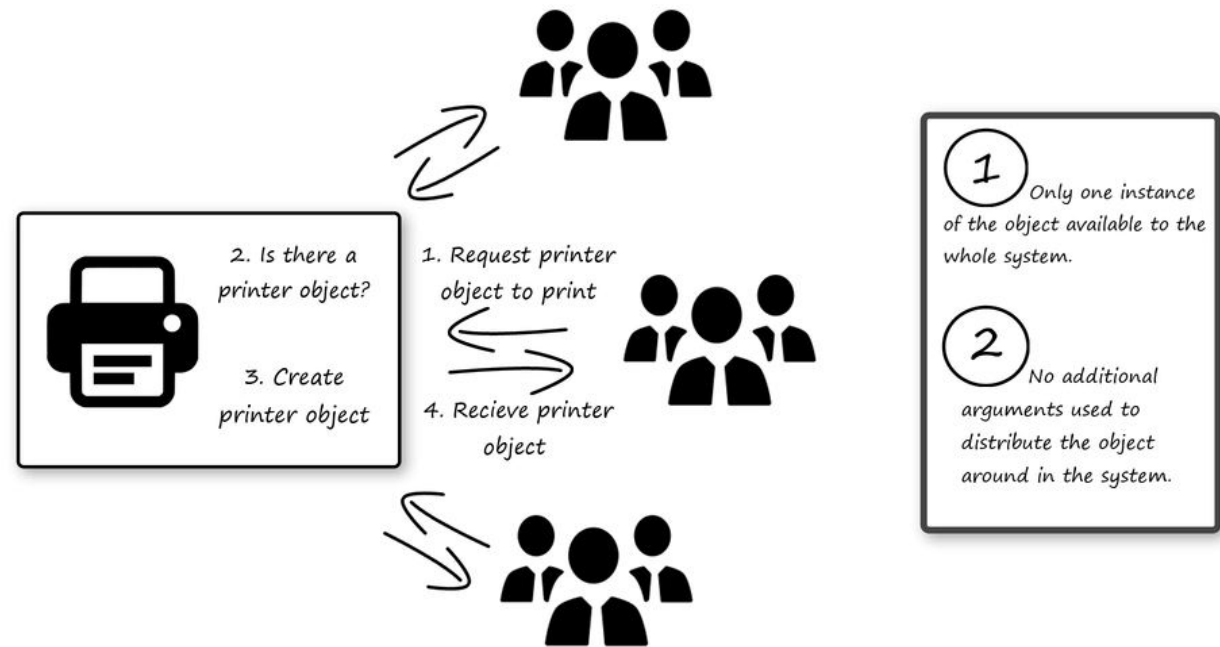
Singleton design pattern is used in core Java classes also (for example, `java.lang.Runtime`, `java.awt.Desktop`).

❑ Problem Statement:

How can a shared office space with many employees, share the same printer. This makes sense, since it is often the reality for corporations and companies with employees sharing office facilities.



- ❑ The concept of the singleton design pattern is based around a private constructor and a public static initialization method.
- ❑ The private constructor ensures that the class can only be initialized by itself, which makes the public static initialization method the only way of getting an instance of the class.

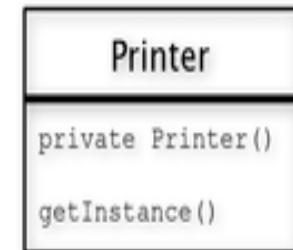


Design Solution

- ❑ However, we do not want the method to start a new instance for each time it is called. Therefore, we save the instance of the class in a static field inside its own class.
- ❑ Now that we have an instance of the class ready to work with inside the class, we null check on it when we call the static initialization method. If the class has not previously been initialized, then we initialize it, otherwise we return our static field instance.
- ❑ This means that there will always only be one instance of the class available, hence the name *singleton* pattern.

Class Diagram (UML)

- ❑ In terms of class diagram, the singleton pattern is not worth talking about. As you might have figured out by now, the singleton pattern is based around two methods inside the same class, which makes the class diagram simple to understand.
- ❑ Thereby said, that if you understand the concept of the singleton pattern from the previous section, you should be ready to dive into the code.



Printer

```
public class Printer {
    private static Printer printer;
    private int nrOfPages;
    private Printer() {
    }
    public static Printer getInstance() {
        return printer == null ?
            printer = new Printer() :
            printer;
    }
    public void print(String text){
        System.out.println(text +
            "n" + "Pages printed today " + ++nrOfPages +
            "n" + "-----");
    }
}
```

Employee

```
public class Employee {
    private final String name;
    private final String role;
    private final String assignment;
    public Employee(String name, String role, String assignment) {
        this.name = name;
        this.role = role;
        this.assignment = assignment;
    }
    public void printCurrentAssignment(){
        Printer printer = Printer.getInstance();
        printer.print("Employee: " + name + "n" +
            "Role: " + role + "n" +
            "Assignment: " + assignment + "n");
    }
}
```

Code (Java)

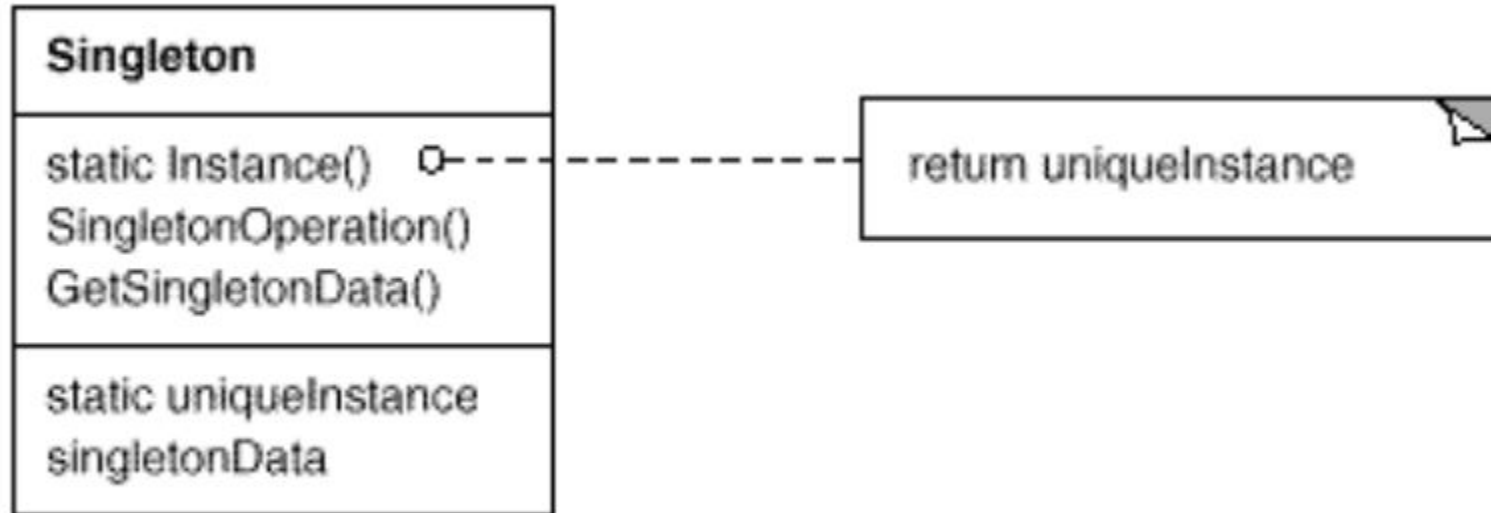
How To Use The Singleton Pattern

```
public class Main {  
    public static void main(String[] args) {  
  
        Employee graham = new Employee("Graham", "CEO", "Making executive decisions");  
        Employee sara = new Employee("Sara","Consultant", "Consuting the company");  
        Employee tim = new Employee("Tim","Salesmen", "Selling the company's products");  
        Employee emma = new Employee("Emma", "Developer", "Developing the latest mobile app.");  
  
        graham.printCurrentAssignment();  
        sara.printCurrentAssignment();  
        tim.printCurrentAssignment();  
        emma.printCurrentAssignment();  
    }  
}
```




Use the Singleton pattern when

- ☐ there must be exactly one instance of a class, and it must be accessible to clients from a well-known access point.
- ☐ when the sole instance should be extensible by subclassing, and clients should be able to use an extended instance without modifying their code.



- ❑ Singleton design pattern has two core participants: Singleton and Client.
- ❑ defines an Instance operation that lets clients access its unique instance. Instance is a class operation (that is, a class method and a static member function).
- ❑ may be responsible for creating its own unique instance.



Clients access a Singleton instance solely through Singleton's Instance operation.

Consequence

The Singleton pattern has several benefits:

1. Controlled access to sole instance. Because the Singleton class encapsulates its sole instance, it can have strict control over how and when clients access it.
1. Reduced name space. The Singleton pattern is an improvement over global variables. It avoids polluting the name space with global variables that store sole instances.
1. Permits refinement of operations and representation. The Singleton class may be subclassed, and it's easy to configure an application with an instance of this extended class. You can configure the application with an instance of the class you need at run-time.

Consequence

The Singleton pattern has several benefits:

4. Permits a variable number of instances. The pattern makes it easy to change your mind and allow more than one instance of the Singleton class. Moreover, you can use the same approach to control the number of instances that the application uses. Only the operation that grants access to the Singleton instance needs to change
5. More flexible than class operations. Another way to package a singleton's functionality is to use class operations (that is, static member functions or class methods). But both of these language techniques make it hard to change a design to allow more than one instance of a class. Moreover, static member functions in class are compile time, so subclasses can't override them polymorphically.

Implementation

Java Singleton Pattern Implementation

1. Add a private static field to the class for storing the singleton instance.
2. Declare a public static creation method for getting the singleton instance.
3. Implement “lazy initialization” inside the static method. It should create a new object on its first call and put it into the static field. The method should always return that instance on all subsequent calls.
4. Make the constructor of the class private. The static method of the class will still be able to call the constructor, but not the other objects.
5. Go over the client code and replace all direct calls to the singleton’s constructor with calls to its static creation method.

various design options for implementing Singleton:

Method 1: lazy instantiation

```
// Classical Java implementation of singleton design pattern
class Singleton
{
    private static Singleton obj;

    // private constructor to force use of
    // getInstance() to create Singleton object
    private Singleton() {}

    public static Singleton getInstance()
    {
        if (obj==null)
            obj = new Singleton();
        return obj;
    }
}
```

- ❑ Here we have declared getInstance() static so that we can call it without instantiating the class.
- ❑ The first time getInstance() is called it creates a new singleton object and after that it just returns the same object.
- ❑ Note that Singleton obj is not created until we need it and call getInstance() method. This is called lazy instantiation.

Implementation

The main problem with above method is that it is not thread safe. Consider the following execution sequence.

Thread one

```
public static Singleton getInstance(){  
    if(obj==null)  
  
    obj=new Singleton();  
    return obj;  
}
```

Thread two

```
public static Singleton getInstance(){  
    if(obj==null)  
  
    obj=new Singleton();  
    return obj;  
}
```

This execution sequence creates two objects for singleton. Therefore this classic implementation is not thread safe.

Method 2: make getInstance() synchronized

```
// Thread Synchronized Java implementation of
// singleton design pattern
class Singleton
{
    private static Singleton obj;

    private Singleton() {}

    // Only one thread can execute this at a time
    public static synchronized Singleton getInstance()
    {
        if (obj==null)
            obj = new Singleton();
        return obj;
    }
}
```

- ❑ Here using synchronized makes sure that only one thread at a time can execute getInstance().
- ❑ The main disadvantage of this is method is that using synchronized every time while creating the singleton object is expensive and may decrease the performance of your program.
- ❑ However if performance of getInstance() is not critical for your application this method provides a clean and simple solution.

Method 3: Eager Instantiation

```
// Static initializer based Java implementation of
// singleton design pattern
class Singleton
{
    private static Singleton obj = new Singleton();

    private Singleton() {}

    public static Singleton getInstance()
    {
        return obj;
    }
}
```

- ❑ Here we have created instance of singleton in static initializer.
- ❑ JVM executes static initializer when the class is loaded and hence this is guaranteed to be thread safe.
- ❑ Use this method only when your singleton class is light and is used throughout the execution of your program.

Method 4 (Best): Use “Double Checked Locking”

- ❑ By considering method 2, once an object is created, the synchronization is no longer useful because obj will not be null and any sequence of operations will lead to consistent results.
- ❑ So we will only acquire lock on the getInstance() once, when the obj is null.
- ❑ This way we only synchronize just what we want.

```
// Double Checked Locking based Java implementation of
// singleton design pattern
class Singleton
{
    private static volatile Singleton obj = null;

    private Singleton() {}

    public static Singleton getInstance()
    {
        if (obj == null)
        {
            // To make thread safe
            synchronized (Singleton.class)
            {
                // check again as multiple threads
                // can reach above step
                if (obj==null)
                    obj = new Singleton();
            }
        }
        return obj;
    }
}
```

- ❑ We have declared the obj volatile which ensures that multiple threads offer the obj variable correctly when it is being initialized to Singleton instance.
- ❑ This method drastically reduces the overhead of calling the synchronized method every time.

PROS

1. You can be sure that a class has only a single instance.
2. You gain a global access point to that instance.
3. The singleton object is initialized only when it's requested for the first time.

CONS

1. Violates the Single Responsibility Principle. The pattern solves two problems at the time.
2. The Singleton pattern can mask bad design, for instance, when the components of the program know too much about each other.
3. The pattern requires special treatment in a multithreaded environment so that multiple threads won't create a singleton object several times.
4. It may be difficult to unit test the client code of the Singleton because many test frameworks rely on inheritance when producing mock objects.

- ☐ A **Facade** class can often be transformed into a **Singleton** since a single facade object is sufficient in most cases.
- ☐ **Flyweight** would resemble **Singleton** if you somehow managed to reduce all shared states of the objects to just one flyweight object. But there are two fundamental differences between these patterns:
 - ☐ There should be only one Singleton instance, whereas a Flyweight class can have multiple instances with different intrinsic states.
 - ☐ The Singleton object can be mutable. Flyweight objects are immutable.
- ☐ **Abstract Factories**, **Builders** and **Prototypes** can all be implemented as **Singltons**.
 - ☐ Facade objects are often Singletons because only one Facade object is required.

Object Oriented Analysis and Design

Demo



DEMO

MCQs

Q1. Which of the following is the primary purpose of the Singleton design pattern?

- A. To allow multiple instances of a class
- B. To restrict a class to only one instance and provide global access
- C. To improve code readability
- D. To separate interface from implementation

Q2. Which keyword is essential in implementing Double-Checked Locking correctly?

- A. static
- B. final
- C. volatile
- D. transient

MCQs

Q3. What is the role of a private constructor in Singleton?

- A. To allow subclassing
- B. To prevent object creation from outside the class
- C. To improve performance
- D. To enable inheritance

Q4. What is a major drawback of using Singleton excessively?

- A. Increased memory usage
- B. Difficulty in unit testing due to tight coupling
- C. Slower execution speed
- D. Lack of encapsulation

Object Oriented Analysis and Design

MCQs - Answers

Q1. B

Q2. C

Q3. B

Q4. B

Text Reference

- Design Patterns: Elements of Reusable Object-Oriented Software, GOF

Web Reference

<https://integu.net/singleton-pattern/>

<https://refactoring.guru/design-patterns/singleton>

<https://www.geeksforgeeks.org/singleton-design-pattern/>

<https://refactoring.guru/design-patterns/singleton/java/example>

https://sourcemaking.com/design_patterns/singleton



THANK YOU

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

shridevias@pes.edu



Object Oriented Analysis and Design

UE23CS352B

Prof Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

UE23CS352B: Object Oriented Analysis and Design

OO Design - Creational Design Patterns

Factory

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements:

Prof. Nivedita Kasturi

Dept. of Computer Science and Engineering

- ☐ Factory-definition
- ✓ Motivation
- ✓ Intent
- ✓ Implementation
- ✓ Applicability
- ✓ Structure-Consequence
- ✓ Issues

Introduction

The Factory Design Pattern or Factory Method Design Pattern is one of the most used design patterns in Java.

According to GoF, this pattern **“defines an interface for creating an object, but let subclasses decide which class to instantiate.** The Factory method lets a class defer instantiation to subclasses”.

This pattern delegates the responsibility of initializing a class from the client to a particular factory class by creating a type of virtual constructor.

To achieve this, we rely on a factory which provides us with the objects, hiding the actual implementation details. The created objects are accessed using a common interface.

Also Known As Virtual
Constructor

Factory: Class, Object Structural

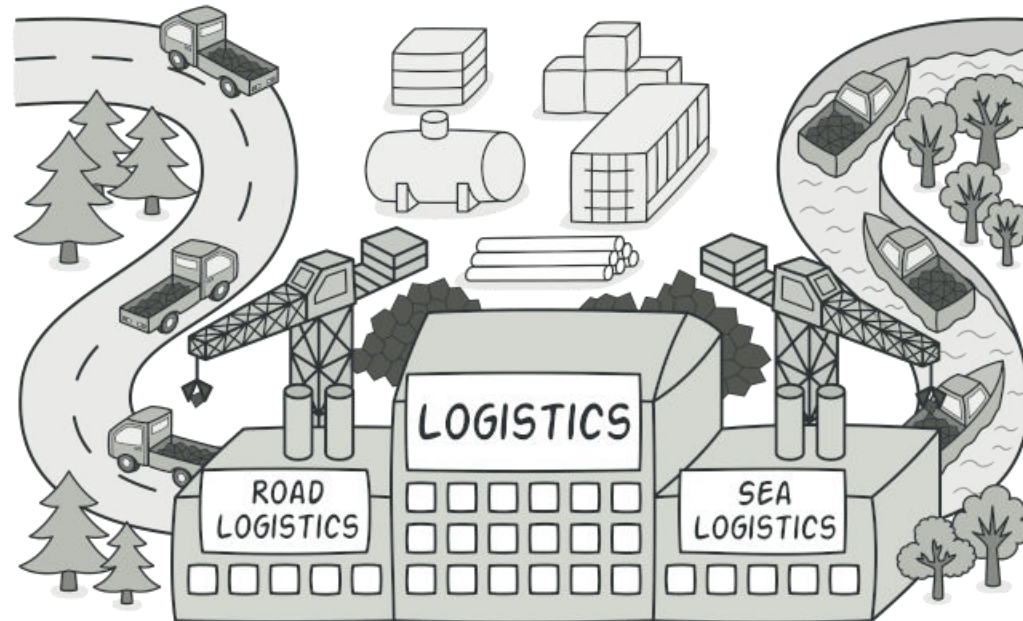
Motivation

- ❑ To deal with the problem of creating objects without having to specify the exact class of the object that will be created.
- ❑ Creating an object often requires complex processes not appropriate to include within a composing object.
- ❑ The object's creation may lead to a significant duplication of code, may require information not accessible to the composing object, may not provide a sufficient level of abstraction, or may otherwise not be part of the composing object's concerns.
- ❑ The factory method design pattern handles these problems by defining a separate method for creating the objects, which subclasses can then override to specify the derived type of product that will be created.

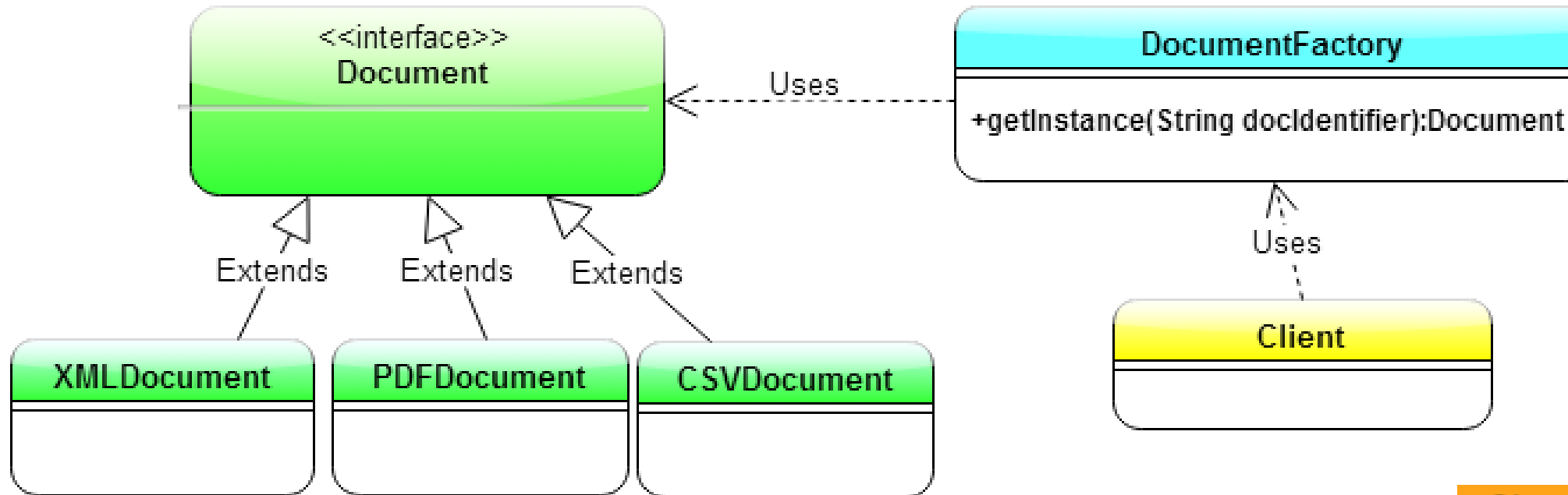
Factory: Class, Object Structural

Intent

Factory Method is a creational design pattern that provides an interface for creating objects in a superclass, but allows subclasses to alter the type of objects that will be created.



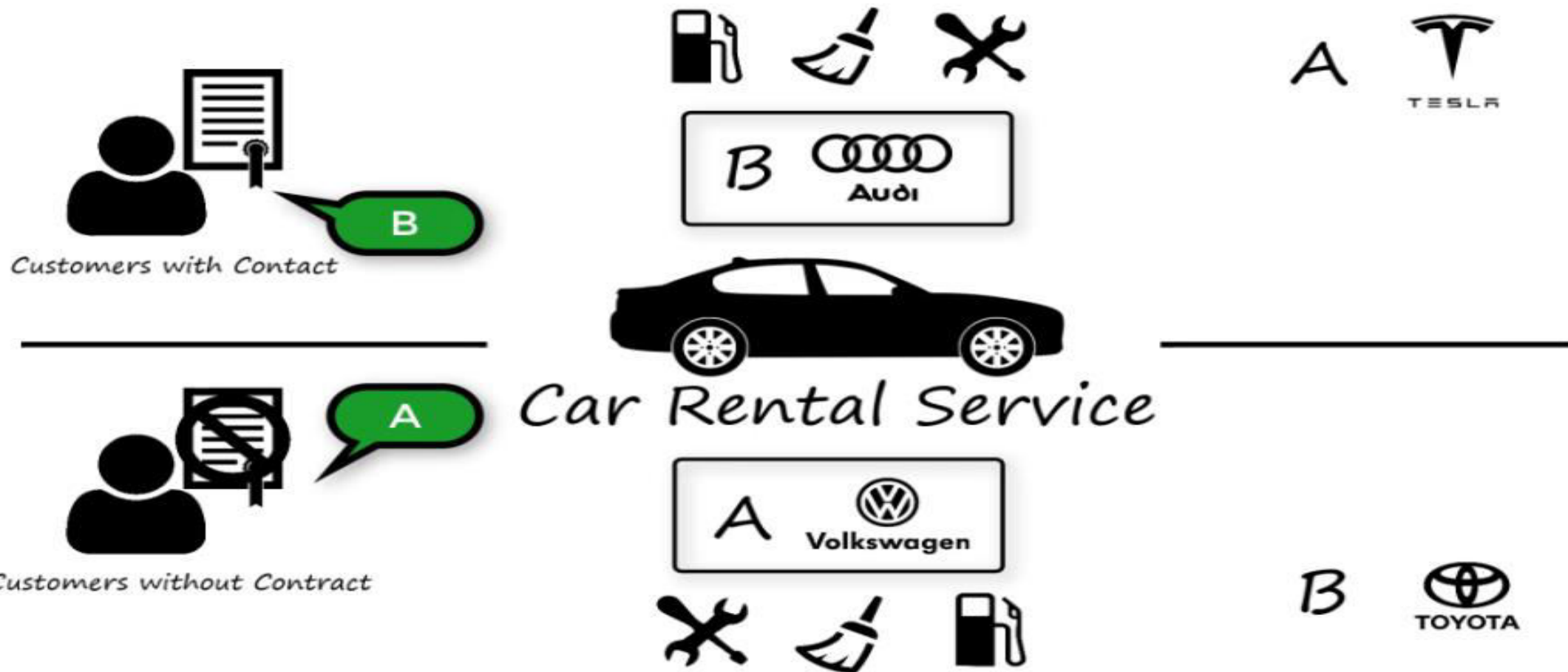
UML class diagram for the Factory Pattern



Object Oriented Analysis and Design

Factory: Implementation example-1

Problem Statement:



Solution

Open Creation

As a beginning, we start by checking whether our customer has a company contract. If so, we retrieve their requested grade of car and runs through the options to see if we have a match.

For this scenario, we have already a Car super-class, which can be extended by all car brands sub-classes (Tesla, Audi, Volkswagen, and Toyota). This allows us to utilize the common methods of the Car super-class to perform a mechanical service check, clean up the car, and put fuel on it.

All of the above mentioned steps have all be accomplished inside the main-method of our system, as can be seen from the code sample below.

Solution

Although this code sample solves the objective, it has multiple problems.

Firstly, it is not desirable to have all the car selection and preparation logics at the same level of abstraction as where the customer is. We want that kind of logic hidden away from the customer so that they only see the ready and shiny car. In code practice we also want this logic hidden away in its own class, to avoid modifications, while working on other parts of the main-method.

Secondly, we also want each contract type to be handled individually. Let's say that the rental service one day shut down their company contract agreement. In this case, we want to easily be able to remove the logics related to the company contracts, without jeopardizing changes anywhere else in the system. With these issues in mind, let's see what the factory pattern can do for us.



1 Creation of the customer- and car object should not be associated

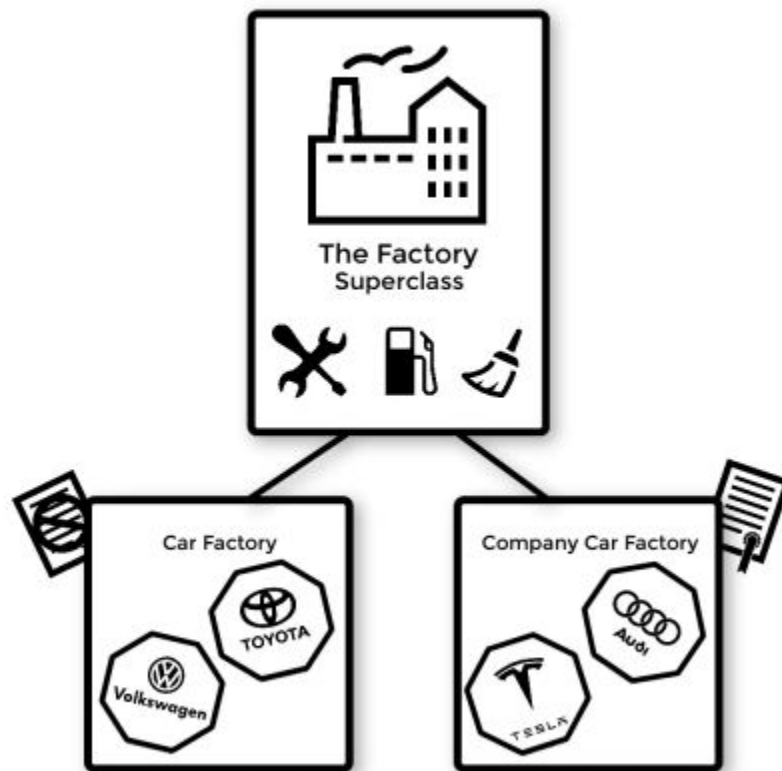
2 Each contract type should be handled individually


```
public static void main(String[] args) {  
  
    Customer customerOne = new Customer("B", true);  
    Car carOne;  
  
    if (customerOne.hasCompanyContract()) {  
  
        switch (customerOne.getGradeRequest()) {  
            case "A":  
                carOne = new Tesla();  
                break;  
  
            case "B":  
                carOne = new Audi();  
                break;  
  
            default:  
                System.out.println("The requested car was  
unfortunately not available.");  
                carOne = null;  
        }  
    }  
}
```

```
} else {  
  
    switch (customerOne.getGradeRequest()) {  
        case "A":  
            carOne = new Volkswagen();  
            break;  
  
        case "B":  
            carOne = new Toyota();  
            break;  
  
        default:  
            System.out.println("The requested car was  
unfortunately not available.");  
            carOne = null;  
    }  
}  
  
carOne.clean();  
carOne.mechanicCheck();  
carOne.fuelCar();  
carOne.startEngine();  
}
```

Solution

Concept Of The Factory Pattern



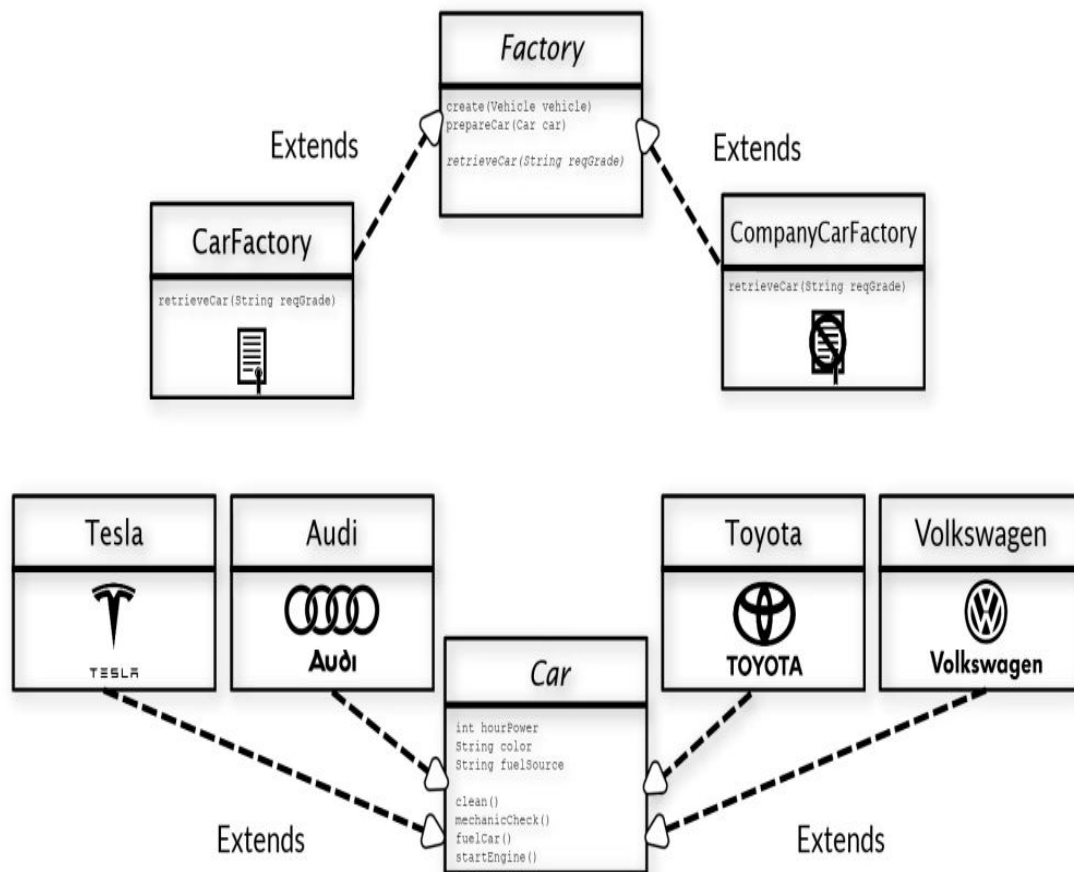
- 1 Each of the contract types are handled individually
- 2 Creation of the car object has been separate from the customer object by the factory



The core concept of the factory pattern is to make a class (the factory), which defines the rules of how to create a specific type of object (the product). The pattern can always be recognized by the create-method.

Solution

Class Diagram (UML)

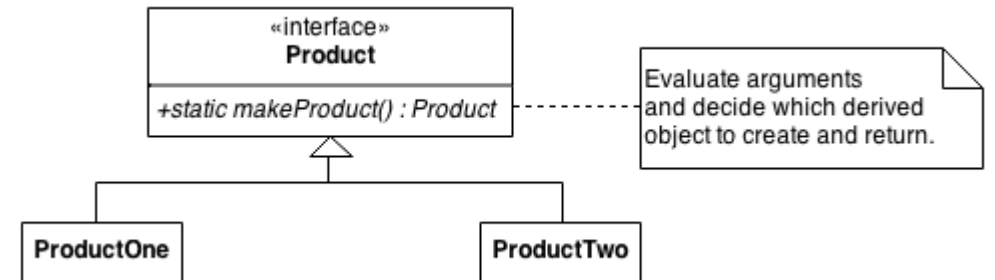
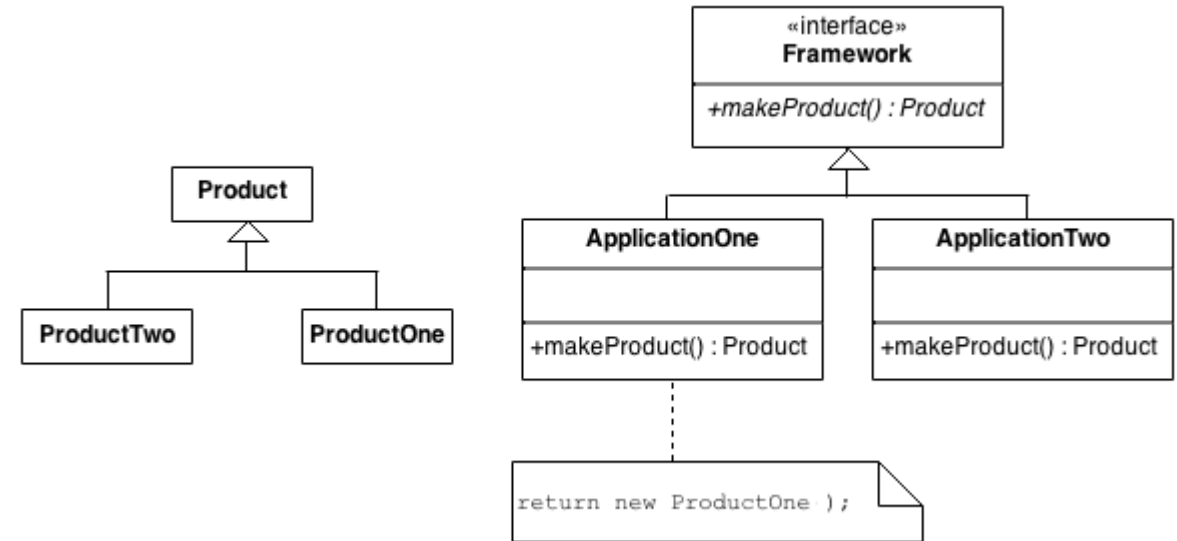
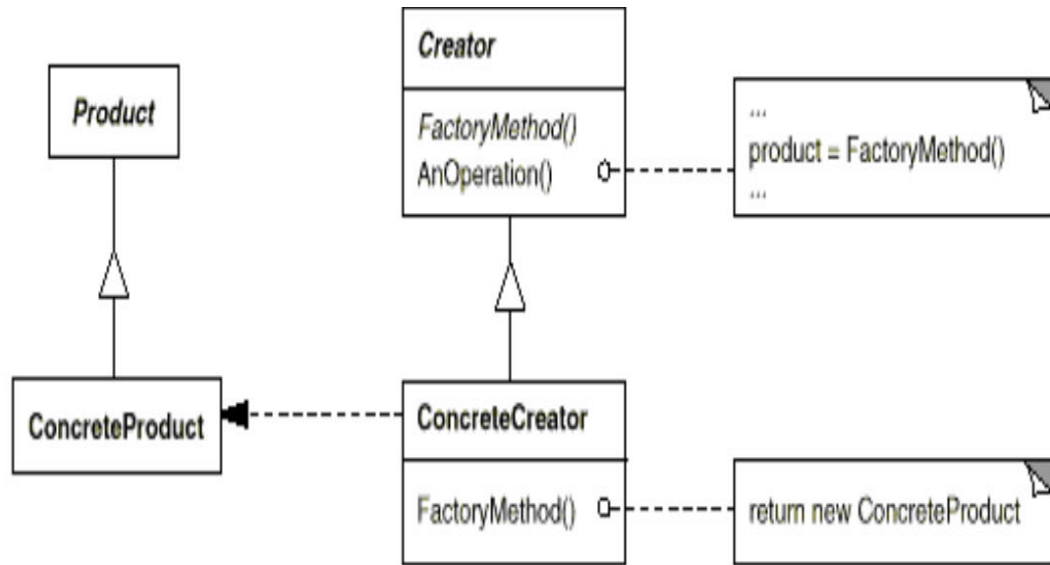


- ❑ The Factory super-class contains the **create-method**, which is close to obligatory for any factory pattern. Within this method the initial step is to retrieve the requested car. However, since the super-class does not know anything about the selection logic, *it simply states the retrieveCar-method as abstract*.
- ❑ Thereby it requires any class, which extends it, must override the `retrieveCar-method` and implement its own selection logic.
- ❑ After having retrieved the car, we come back to the `create-method`. The remaining part of the method then goes into preparation procedures. This procedure is the same for all cars and we can, therefore, state the steps within the factory super-class.

DEMO

Use the Factory pattern when

- ☐ a class can't anticipate the class of objects it must create.
- ☐ a class wants its subclasses to specify the objects it creates.
- ☐ classes delegate responsibility to one of several helper subclasses, and you want to localize the knowledge of which helper subclass is the delegate.



Participants

- ❑ Product -defines the interface of objects the factory method creates.
- ❑ ConcreteProduct - implements the Product interface.
- ❑ Creator - declares the factory method, which returns an object of type Product.Creator may also define a default implementation of the factory method that returns a default ConcreteProduct object.
 - may call the factory method to create a Product object.
- ❑ ConcreteCreator -overrides the factory method to return an instance of a ConcreteProduct.

Creator relies on its subclasses to define the factory method so that it returns an instance of the appropriate ConcreteProduct.

Consequence

1. Provides hooks for subclasses. Creating objects inside a class with a factory method is always more flexible than creating an object directly. Factory Method gives subclasses a hook for providing an extended version of an object.

2. Connects parallel class hierarchies. In the examples we've considered so far, the factory method is only called by Creators. But this doesn't have to be the case; clients can find factory methods useful, especially in the case of parallel class hierarchies.

Parallel class hierarchies result when a class delegates some of its responsibilities to a separate class. Consider graphical figures that can be manipulated interactively; that is, they can be stretched, moved, or rotated using the mouse. Implementing such interactions isn't always easy. It often requires storing and updating information that records the state of the manipulation at a given time.

Pros and Cons

- ✓ You avoid tight coupling between the creator and the concrete products.
- ✓ *Single Responsibility Principle*. You can move the product creation code into one place in the program, making the code easier to support.
- ✓ *Open/Closed Principle*. You can introduce new types of products into the program without breaking existing client code.
- ✗ The code may become more complicated since you need to introduce a lot of new subclasses to implement the pattern. The best case scenario is when you're introducing the pattern into an existing hierarchy of creator classes.

Relationship with other pattern

↔ Relations with Other Patterns

- Many designs start by using Factory Method (less complicated and more customizable via subclasses) and evolve toward Abstract Factory, Prototype, or Builder (more flexible, but more complicated).
- Abstract Factory classes are often based on a set of Factory Methods, but you can also use Prototype to compose the methods on these classes.
- You can use Factory Method along with Iterator to let collection subclasses return different types of iterators that are compatible with the collections.
- Prototype isn't based on inheritance, so it doesn't have its drawbacks. On the other hand, *Prototype* requires a complicated initialization of the cloned object. Factory Method is based on inheritance but doesn't require an initialization step.
- Factory Method is a specialization of Template Method. At the same time, a *Factory Method* may serve as a step in a large *Template Method*.

Issues to consider when using the Factory pattern

1. Two major varieties. The two main variations of the Factory Method pattern are (1) the case when the Creator class is an abstract class and does not provide an implementation for the factory method it declares, and (2) the case when the Creator is a concrete class and provides a default implementation for the factory method. It's also possible to have an abstract class that defines a default implementation, but this is less common. The first case requires subclasses to define an implementation, because there's no reasonable default.

2. Parameterized factory methods. Another variation on the pattern lets the factory method create multiple kinds of products. The factory method takes a parameter that identifies the kind of object to create. All objects the factory method creates will share the Product interface.

3. Language-specific variants and issues. Different languages lend themselves to other interesting variations and caveats. Smalltalk programs often use a method that returns the class of the object to be instantiated. A Creator factory method can use this value to create a product, and a ConcreteCreator may store or even compute this value. The result is an even later binding for the type of ConcreteProduct to be instantiated.

Object Oriented Analysis and Design

Demo



DEMO -

MCQ

1. What is the main purpose of the Factory Method pattern?

- A. To hide database queries
- B. To define an interface for object creation while allowing subclasses to decide the class to instantiate
- C. To increase coupling
- D. To remove inheritance

2. Factory Method is also known as:

- A. Static Constructor
- B. Virtual Constructor
- C. Smart Constructor
- D. Dynamic Builder

MCQ

3. In Factory pattern, the client accesses objects using:

- A. Direct constructor calls
- B. Global objects
- C . Static classes
- D. A common interface

4. Factory pattern mainly helps to:

- A. Increase complexity
- B. Hide object creation logic
- C. Remove abstraction
- D. Avoid inheritance

MCQ

5. Factory Method improves flexibility by:

- A. Removing classes
- B. Delegating object creation to subclasses
- C. Using static variables
- D. Avoiding interfaces

6. The factory method usually returns:

- A. Objects of Product interface
- B. Primitive data types
- C. Only concrete classes
- D. Database connections

MCQ SOLUTION

1.B

2.B

3.D

4.B

5.B

6.A

Text Reference

- Design Patterns: Elements of Reusable Object-Oriented Software, GOF

Web Reference

https://www.tutorialspoint.com/design_pattern/factory_pattern.htm

<https://www.javatpoint.com/factory-method-design-pattern>

<https://refactoring.guru/design-patterns/factory-method>

<https://www.geeksforgeeks.org/factory-method-design-pattern-in-java/>

<https://www.digitalocean.com/community/tutorials/factory-design-pattern-in-java>

<https://www.baeldung.com/java-factory-pattern>

https://sourcemaking.com/design_patterns/factory_method

<https://www.scaler.com/topics/factory-design-pattern-in-java/>



THANK YOU

Prof Shridevi A Sawant

Department of Computer Science and Engineering

shridevias@pes.edu



Object Oriented Analysis and Design

UE23CS352B

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

UE23CS352B: Object Oriented Analysis and Design

OO Design – Creational Design Patterns

Builder

Prof Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements:

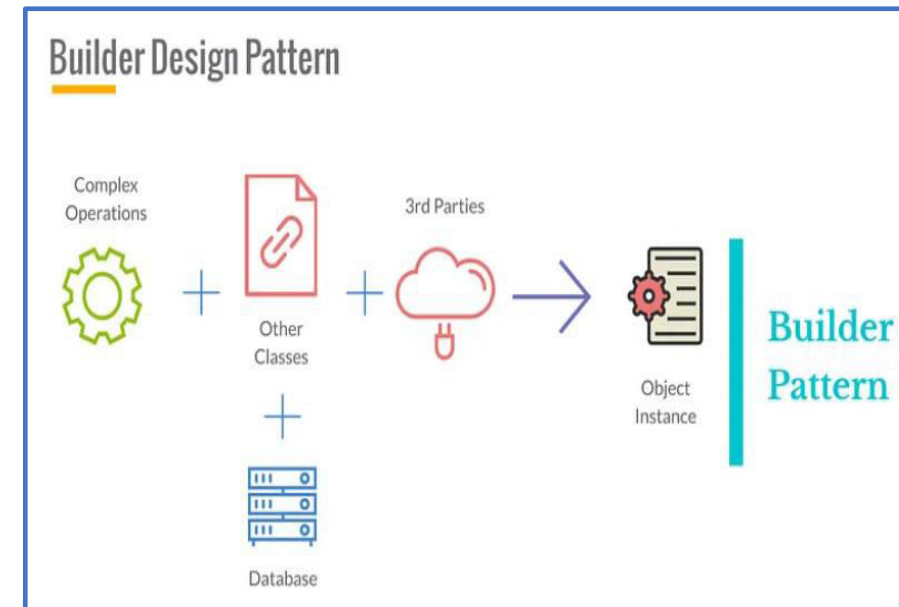
Prof. Nivedita Kasturi

Dept. of Computer Science and Engineering

- ☐ Builder-definition
- ✓ Motivation
- ✓ Intent
- ✓ Implementation
- ✓ Applicability
- ✓ Structure-Consequence
- ✓ Issues

Introduction

- ❑ **Builder Design Pattern** as it was intended implies that a sequence of complex operations is needed in order to **produce an object instance**.
- ❑ The idea is that we delegate the construction of an object to a specialized class, which is aware of the complex process and logic required to build that specific instance.
- ❑ This helps us create Single Responsibility classes for complex object creation while at the same time ensuring separation of object creation from business logic.
- ❑ Based on the nature of the application, Builder implementations might lead to code re-usability, reducing the code base and improving SOLID compliance of our code.



Definition:

The Builder Design Pattern is another creational pattern designed to deal with the construction of comparatively complex objects.

When the complexity of creating object increases, the Builder pattern can separate out the instantiation process by using another object (a builder) to construct the object.

This builder can then be used to create many other similar representations using a simple step-by-step approach.

Why Builder Pattern?

The Builder design pattern solves problems like:

- ☐ How can a class (the same construction process) create different representations of a complex object?
- ☐ How can a class that includes creating a complex object be simplified?

The Builder design pattern describes how to solve such problems:

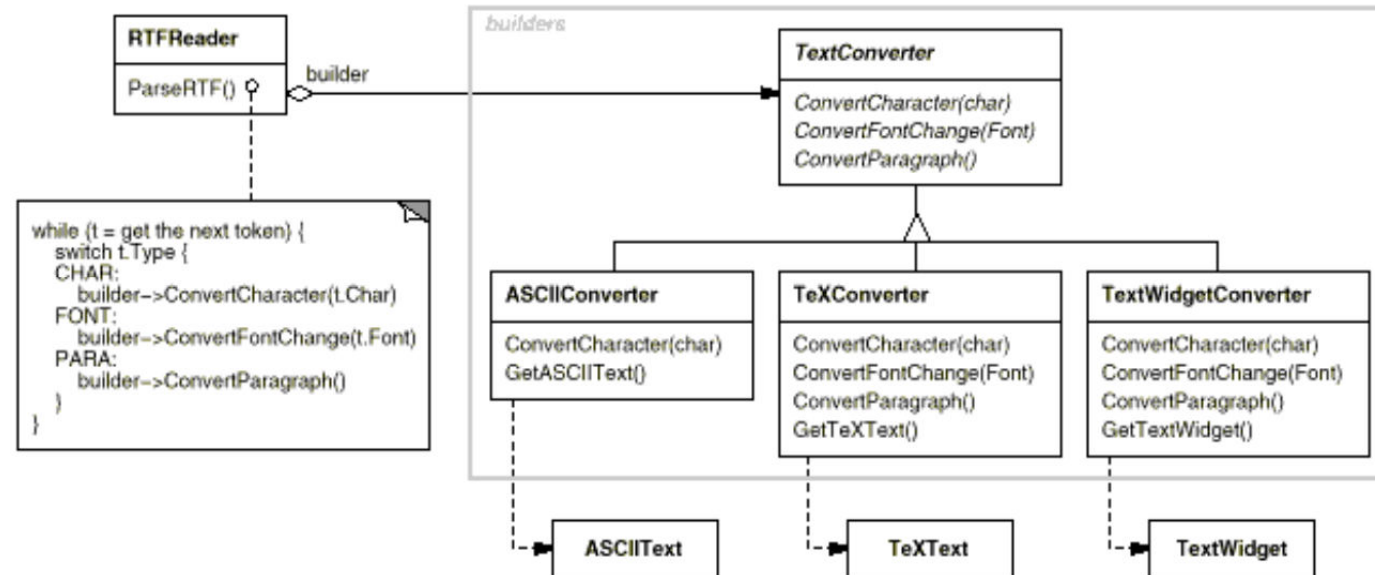
- ☐ Encapsulate creating and assembling the parts of a complex object in a separate Builder object.
- ☐ A class delegates object creation to a Builder object instead of creating the objects directly.

A class (the same construction process) can delegate to different Builder objects to create different representations of a complex object.

Builder : Class, Object Structural

Motivation

- ❑ A reader for the RTF (Rich Text Format) document exchange format should be able to convert RTF to many text formats. The reader might convert RTF documents into plain ASCII text or into a text widget that can be edited interactively.
- ❑ A **solution** is to configure the RTFReader
- ❑ The **problem**, however, is that the number of possible conversions is open-ended. So it should be easy to add a new conversion without modifying the reader.
- ❑ class with a TextConverter object that converts RTF to another textual representation.
- ❑ The Builder pattern captures all these relationships. **Each converter class is called a builder** in the pattern, and the reader is called the **director**.

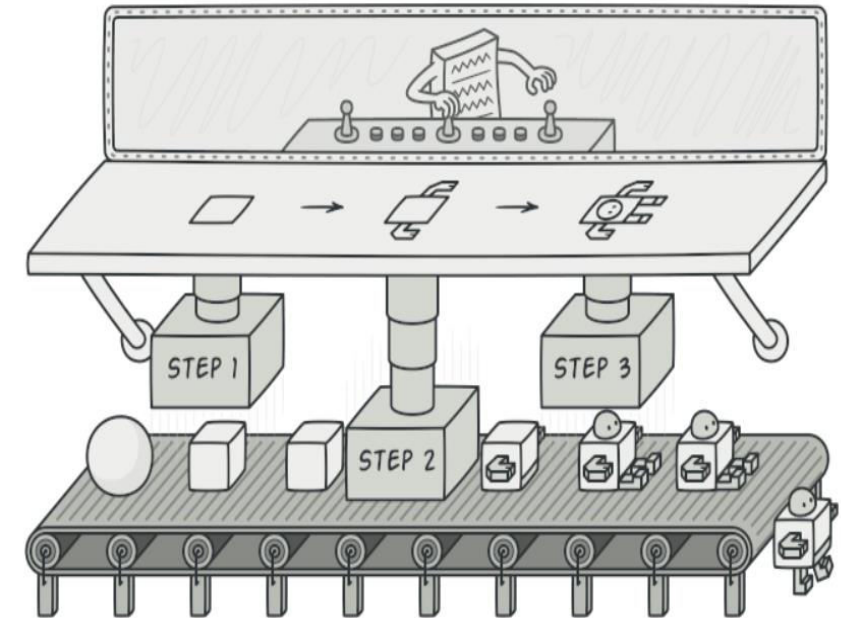


Intent

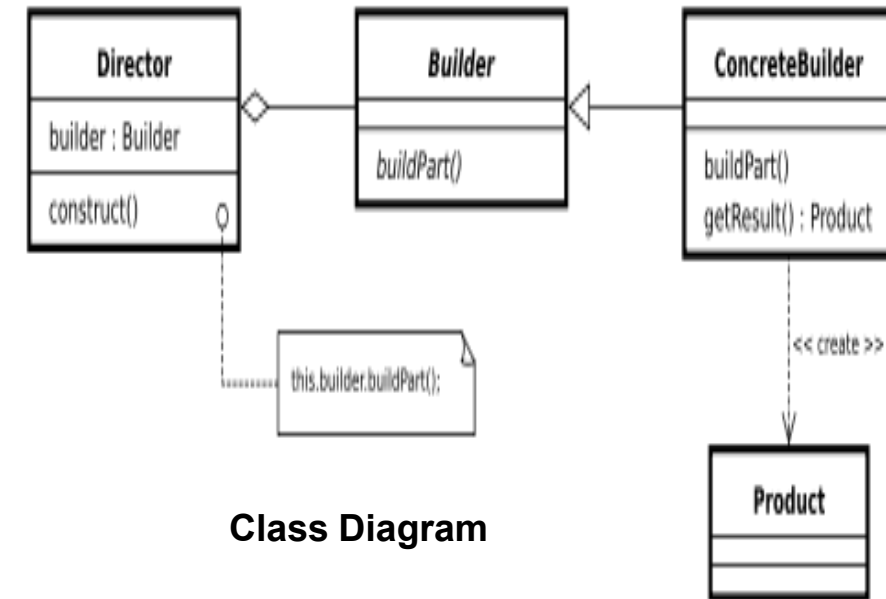
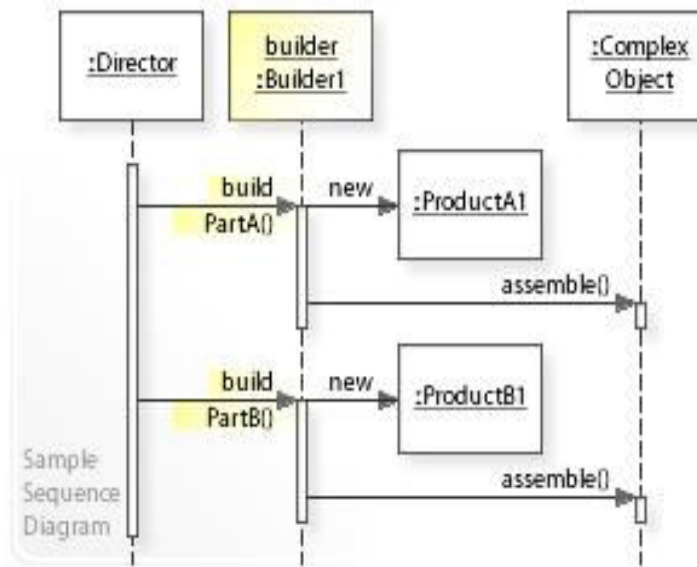
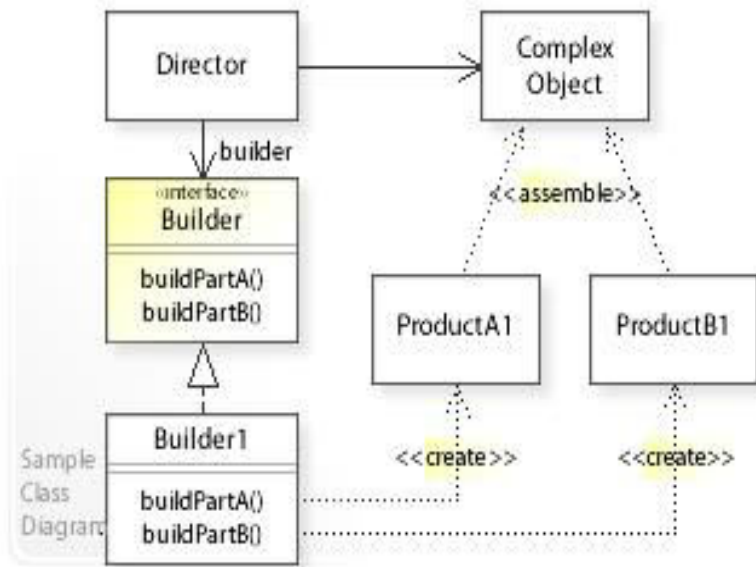
Separate the construction of a complex object from its representation so that the same construction process can create different representations

Think of a car factory

Boss tells workers (or robots) to build each part of a car
Workers build each part and add them to the car being constructed



UML class diagram for the Builder Pattern



Builder

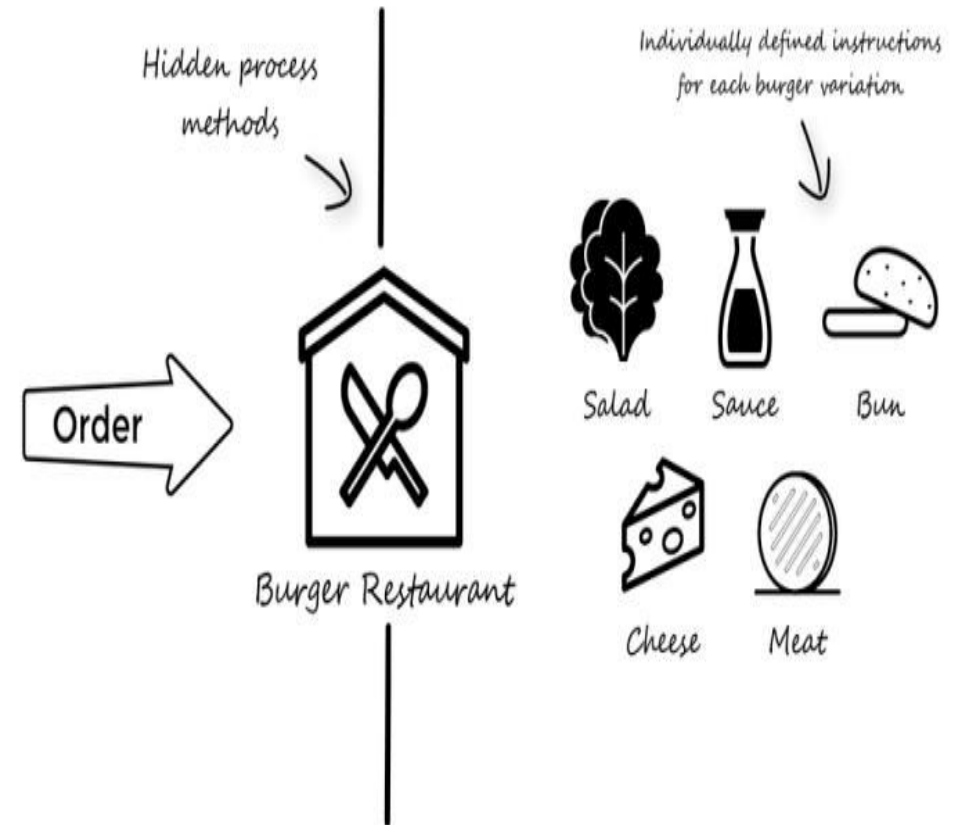
Abstract interface for creating objects (product).

ConcreteBuilder

Provides implementation for Builder. It is an object able to construct other objects. Constructs and assembles parts to build the objects.

Problem Statement: Case study

- ❑ We will set the objective of making a burger restaurant, which can make different variations of burgers.
- ❑ Secondly it would also be preferable to have defined instructions for how to build each of the different variations of burgers (e.g. cheeseburger), so that we do not have to provide all the ingredients each time when we are making a burger.
- ❑ Before, how the builder design pattern will be able to solve these issues, we will start by seeing how this system could be build based on construction of the burger object in the main context.

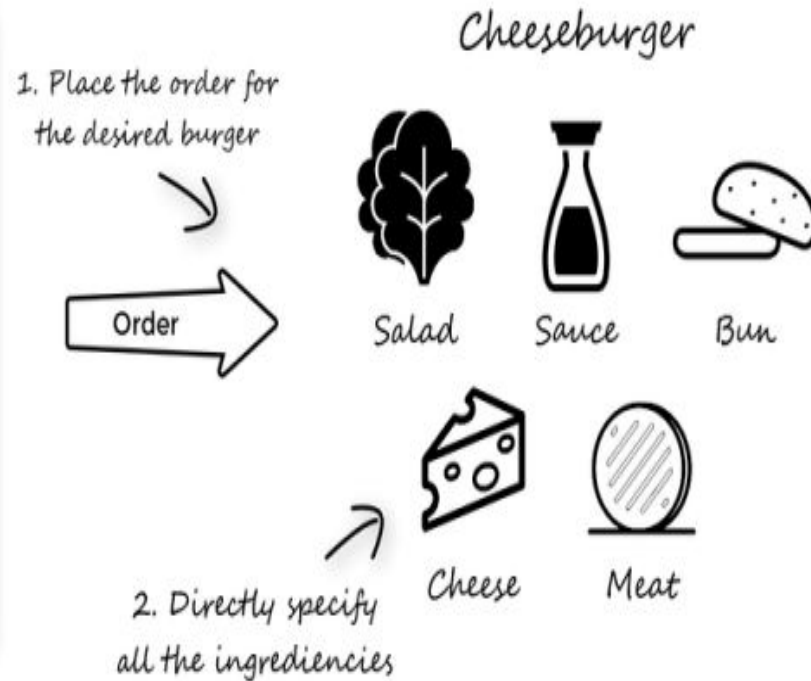


Object Oriented Analysis and Design

Builder : Implementation example-1

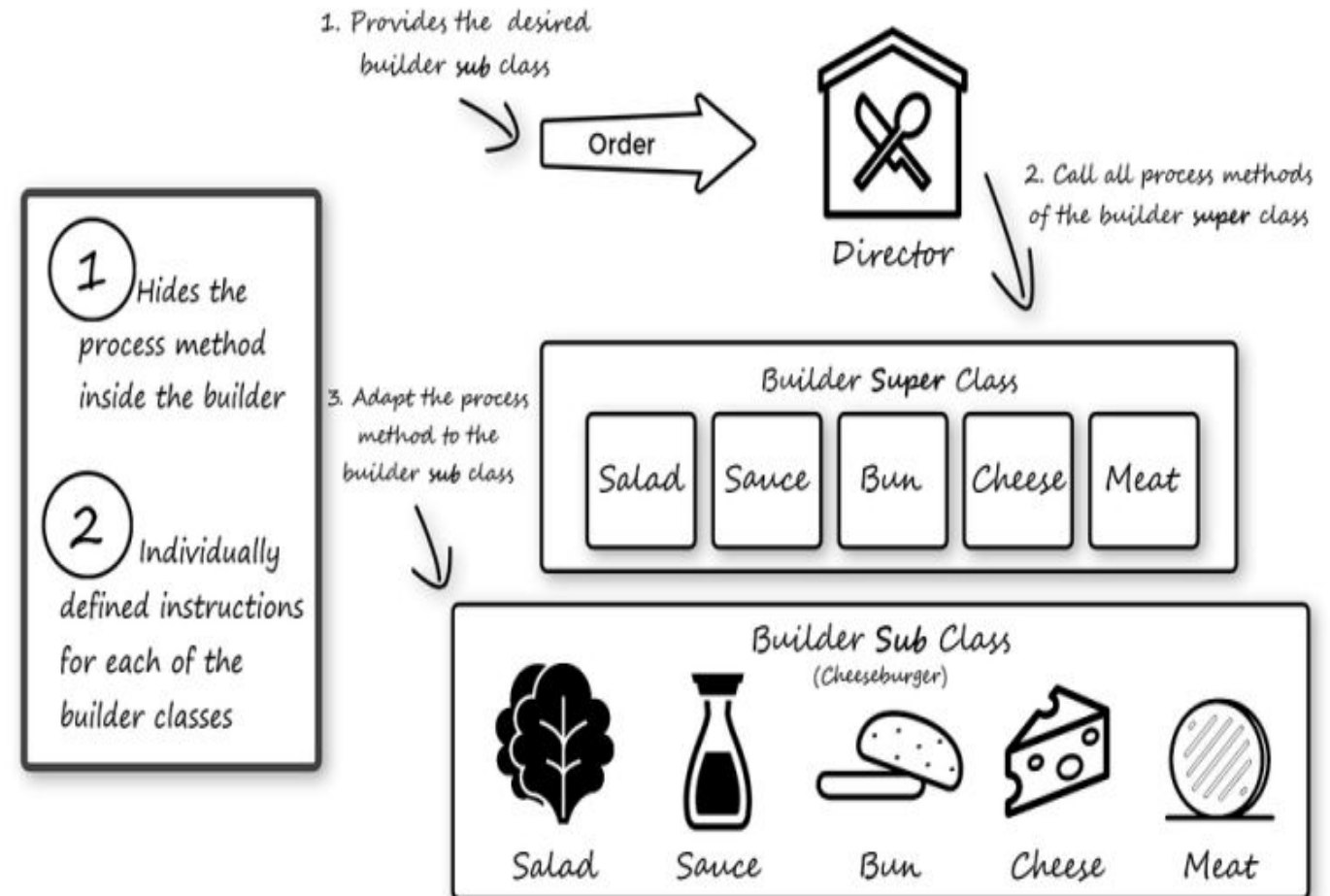
Solution: Construction In The Main Context

- 1 All the process methods are out in the open and visible to anyone in the main context
- 2 Instructions for how to build each burger have to be supplied every time



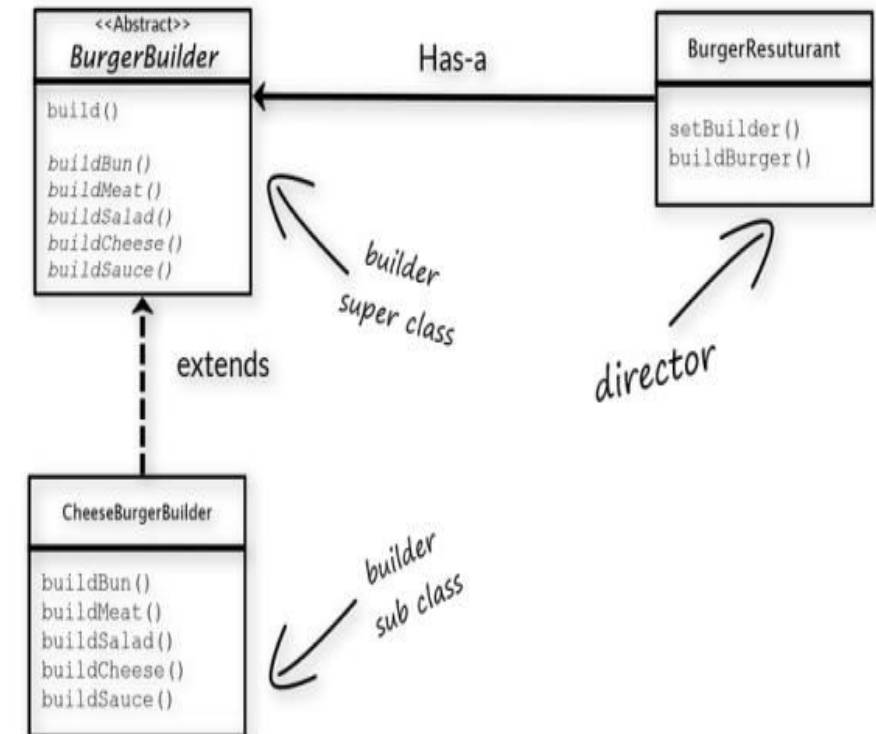
```
public class Main {  
    public static void main(String[] args) {  
        Burger cheeseBurger = new Burger();  
        cheeseBurger.setBun("White Bread");  
        cheeseBurger.setMeat("Beef");  
        cheeseBurger.setSalad("Iceberg");  
        cheeseBurger.setCheese("American Chesse");  
        cheeseBurger.setSauce("Secret Sauce");  
        cheeseBurger.print();  
    }  
}
```


- ❑ When talking about the builder design pattern, it is important to understand the concept of the Director and the Builder.
- ❑ The director's job is to invoke the building process of the builder. The builder's job is to manage the different building procedures associated with each of the different variations of objects, in this case the burgers.
- ❑ The builder pattern consists of two classes, a sub- and super class.



Design Solution :Class Diagram (UML)

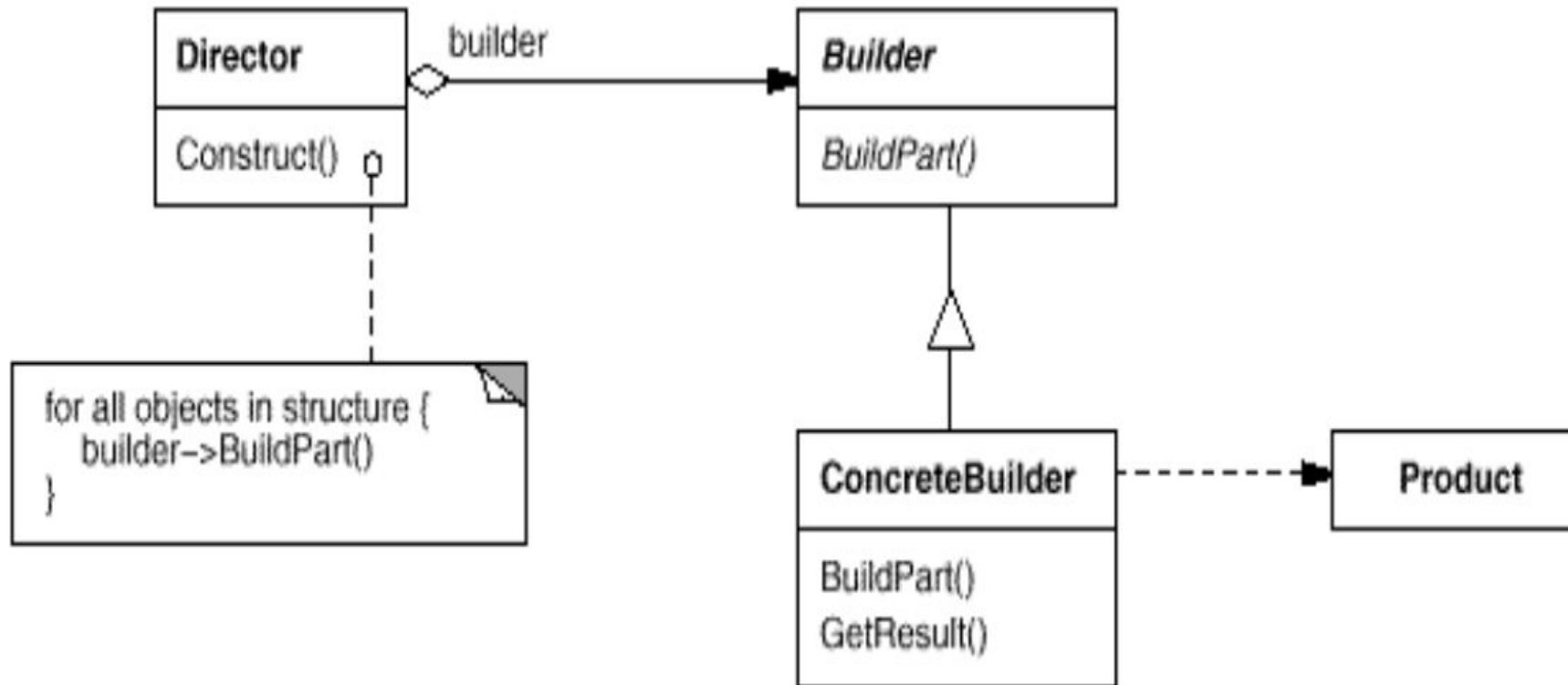
- ❑ Builder pattern consists of two main class types: the builder and the director
- ❑ With the context of program we will be using the builder pattern, it means we actually only want to be interacting with the director.
- ❑ The director will ensure that we build the correct burger object based on which builder we provide it with.
- ❑ Inside the director, we set its builder object field (setBuilder-method) and afterwards ask it to build the object based on the provided builder (build-method).



Applicability

Use the Builder pattern when

- ☐ The algorithm for creating a complex object should be independent of the parts that make up the object and how they're assembled.
- ☐ The construction process must allow different representations for the object that's constructed.
- ☐ You want to get rid of a “telescoping constructor”. Say you have a constructor with ten optional parameters. Calling such a beast is very inconvenient; therefore, you overload the constructor and create several shorter versions with fewer parameters.
- ☐ You want your code to be able to create different representations of some product (for example, stone and wooden houses). The Builder pattern can be applied when construction of various representations of the product involves similar steps that differ only in the details.



Builder (BurgerBuilder)

- o specifies an abstract interface for creating parts of a Product object.

ConcreteBuilder (CheeseBurgerBuilder, VegBurgerBuilder)

- o constructs and assembles parts of the product by implementing the Builder interface.
- o defines and keeps track of the representation it creates.
- o provides an interface for retrieving the product (e.g., getBurger()).

Director (Chef)

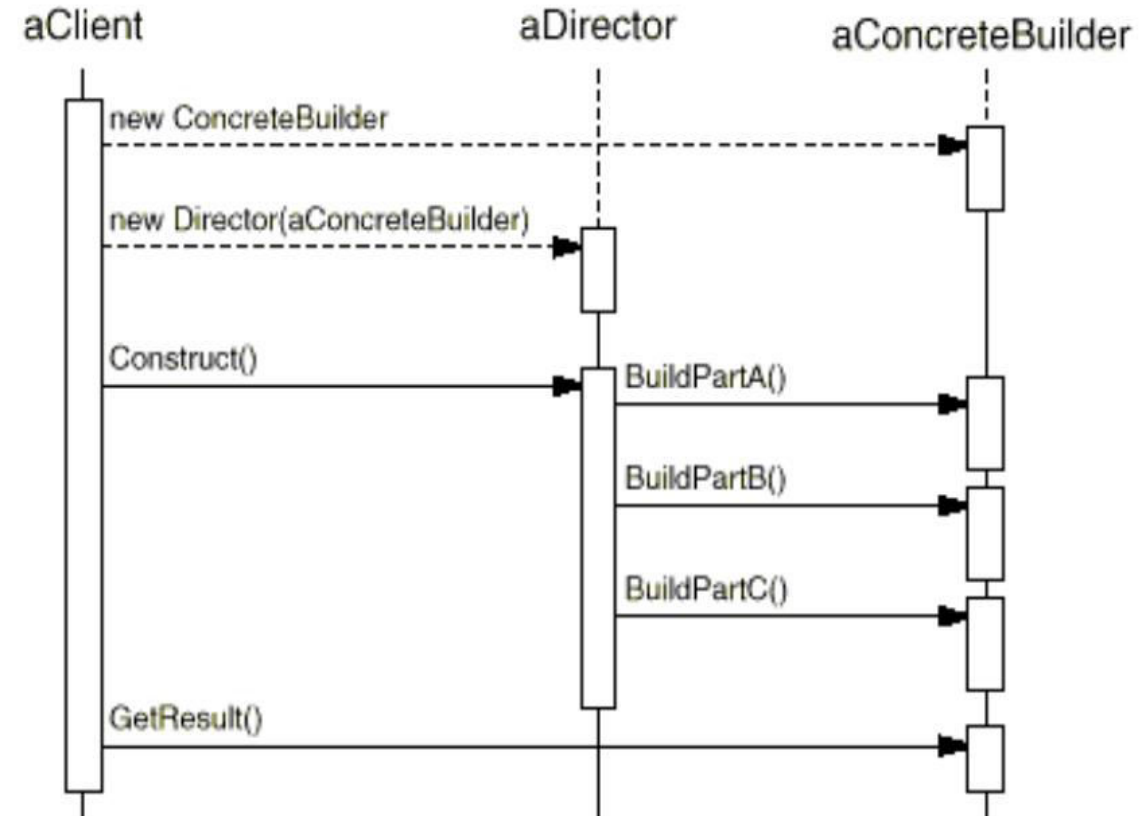
- o constructs an object using the Builder interface.

Product (Burger)

- o represents the complex object under construction. ConcreteBuilder builds the product's internal representation and defines the process by which it's assembled.
- o includes classes that define the constituent parts, including interfaces for assembling the parts into the final result.

- ❑ Client creates Director object and configures it with a Builder
- ❑ Director notifies Builder to build each part of the product
- ❑ Builder handles requests from Director and adds parts to the product
- ❑ Client retrieves product from the Builder

The following interaction diagram illustrates how Builder and Director cooperate with a client.



- ☐ Lets you vary a product's internal representation by using different Builders
- ☐ Isolates code for construction and representation
- ☐ Gives finer-grain control over the construction process

Issues to consider when using the Builder pattern

- ☐ Assembly and construction interface: generality
- ☐ Is an abstract class for all Products necessary?
- ☐ Usually products don't have a common interface
- ☐ Usually there's an abstract Builder class that defines an operation for each component that a director may ask it to create.
- ☐ These operations do nothing by default (empty, static methods)
- ☐ The ConcreteBuilder overrides operations selectively

How to Implement

1. Make sure that you can clearly define the common construction steps for building all available product representations.
1. Declare these steps in the base builder interface.
1. Create a concrete builder class for each of the product representations and implement their construction steps.
1. Think about creating a director class. It may encapsulate various ways to construct a product using the same builder object.
1. The client code creates both the builder and the director objects. Before construction starts, the client must pass a builder object to the director.
1. The construction result can be obtained directly from the director only if all products follow the same interface. Otherwise, the client should fetch the result from the builder.

Pro and Cons

Pros

- ❑ You can construct objects step-by-step, defer construction steps or run steps recursively.
- ❑ You can reuse the same construction code when building various representations of products.
- ❑ *Single Responsibility Principle*. You can isolate complex construction code from the business logic of the product.

Cons

- ❑ The overall complexity of the code increases since the pattern requires creating multiple new classes.

Relations with Other Patterns

- ❑ Many designs start by using **Factory Method** (less complicated and more customizable via subclasses) and evolve toward **Abstract Factory**, **Prototype**, or **Builder** (more flexible, but more complicated).
- ❑ **Builder** focuses on constructing complex objects step by step. **Abstract Factory** specializes in creating families of related objects. *Abstract Factory* returns the product immediately, whereas *Builder* lets you run some additional construction steps before fetching the product.
- ❑ You can use **Builder** when creating complex **Composite** trees because you can program its construction steps to work recursively.
- ❑ You can combine **Builder** with **Bridge**: the director class plays the role of the abstraction, while different builders act as implementations.
- ❑ **Abstract Factories**, **Builders** and **Prototypes** can all be implemented as **Singletons**.

DEMO

MCQ

1. In the Builder pattern, who controls the building process?

- A. Product
- B. Director
- C. Client
- D. Constructor

2. The Builder interface is responsible for:

- A. Running the application
- B. Creating parts of a product
- C. Database connectivity
- D. User interface rendering

3. ConcreteBuilder mainly:

- A. Implements building steps
- B. Stores client data
- C. Executes main method
- D. Replaces Director

MCQ

4. Which component represents the complex object under construction?

- A. Director
- B. Builder
- C. Product
- D. Client

5. One benefit of the Builder pattern is:

- A. Removes abstraction
- B. Provides finer control over construction
- C. Eliminates interfaces
- D. Avoids object creation

6. Builder pattern helps avoid:

- A. Polymorphism
- B. Telescoping constructors
- C. Inheritance
- D. Interfaces

MCQ SOLUTION

1.B

2.B

3.A

4.C

5.B

6.B

Text Reference

- Design Patterns: Elements of Reusable Object-Oriented Software, GOF

Web Reference

<https://www.baeldung.com/creational-design-patterns>

<https://www.javatpoint.com/builder-design-pattern>

https://sourcemaking.com/design_patterns/builder

<https://integu.net/builder-pattern/>

<https://www.c-sharpcorner.com/article/builder-design-pattern-with-java/>



THANK YOU

Prof Shridevi A Sawant

Department of Computer Science and Engineering

shridevias@pes.edu



Object Oriented Analysis and Design

UE23CS352B

Prof Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

UE23CS352B: Object Oriented Analysis and Design

OO Design – Creational Design Patterns

Prototype

Prof Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements:

Prof. Nivedita Kasturi

Dept. of Computer Science and Engineering

- ☐ Prototype-definition
- ✓ Motivation
- ✓ Intent
- ✓ Implementation
- ✓ Applicability
- ✓ Structure-Consequence
- ✓ Issues

Prototype : Class, Object Structural

Motivation

The Prototype Pattern specify the kind of objects to create using a prototypical instance, and create new objects by copying this prototype.

Use the Prototype Pattern when a client needs to create a set of objects that are alike or differ from each other only in terms of their state and creating an instance of a such object (e.g., using the “new” keyword) is either expensive or complicated.

The Prototype Pattern allows you to make new instances by copying existing instances.

- In Java this typically means using the clone() method or de-serialization when you need deep copies

Key aspect of this pattern:

- Client code can make new instances without knowing which specific class is being instantiated

Prototype : Class, Object Structural

Intent

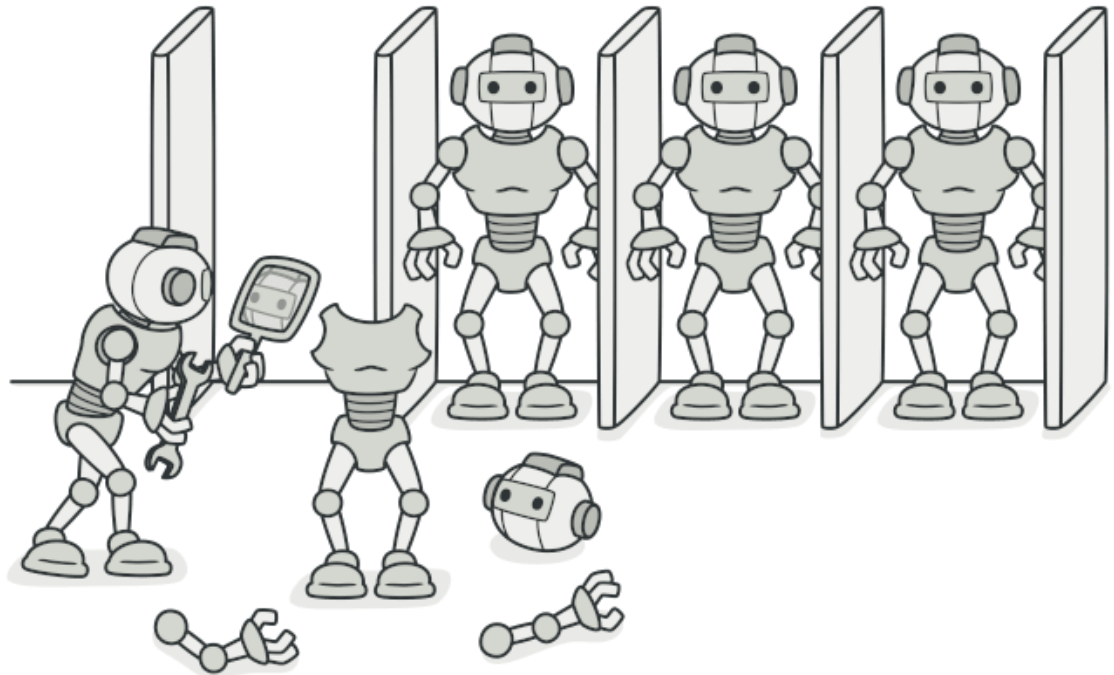
Prototype is a creational design pattern that lets you copy existing objects without making your code dependent on their classes.

Usage examples: The Prototype pattern is available in Java out of the box with a Cloneable interface.

Any class can implement this interface to become cloneable.

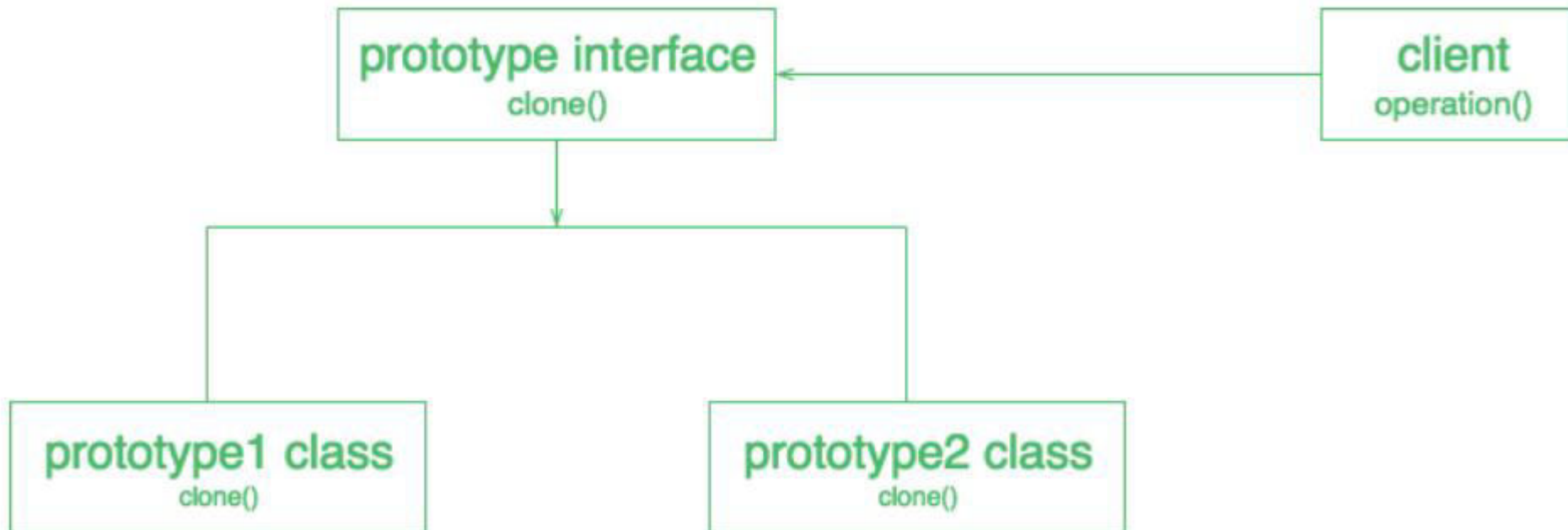
`java.lang.Object#clone()` (class should implement the `java.lang.Cloneable` interface)

Identification: The prototype can be easily recognized by a clone or copy methods, etc.



Prototype : Implementation

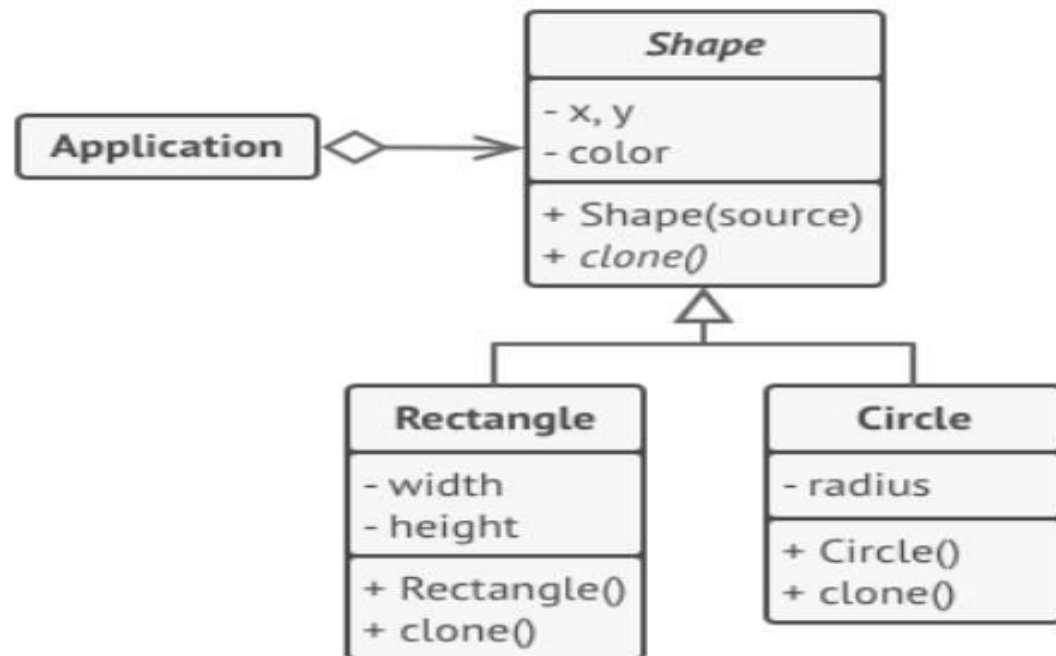
UML class diagram for the Prototype Pattern



Prototype : Implementation example-1

Problem Statement:

In this example, the **Prototype** pattern lets you produce exact copies of geometric objects, without coupling the code to their classes.



All shape classes follow the same interface, which provides a cloning method. A subclass may call the parent's cloning method before copying its own field values to the resulting object.

Cloning a set of objects that belong to a class hierarchy.

Solution

Let's take a look at how the Prototype can be implemented without the standard Cloneable interface.

DEMO

Note: Its example for prototype with different shape objects

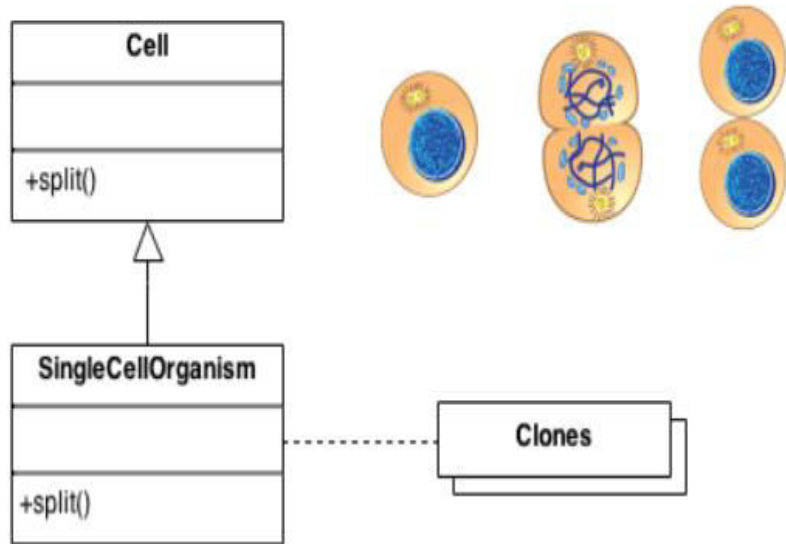
Prototype registry

You could implement a centralized prototype registry (or factory), which would contain a set of pre-defined prototype objects. This way you could retrieve new objects from the factory by passing its name or other parameters. The factory would search for an appropriate prototype, clone it and return you a copy.

Prototype : Scenario example-2

- ☐ The Prototype pattern specifies the kind of objects to create using a prototypical instance.
- ☐ Prototypes of new products are often built prior to full production.
- ☐ The mitotic division of a cell - resulting in two identical cells - is an example of a prototype that plays an active role in copying itself and thus, demonstrates the Prototype pattern.
- ☐ When a cell splits, two cells of identical genotype result. In other words, the cell clones itself.

Prototype : Scenario example-2



✓ Check list

- Add a `clone()` method to the existing "product" hierarchy.
- Design a "registry" that maintains a cache of prototypical objects. The registry could be encapsulated in a new Factory class, or in the base class of the "product" hierarchy.
- Design a factory method that: may (or may not) accept arguments, finds the correct prototype object, calls `clone()` on that object, and returns the result.
- The client replaces all references to the new operator with calls to the factory method.

Applicability

Use the Prototype pattern when

Use the Prototype pattern when a system should be independent of how its products are created, composed, and represented; and

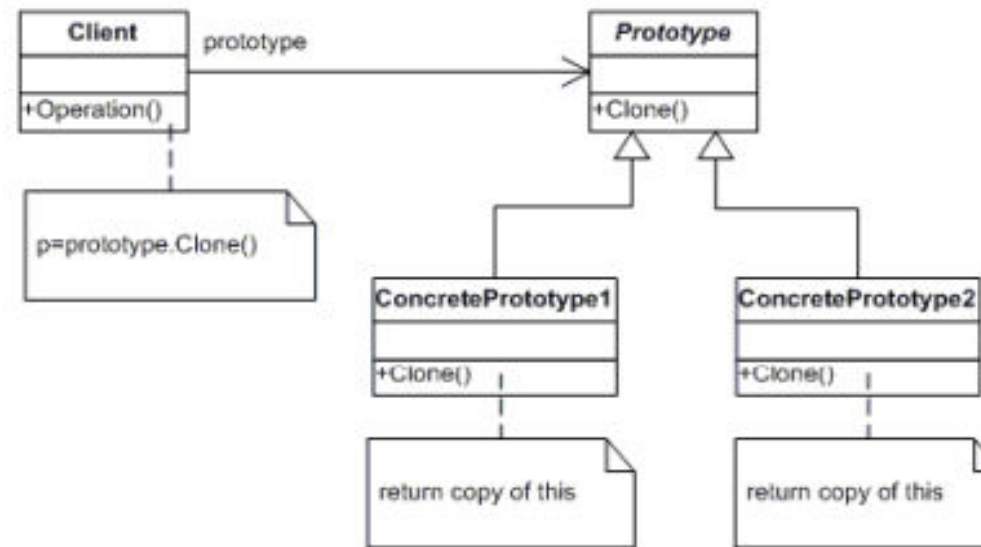
- ☐ when the classes to instantiate are specified at run-time, for example, by dynamic loading; or
- ☐ to avoid building a class hierarchy of factories that parallels the class hierarchy of products; or
- ☐ when instances of a class can have one of only a few different combinations of state. It may be more convenient to install a corresponding number of prototypes and clone them rather than instantiating the class manually, each time with the appropriate state.

Participants

Prototype: declares an interface for cloning itself.

ConcretePrototype: implements an operation for cloning itself.

Client: creates a new object by asking a prototype to clone itself and then making required modifications.



A client asks a prototype to clone itself.

Consequence

Additional benefits of the Prototype pattern are listed below.

- ☐ Adding and removing products at run-time.
- ☐ Specifying new objects by varying values
- ☐ Specifying new objects by varying structure.
- ☐ Reduced subclassing.
- ☐ Configuring an application with classes dynamically.

Issues to consider when using the Prototype pattern

Consider the following issues when implementing prototypes:

- ❑ **Using a prototype manager.** : When the number of prototypes in a system isn't fixed (that is, they can be created and destroyed dynamically), keep a registry of available prototypes. Clients won't manage prototypes themselves but will store and retrieve them from the registry. A client will ask the registry for a prototype before cloning it. We call this registry a prototype manager.
- ❑ **Implementing the Clone operation.** The hardest part of the Prototype pattern is implementing the Clone operation correctly. It's particularly tricky when object structures contain circular references.
- ❑ **Initializing clones.** While some clients are perfectly happy with the clone as is, others will want to initialize some or all of its internal state to values of their choosing. You generally can't pass these values in the Clone operation, because their number will vary between classes of prototypes. Some prototypes might need multiple initialization parameters; others won't need any. Passing parameters in the Clone operation precludes a uniform cloning interface.

Pros and Cons

- ✓ You can clone objects without coupling to their concrete classes.
- ✓ You can get rid of repeated initialization code in favor of cloning pre-built prototypes.
- ✓ You can produce complex objects more conveniently.
- ✓ You get an alternative to inheritance when dealing with configuration presets for complex objects.
- ✗ Cloning complex objects that have circular references might be very tricky.

MCQ

1. The Prototype pattern creates objects by:

- A. Using constructors repeatedly
- B. Copying existing objects
- C. Using static methods
- D. Using interfaces only

2. The main intent of the Prototype pattern is to:

- A. Hide inheritance
- B. Copy existing objects without depending on their classes
- C. Increase object creation complexity
- D. Eliminate interfaces

MCQ

3. Prototype pattern is useful when:

- A. Objects are simple
- B. Object creation is expensive or complex
- C. Only one instance is needed
- D. Classes are final

4. A prototype registry is used to:

- A. Store and retrieve prototype objects
- B. Delete objects
- C. Compile classes
- D. Execute programs

MCQ

5. In **Prototype** pattern, the client:

- A. Directly creates objects using new
- B. Requests an object to clone itself
- C. Avoids object creation
- D. Uses static classes

6. One benefit of **Prototype** pattern is:

- A. Reduced subclassing
- B. Increased subclassing
- C. Removal of interfaces
- D. Elimination of objects

MCQ SOLUTION

1.B

2.B

3.B

4.A

5.B

6.A

Text Reference

- Design Patterns: Elements of Reusable Object-Oriented Software, GOF

Web Reference

https://sourcemaking.com/design_patterns/prototype

<https://www.geeksforgeeks.org/prototype-design-pattern/>



THANK YOU

Prof Shridevi A Sawant

Department of Computer Science and Engineering

shridevias@pes.edu